

AI(Artificial Intelligence)

20/09/03 ~ 20/12/20

박지원

목차

1. 0강_인공지능 개요
2. 1-1강_Introduction
3. 1-2강_The foundations of AI
4. 1-3강_History of AI
5. 1-4강_The State of Art
6. 2-1강_Agent
7. 2-2강_Related Concepts
8. 2-3강_Task Environment
9. 3-1 ~ 3-3강_Problem Solving Agent
10. 3-4강_Uninformed Search
11. 3-5강_Informed Search
12. 3-6강_Make Heuristic Function
13. 4-1강_Local Search
14. 4-3강_Non-Deterministic Problem
15. 4-4강_No Observable Problem

16. 4-5강_Partially Observable Problem
17. 5-1강_Games
18. 5-2강_Game Algorithm
19. 5-3강_State-of-the-Art Game Programs
20. 6-1강_Constraint Satisfaction Problems
21. 6-3강_Local Search for CSP
22. 6-4강_Structure of CSP
23. 7-1강_Knowledge-Based Agents
24. 7-2강_Knowledge-Based Agents Example : The Wumpus World
25. 이산수학 : 명제
26. 7-3강_Logic
27. 7-4강_Syntax of Propositional Logic
28. 7-5강_The Resolution Inference Rule

<0강_인공지능 개요> 20/09/03

1. 정의

1) 인공지능

= 컴퓨터로 구현한 지능

= 다트머스 워크샵에서 1956년 등장했습니다.

ex) 알파고

2) 머신러닝

= 컴퓨터 프로그램이 데이터와 처리 경험을 이용한 학습을 통해 정보처리 능력을 향상 시키는 것

= 인공지능의 한 분야

2. 컴퓨터와 인공지능의 등장

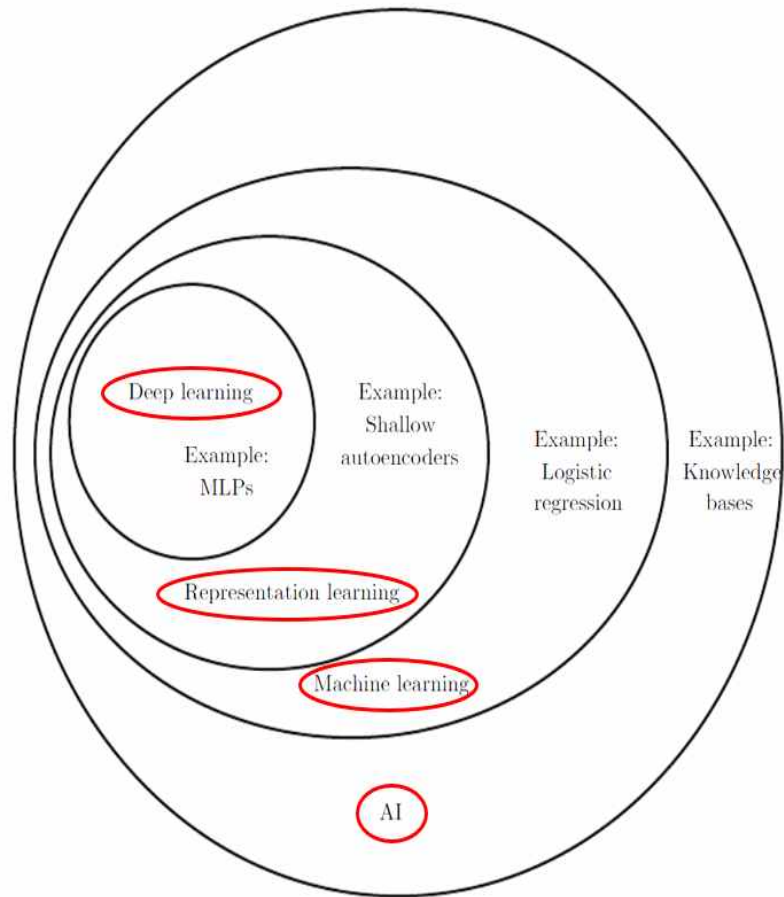
= 컴퓨터가 발명 될 때 인공지능도 사실 비슷한 시기에 발명되었습니다.

= 컴퓨터와 인간의 지능은 같지 않을까? 알 수 없습니다.

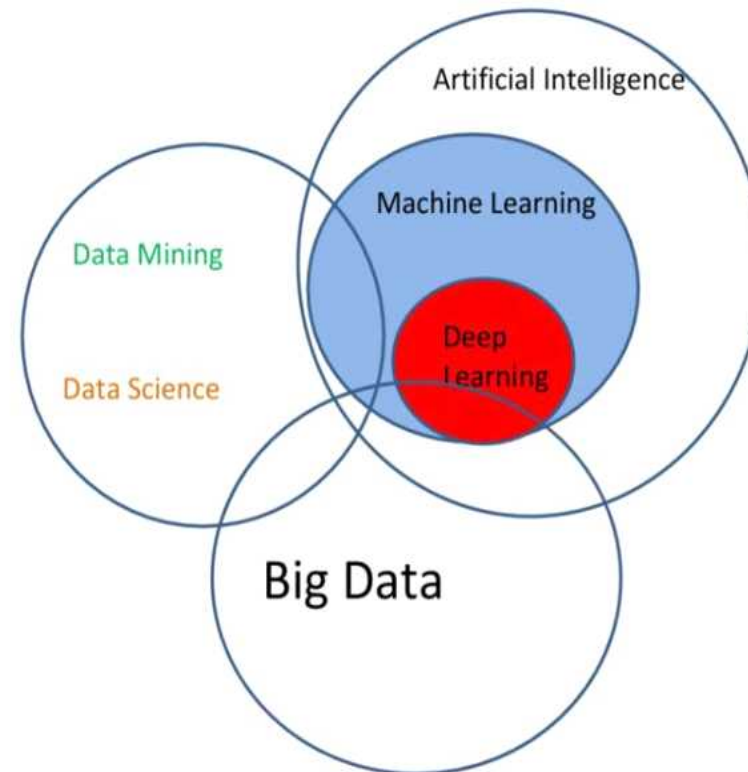
= 인공지능의 역사 및 발전과정을 설명해드립니다.

= 현 실태는 생각보다 발전이 느립니다. -> 너무 변수가 많고 어려운 학문

인공지능이라는 학문 분야의 계층도



인접 학문 분야와의 관계



Artificial Intelligence

인공지능

사고나 학습 등 인간이 가진
지적 능력을 컴퓨터를 통해
구현하는 기술



Machine Learning

머신러닝

컴퓨터가 스스로 학습하여
인공지능의 성능을
항상 시키는 기술 방법



Deep Learning

딥러닝

인간의 뉴런과 비슷한
인공신경망 방식으로
정보를 처리

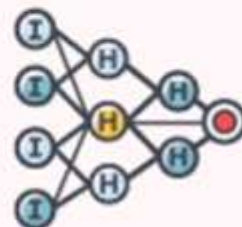
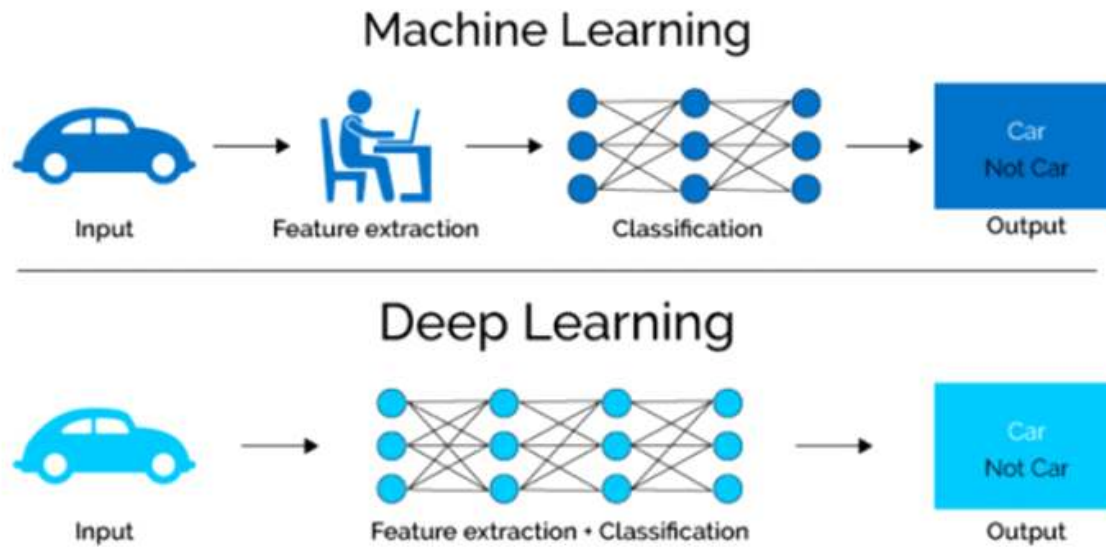


그림4 머신러닝 VS 딥러닝



자료: Towards Data Science, 메리츠증권 리서치센터

- 머신러닝은 인간이 데이터에 대한 결과 값을 미리 알려주어야 하고 목표치에 가까운 결과 값의 특징을 미리 정의해야 합니다.
- 반면에 딥러닝은 인간의 신경망인 뉴런의 작동 원리를 모방한 인공 신경망을 이용하여 복잡하고 방대한 데이터로부터 결과값을 추출하는 원리로 작동합니다.

<1-1강_Introduction> 20/09/07

1. 인공지능의 세부분야

1) 기능으로 구분

- Learning (기계학습 = machine learning)
- perception (인지)

2) 응용분야로 구분

- 체스 , 시 , 질병 , 등등

2. 인공지능이란 무엇인가

1) Acting Humanly

= 사람과 같이 행동하는 AI

- **Turing Test**를 통과하면 지능이 있다고 판단합니다. 나중에 Total Turing Test로 업그레이드 됩니다.
- 자연언어를 사용할 수 있어야 합니다.
- 내부에 지식을 갖고 있고 그 지식을 표현할 수 있어야 합니다.
- 추론 가능(Automated reasoning)
- 기계 학습(배운 지식을 토대로 학습이 가능합니다)
- 시각적 기능
- **위의 기능을 할 수 있지만 하면 되지 어떻게 만드는 지는 중요하지 않습니다.**
- 사람을 흉내 내는 방법만이 인공지능이라는 의문입니다. 사람만을 모방하지 않고 비행기를 모방한 사례(artificial flight)

2) Thinking humanly

= 사람의 감성 + 이성(mind)이 어떻게 동작하는지 연구하여 구현한 AI

- Introspection(생각 실험)
- psychological experiments(관찰 실험 & 심리)
- Brain imaging(뇌의 혈류를 관찰)
- Cognitive Science(인지 과학)와 매우 연관이 깊습니다.

3) Thinking rationally

= 사람의 감성을 배제한 이성(생각하는 원리)만을 연구하여 구현한 AI

- 명제 논리 & 생각의 법칙
- 20세기 초에 모든 객체를 논리적인 기호, 표기법으로 나타낼 수 있게 되었습니다.
- 한계 1) 자율 주행, 질병 진단은 완벽하게 논리적으로 표현하는 데에는 한계가 있습니다.
- 한계 2) 논리적으로 모두 표기할 수 있으나 실제와는 거리가 먼 것이 많습니다.

4) Acting rationally

= 사람은 이성적으로만 행동하면 문제점이 많습니다. (ex : 무조건 반사)

- Thinking rationally를 포함하는 개념입니다.
- 가장 확립되고 현대에서 가장 잘 받아들이는 개념이므로 이를 기반으로 AI를 공부합니다.

<1-2강_The foundation of AI> 09/07 ~ 09/13

- 인공지능과 관련이 깊은 학문들에서 인공지능에 관한 역사와 발전과정을 공부합니다.

1. Philosophy(철학)

- 인공지능에 무엇까지 구현해야 하는가?

1) Mind and Matter

= 정신이 인간의 어디에 있는가?

- Dualism(이중주의) = 정신세계와 물질세계도 존재합니다.
- Materialism(물질주의) = 물질세계만이 존재합니다. (현대)
- 과연 AI에는 정신세계를 포함시켜야 하는가?

2) Knowledge and Action

= 지식이 어떻게 행동으로 이어지는가?

- 아리스토텔레스의 고전적인 생각을 Newell 과 Simon이 구현해 내었습니다.

2. Mathematics(수학)

- 유효한 결론을 도출하기 위한 공식적인 룰은 무엇이 있는가?
- 어떤 것이 계산 되는가?
- 불확실한 정보를 어떻게 다루는가?

1) Logic(논리)

= Boolean logic(불 대수)

2) Computation

= 이론만으로는 참인지 거짓인지 판별할 수 없는 문제가 반드시 존재합니다.

= Computer Science의 개념으로 보면 어떤 함수는 알고리즘(튜링 머신)으로 만들 수 없다는 의미를 갖습니다.

- 튜링 머신으로 풀 수 없는 문제를 halting problem 이라고 하고 **not computable** 하다고 합니다.

① Tractability

= 어떤 문제를 풀 수 있는가를 나타내는 지표입니다.

- 알고리즘의 복잡도와 연관된 내용입니다.

= Polynomial(다항시간)은 Tractable(풀 수 있는) 하지만 Exponential(지수시간)은 Tractable하지 않습니다.

= completable한 문제도 Exponential하다면 현실적으로는 풀지 못합니다.

② NP 이론

= 'Exponential 문제를 연구하고 시간이 흐른다면 Polynomial 안에 해결할 수 있는 문제가 될 수 있는가?'에 대한 이론입니다.

- 알고리즘 분야입니다.

- 대부분의 AI 문제는 NP-complete or NP-hard 문제입니다.

- 다시 말해 **대부분의 AI 문제는 대부분 Exponential할 가능성이 매우 높기 때문에 풀 수 없을 것**입니다.

3) Probability

= 확률과 관련된 내용입니다.

3. Neuroscience(=Brain Science)(신경학)

- 뇌가 어떻게 정보를 처리하는가?
- 뇌의 어떤 영역이 정보를 처리할 것입니다.
- 뇌의 특정 지역의 혈액량을 측정하여 어떤 영역이 활성화되는지 알아냅니다.

<Brains cause minds>

= 뇌에서 우리의 이성이 나옵니다.

4. Psychology(심리학)

- 사람은 어떻게 생각하고 행동하는가?

① Cognitive psychology(인지 심리학)

= 뇌를 정보를 처리하는 장치로 생각

② Knowledge-based agent

= 2장에서 agent를 배우고 7장에서 Knowledge-based agent를 배웁니다.

③ Cognitive Science(인지 과학)

= 사람은 숫자를 8자리까지는 잘 외웁니다.

= 언어의 3가지 모델에 관한 내용입니다.

5. Computer Engineering(컴퓨터 공학)

- 초반에는 역사를 설명합니다.

① Computer Engineering이 AI에게 영향을 준 학문

= OS , Programme Language , 등등

② AI가 Computer Engineering에게 영향을 준 학문

= Time Sharing(시분할), Interactive interpreters(중간 중간 해석하는 인터프리터), Linked_List, 메모리 자동 관리

6. Control Theory and Cybernetics(제어 이론과 사이버네틱)

- AI가 어떻게 자기 자신을 제어하는가?

ex) 자동차 엔진이 뜨거워지면 냉각수를 넣는 과정도 제어 과정입니다.

7. Linguistics(언어학)

- 언어가 생각과 어떻게 관련이 있는가?

ex) 음성인식 , 기계번역

<1-3강_History of AI> 09/14 ~ 09/17

- = 인공지능의 역사
- = 인공지능의 역사가 주는 교훈이 매우 중요하므로 배웁니다.

1. AI의 탄생 (1943 ~ 1955)

- Donald Hebb = 학습과 신경망에 대한 내용
- Alan Turing = 튜링 테스트 , 머신 러닝, 유전 내용
- = Logic Therist(LT)개발 = 수학 공식 증명 , 프로그래밍 언어를 만들었으나 컴파일 하지 못했습니다.

2. AI가 인기가 초기에 인기가 많았던 시기 (1952 ~ 1969)

- = 컴퓨터가 이전에 할 수 없었던 X라는 일을 할 수 있게 되었습니다.
- = 인공지능이 기초적인 일을 수행하기 시작합니다.
- Allen Newell and Herbert Simon = 수학 공식을 증명하는 프로그램 개발 -> Think Humanly
- IBM = Checkers(체스 하위호환)를 할 수 있는 프로그램 개발 -> 아마추어 까지는 이김
- John McCarthy = Lisp(고전의 프로그래밍 언어) 개발 , Time-sharing(시분할), Resolution Algorithm
- Marvin Minsky = 최초의 신경망 컴퓨터 개발
- **Neural networks(신경망)** = 신경망을 연구하였고 , 이는 딥 러닝에 사용 되는 Neural Model의 기초가 됩니다.

3. AI의 급격한 쇠퇴 (1966 ~ 1973)

- = AI가 사람들의 기대를 만족시키지 못함을 깨닫고 인기가 점점 사라집니다. 아래는 왜 쇠퇴했는지에 대한 근거의 예시를 나열했습니다.
- Herbert Simon = 10년 안에 AI가 체스 챔피언이 될 것 이고 , 모든 수학공식의 증명을 해낼 것 이다. 하지만 40년 뒤에 AI가 체스 챔피언이 되었습니다.
- Machine Translation = 세계대전 이후 과학기술이 발달한 소련의 언어를 미국이 번역해야 했던 상황인데 이에 AI를 투입해 진행했지만 불가능했습니다.
- Combinatorial explosion = 알고리즘 복잡도에 대한 지식이 없던 시대에 지수수준의 복잡도는 해결 불가능 하다고 예측했습니다.

- The Lighthill report = 영국은 AI연구를 완전히 중단시켰습니다.
- Minsky 와 Papert가 “Perceptron” 책에서 **두 개의 Perceptron으로는 XOR 함수를 나타낼 수 없다고 했습니다.** 현대에는 다른 Perceptron 방식으로 XOR 함수를 나타낼 수 있습니다.
- Backpropagation Algorithm = 나중에 배울 매우 중요한 알고리즘

4. 상대적으로 많이 줄은 AI연구 , 그럼에도 꾸준히 연구를 한 시기 (1969 ~ 1979)

- = 일반적인 문제는 어렵고 , 가치 있고 특별한 문제로 시선을 돌렸습니다.
- DENDRAL = 특수한 분자 구조 연구 , 지수 시간의 복잡도 였으나 분자화학자들의 지식으로 복잡도를 줄였습니다.
- Expert Systems(전문가 시스템) = MYCIN(혈액 감염을 진단하는 시스템), 전문가들의 지식을 표현하는 시스템

5. AI가 산업이 된 시기 (1980 ~ 1988)

1) R1

- = 기업에서 상업적으로 사용한 첫 AI

2) DEC , DuPont

- = AI expert systems를 1000개 정도 생산
- 세계 각국에서 AI에 대한 관심이 다시 증가했습니다.(일본 , 미국 , 영국)
- AI 규모가 100배 이상 증가했으나 다시 “AI Winter”가 오며 쇠퇴합니다.

6. 뇌 신경망의 복귀 (1986 ~ 현재)

- **Back-Propagation Algorithm**이 다시 떠오릅니다. 이 알고리즘은 신경망 연구의 활성화를 가져왔고 , Deep-Learning의 시초입니다.

7. 오늘날의 인공 지능 (현재)

- 이론적으로 탄탄한 토대를 갖췄습니다.
- 실세계 문제들을 서서히 해결하고 있습니다.
- 실세계에 아무도 모르게 이미 사용되고 있습니다.

ex) 영화 추천 시스템 , 검색 엔진

- Artificial General Intelligence(인간 수준의 인공지능) 만들기를 목표로 합니다.
- Large Data Sets = 인터넷의 발달로 거대한 데이터를 얻을 수 있게 되었고 , **데이터가 많을수록 성능이 증가합니다.**

ex) 1억 개의 데이터 + 중급 알고리즘의 성능 >>>> 100만개의 데이터 + 최고급 알고리즘의 성능

<1-4강_The state of the Art> 9/15

= 2020년의 AI

1. Robotic Vehicles(자율 주행)

- 혁신적인 알고리즘 적으로는 성과는 없습니다.
- 자율 주행의 수준 단계를 5단계로 나누었는데 현재는 레벨2 입니다.

2. Speech Recognition(음성 지원)

- 인공지능 스피커(헤이 카카오)
- 아직 사람과의 대화에는 한참 못 미칩니다.

3. Game

- 알파고 , 스타크래프트
- 성공했습니다.

4. Robotics

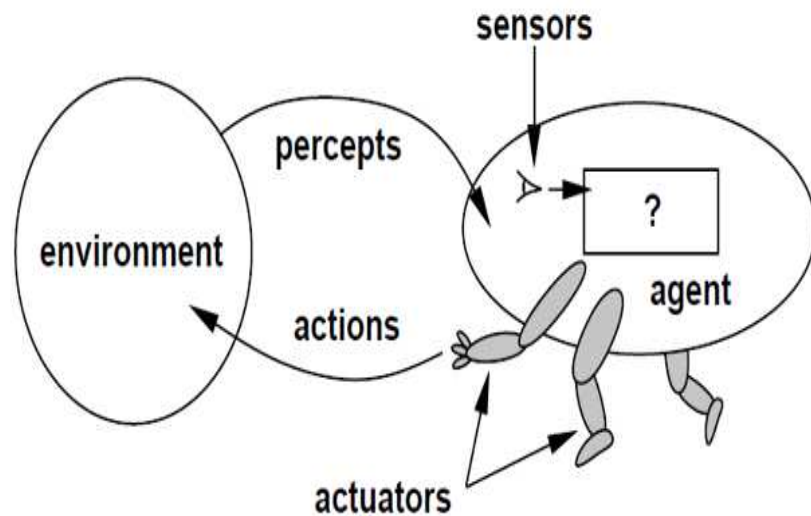
- 로봇청소기의 성능은 좋습니다.

5. Translation

- 파파고 , 구글 번역기
- 해당 언어를 전혀 모르는 사람이 번역본을 보면 유용하나 소설 , 관용구 같은 어려운 글은 이상하게 번역합니다.

<2-1강_Agent> 9/20

1. Agent의 구성요소



1) Sensor

= 감각기관이며 environment를 Input으로 입력 받는 기관입니다.

2) Actuator

= 동작기관

3) percept

= Agent가 입력 받는 상황 데이터

= 특정한 순간의 시각에 관한 지각

- Agent는 매순간 변하기 때문에 보통 Percept 보다는 **Percept Sequence(연속된 인지)** 개념을 사용합니다.

4) Action

- 입력 받은 Percept Sequence에 대한 **출력**입니다.

5) Agent Function

= **Percept Sequence를 Input으로 받고 Action을 Output으로 수행하는 함수**입니다.

- **Agent Function**은 추상적인 S/W이고 **Agent Program**은 구체적인 H/W입니다.

6) Environment

= Agent가 존재하는 곳의 환경.

- Environment에 따라 여러 가지 Percepts를 발생시킵니다.

2. Rationality(이성)

= Rational Agent가 Percept Sequence 와 사전지식이 주어졌을 때 Performance Measure를 Maximize 하는 액션을 선택하는 것을 의미합니다.

A rational agent selects an action that is **expected to maximize its performance measure**, given the percept sequence and its built-in knowledge

- = 옳고 그름을 가름을 가릴 수 있는 능력
- 감성(감정)과 반대되는 개념

1) Right-Thing

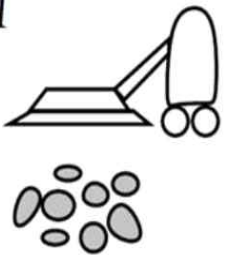
- = Action으로 State(환경)가 바람직한 방향으로 변화하는 것을 의미합니다. 이를 평가하기 위해서는 다양한 평가기준이 필요합니다.
- **Environment-States(환경의 상태)**가 얼마나 올바르게 변했는지에 대한 내용입니다.
- Rational-action의 결과이므로 rational-action과 비슷하게 묶어서 생각해도 괜찮습니다.

2) Rationality를 평가하기 위해 사전에 정해져야 할 개념

- Performance measure(성능 기준) : Agent가 얼마나 성공적으로 일을 처리했는지.
- Prior knowledge of the environment(자신이 있는 환경에 대한 사전지식)
- Actions
- Percept sequence

3. <EXAMPLE> 간단한 가상 세계의 Agent인 진공청소기

*** 2강 ~ 4강 까지 자주 사용할 예제입니다. ***

<i>A</i> 	<i>B</i> 	✓ <table><tr><th>Percept sequence</th><th>Action</th></tr><tr><td><i>[A, Clean]</i></td><td><i>Right</i></td></tr><tr><td><i>[A, Dirty]</i></td><td><i>Suck</i></td></tr><tr><td><i>[B, Clean]</i></td><td><i>Left</i></td></tr><tr><td><i>[B, Dirty]</i></td><td><i>Suck</i></td></tr><tr><td><i>[A, Clean], [A, Clean]</i></td><td><i>Right</i></td></tr><tr><td><i>[A, Clean], [A, Dirty]</i></td><td><i>Suck</i></td></tr><tr><td>⋮</td><td>⋮</td></tr></table>	Percept sequence	Action	<i>[A, Clean]</i>	<i>Right</i>	<i>[A, Dirty]</i>	<i>Suck</i>	<i>[B, Clean]</i>	<i>Left</i>	<i>[B, Dirty]</i>	<i>Suck</i>	<i>[A, Clean], [A, Clean]</i>	<i>Right</i>	<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>	⋮	⋮
Percept sequence	Action																	
<i>[A, Clean]</i>	<i>Right</i>																	
<i>[A, Dirty]</i>	<i>Suck</i>																	
<i>[B, Clean]</i>	<i>Left</i>																	
<i>[B, Dirty]</i>	<i>Suck</i>																	
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>																	
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>																	
⋮	⋮																	

<상황>

- Room_A와 Room_B가 존재합니다.
- Sensor를 통해 Agent의 위치와 먼지의 위치를 알 수 있습니다.
- Room_A에 있으면 Room_B에 먼지의 유무를 알 수 없습니다.
- 진공청소기는 이동 , 먼지 흡입 , 행동X 등의 Action을 합니다.

<Simulation>

- [A,Clean]은 하나의 time_step입니다. [A,Clean] , [A,clean]은 2개의 time_step을 갖는 Percept Sequence입니다.
- 평가 척도 = 진공청소기가 먼지를 흡입한 양 , 깨끗한 방의 개수
- 평가 척도는 매우 신중하게 정해야 합니다.
- performance measure = time_step 1부터 1000까지 깨끗한 방의 개수를 통해 점수를 줍니다.(최대 2000점)
- 사전 지식 : Room_A와 Room_B의 깨끗한 정도는 알지 못하고, 청소를 완료하면 그 방은 완전히 깨끗한 상태(먼지X)가 됩니다.
- 진공청소기는 현재 평가 기준에서는 rational 하지만 , 성능 평가 기준을 행동을 취할 때 마다 점수를 깎는 형태로 바꾸면 두 방이 모두 깨끗해도 왔다 갔다 이동하므로 변경된 평가기준이면 성능이 떨어지게 됩니다.

<2-2강_Related Concepts> 9/20

= 중요한 개념 3가지를 배웁니다.

1. Omniscience

= 전지전능의 **전지** = Agent가 완전히 모든 것을 알고 있는 상태입니다.

- 전지하다면 performance를 최대로 할 수 있습니다.

- Omniscience는 **현실로 불가능 한 가상의 개념**입니다.

ex) 미국에서 횡단보도 건너편에 친구를 만났다고 가정 합니다. rational한 사고라면 건너기 위해 좌우를 살피지만 하늘은 살피지 않습니다. 그러므로 극히 낮은 확률로 비행기에서 떨어지는 물체를 맞고 죽을 수 있습니다. 그러나 omniscience라 하면 하늘조차도 살피기 때문에 죽지 않습니다.

2. Learning

= Rationality와 연관이 큰 개념입니다.

3. Autonomy

= Rationality와 연관이 큰 개념입니다.

= adaptive(적응)와 같은 의미로 봅니다.

<2-3강_Task Environment> 9/20

<Task Environment>

= Agent가 해결해야 하는 문제.(Agent를 개발하는 근본적 이유 같습니다.)

- 4가지 구성 요소가 정해지면 Agent가 해결해야 하는 1개의 Task Environment가 정해집니다.

ex) 알파고와 이세돌 선수의 대전 , 자율 주행 택시 , 체스 , 포커 , 자동화 공정에서 불량품 거르는 로봇.

1. Task Environment의 4가지 구성요소

- Performance Measure
- Environment
- Actuators
- Sensors

<ex) Automated taxi>

- 자동화 택시의 Agent인 택시 운전사를 예시로 4가지 구성요소를 설명합니다.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

- Performance Measure = 안전성 , 속도 , 준법 , 편안함
- Environment = 길 , 다른 자동차 , 보행자 , 고객
- Actuators = 핸들링 , 가속 페달 , 경적 , 손님과 이야기
- Sensors = GPS , 카메라

2. Task Environment의 특성

1) Fully Observable(완전한 관찰) vs Partially Observable(부분 관찰)

= Sensor가 모든 측면을 탐지하면 Fully Observable 하고 부분만을 탐지하면 Partially Observable 합니다.

ex) 바둑은 Fully Observable , 포커는 Partially Observable 합니다.

2) Single agent vs Multi agent

= Agent의 개수입니다.

ex) 크로스 퍼즐은 1명 , 체스는 2명의 Agent로 구성된 게임입니다.

3) Deterministic(결정적) vs stochastic(확률적)

= Environment의 다음 State가 Environment의 최근 상태와 Agent의 행동으로 100% 결정되면 Deterministic 합니다. 100%가 아니라면 Stochastic 합니다.

ex) Vacuum 은 Deterministic , Monopoly = Stochastic 합니다.

4) Sequential vs Episodic

= Agent의 Task Environment를 여러 개의 episode로 쪼갰을 때 , episode들 끼리 연관되어 있으면 Sequential , episode들 끼리 독립적이면 Episodic 합니다.

= 체스는 Sequential 하고 공장 불량 검사는 Episodic 합니다.

5) Static vs Dynamic

= Agent가 계산하는 동안 다른 오브젝트가 기다려 주면 Static 하고 기다리지 않고 독립적으로 행동을 하면 Dynamic 합니다.

= Chess는 내 턴에 상대가 기다려주므로 static하고 Automated-taxi는 기사가 생각하는 동안에도 다른 오브젝트 들은 움직이므로 Dynamic합니다.

6) Discrete(별개의) vs Continuous

= Percepts 와 Actions 에서 받거나 할 수 있는 경우가 고정된 개수라면 Discrete 고정되지 않고 연속된 개수라면 Continuous 합니다.

ex) 장기에서 졸병은 앞으로 가거나 옆으로 가는 경우만 가능하지만 , 자동화 택시에서 택시의 속도는 연속된 실수의 값 입니다.

7) Known vs Unknown

= Agent가 주변 상황을 알면 known 모르면 Unknown 합니다.

<2-4강_Agent Structure> 9/25

- 다양한 Agent Structure 들이 어떤 구조로 구성되어 있는지 알아보시다.
- 단순한 프로그램(vacuum , automated taxi)도 엄청난 양의 메모리를 요구합니다. Agent는 어떠한 Percept에도 오류 없이 rational한 Action을 취해야 합니다.
- 당연한 이야기 이지만 Agent가 Rational 하게 일을 수행하려면 **State에 대한 Condition , Rule , Action**을 잘 만들어야 합니다.

1. Simple Reflex Agents

- = **Current percept에만 의존하고 , 기존의 percept history(지금까지 Agent가 입력 받은 Percepts들의 모임)는 무시합니다.**
- 어떠한 경우나 어마어마한 function 이나 조건이 필요한 것은 아닙니다. 간단하거나 자명한 내용들은 간단한 함수로도 구현이 가능합니다.
- 현재의 Percept를 입력 받고 Action을 리턴 하는 함수가 필요하고 , 함수 바디에는 state에 따라 rule(조건문)을 참고하여 Action을 수행하는 코드를 구현합니다.
- Partially Observable 상황에서는 Current percept로 내 state를 정확하게 파악을 못하는 상황이 반드시 존재합니다. (한계)
ex) 진공청소기의 Sensor가 고장 나서 전체 상황을 알 수 없을 때

<Example : 진공청소기>

방이 더럽다(state) -> 더러 우면 청소해야 한다.(rule = condition) -> 나는 청소한다.(Action)

```
if status = Dirty then return Suck
```

2. Model-based Reflex Agents

- = Simple Reflex Agents에 Agent가 존재하는 **세상의 정보(세상이 어떻게 돌아가는지 , 내 행동이 세상에 어떤 영향을 주는지)(=model)**를 추가합니다.
- 진공청소기의 위치 감지 Sensor가 고장 나도 진공청소기의 초기 위치를 안다면 진공청소기의 작동에 문제가 없습니다. 왜냐하면 초기 위치를 알면 right , left의 행동으로 다음의 위치도 알 수 있습니다.

3. Goal-based Agents (ex Automated-Taxi)

= 목표를 설정하여 목표를 성공(binary-select : success or failure)하는 방향으로 agent가 수행하도록 합니다.

- Automated taxi를 Model-based Reflex Agent를 통해 구현하려면 너무 복잡해집니다. 왜냐하면 손님이 변경 될 때 마다 목적지가 다르므로 condition , rule , action을 계속 다시 설정해 주어야하기 때문입니다. 하지만 Goal-based Agents는 목적지(Goal)가 바뀌더라도 Goal만 수정하고 condition , rule , action은 유지하여 이들이 Goal에 맞게 자동으로 적용되도록 합니다.

- more flexible

4. Utility-based Agents (ex Automated-Taxi)

= Goal-based Agents를 업그레이드 한 방식이며 , Goal을 성공하는 방식을 binary로 생각하지 않고 다양한 방법으로 성공함을 고려합니다.

- 목표를 달성하는 방법은 매우 다양합니다. 기존에는 '목적지에 도착 했는가 / 도착하지 못 했는가' 로만 나누었다면 이 방식은 Utility를 추가 했습니다.

ex) 법을 지켜서 목적지에 도착 했는가 / 돈을 많이 벌 수 있는 코스로 운전해서 도착 했는가

5. Learning Agents(머신러닝) -> 여기서 적은 내용은 극도로 추상적이고 실제는 엄청 복잡합니다.

1) Critic(어머니한테 훈수 두는 할아버지)

= Learning element에게 Performance Standard를 기준으로 agent가 행동한 내용이 positive 인지 negative인지 Learning element에게 조언을 해줍니다.

2) Learning element(자식을 가르치는 어머니)

= Performance Element로부터 Knowledge(condition , rule , action , model)를 받아서 그걸 어떻게 바꿀지(Change)를 계산하고 Performance Element 에게 알려줍니다.

ex) 학습되지 않은 Agent의 판단을 학습된 Learning element가 더 좋은 판단으로 바꿔줍니다.

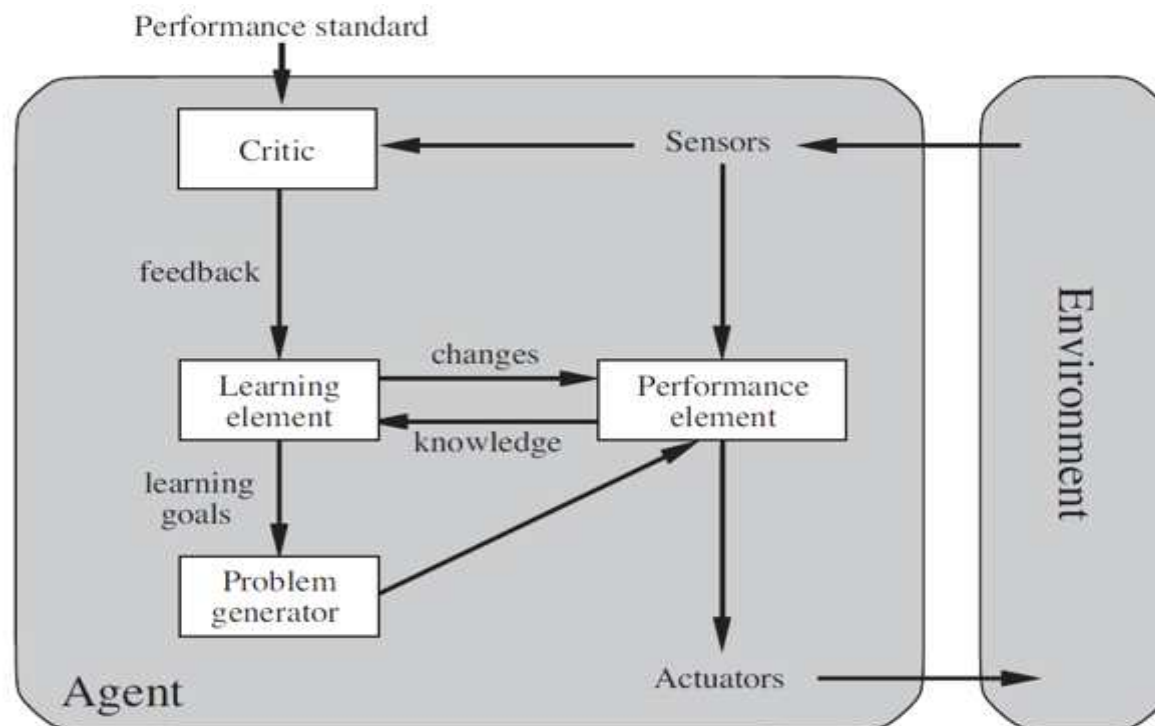
3) Problem generator(자식에게 시험을 내리는 아버지)

= Agent가 학습을 잘 할 수 있도록 어떤 액션을 테스트 해보도록 Agent에게 시킵니다.

ex) Automated-Taxi가 자동차 바퀴를 바꿨습니다. 비가 왔을 때 자동차 바퀴가 잘 작동되는지 실험해 보도록 Agent에게 명령합니다.

4) Performance element(자식)

= 기존의 Agent(=일반적인 Agent)



<3-1 ~ 3-3강_Problem Solving Agent> 9/27 ~ 9/29

1. Problem Solving(PS) Overview

= Agent가 문제를 어떻게 해결하는지에 대한 기본 구조입니다.

- 이번 강의에서는 future percepts는 고려하지 않습니다.

- 이번 강의의 Agent는 Goal-based Agent 입니다.

- **Problem Formulate(문제를 정확하게 정의) -> Solution Search(해결법을 탐색하여 최적의 해결법 찾기) -> Execute(실행) -> Goal !!!**

ex) State_A -> State_B -> State_C -> Goal

ex) 대한민국 로드맵 문제는 서울에서 부산으로 어떻게 효율적으로 갈까에 대한 문제입니다.

ex) TSP(외판원) , VLSI(스마트폰 칩) , Robot , Automatic Assembly Sequencing

2. 용어(자료구조에서 사용)

1) Node

= Agent의 state , parent , action , path-cost를 저장합니다.

- 여러 개의 노드에 같은 state가 들어갈 수 있습니다.

2) Expand

= 선택한 노드에서 관계가 있는 자식 노드들을 생성(확장)합니다.

- **Expand 하여 생성된 자식들은 모두 Frontier로 Push 됩니다.**

- Choose 단계를 포함시키지 않습니다.

3) Choose

= Expand 과정에서 Frontier에는 많은 노드들이 저장되어 있습니다. 그 중에서 노드를 골라 Current_Node로 만들고 Frontier에서 Pop합니다.

4) Frontier

= 다음에 검사할 수 있는 노드들을 저장하는 자료구조.

ex) Queue , Stack , Priority Queue

5) Explored set

= 이미 지난 노드(Expand된 노드)들을 저장하는 자료구조.

ex) Hash table

3. 용어(PS에서 사용)

1) Goal

= Problem Solving의 최종 도착점.

- Goal도 하나의 State 입니다.

2) State

= Agent의 상태입니다.

- 상태는 유한개가 있을 수도 있고 무한개가 있을 수도 있습니다.

- 트리나 그래프의 자료구조에서는 노드 안에 저장됩니다.

- 자료구조 개념에서 광범위한 범위로 생각하면 state = node로 생각해도 되지 않을까?

3) Action

ex) 루마니아 문제에서는 ‘차를 타고 이동합니다.’ 정도로 생각합니다. 자세하게 ‘차의 핸들을 12.5도 꺾어서 80km로 운행합니다.’는 추후에 다룹니다.

4) Task Environment

= Fully-observable , Discrete , Known , Deterministic 같은 것을 정의합니다. (2-3강)

ex) 부산까지 가는 데에 필요한 모든 정보를 알 수 있습니다. 도시 간의 이동으로 구별하므로 연속적인 시간이 아닙니다. 지도가 있으므로 정보를 알고 있습니다. 서울에서 안양으로 이동하면 그 후 상황을 100% 확신할 수 있습니다.

5) Solution

ex) 서울에서 차를 타고 대전을 지나고 울산을 지나서 부산에 도착합니다.

4. Define Problem (시험에서 틀렸습니다.)

= 문제를 정확하게 정의하는 것은 매우 중요합니다.

- 문제를 정의할 때 반드시 5개의 개념은 정의를 해야 합니다.

- puzzle 과 8-Queens Problem 와 Toy Problem with Infinite State Space 같은 어려운 예제도 설명해주셨지만 가장 쉬운 예시인 (방이 2개 , 먼지의 유무 , Agent의 위치)진공청소기를 통해 예시를 설명하겠습니다.

1) Initial State

= 초기 상태

ex) 4*2개의 초기상태가 가능 합니다. (각각의 방의 먼지의 유무 * Agent의 위치)

2) Action

= 행동

ex) 이동 , 먼지 흡입

3) Transition Model (Transition : 변화 , 전이를 의미)

= Agent의 현재 상태가 State_s이고 Action a를 했을 때 State_s'으로 변하게 됩니다.

ex) Agent가 왼쪽 방에 있는 상태 일 때 왼쪽으로 이동하는 Action을 실행하면 벽에 박는 상태로 변합니다.

4) Goal test

= Goal 인지 Goal이 아닌지

ex) 방 2개가 모두 깨끗하면 Goal을 달성한 것 입니다.

5) Path Cost

= 골을 달성하는데 드는 비용

ex) 청소마다 드는 전기비용

5. Solution(ex : Search Algorithm)

= 두 가지 방식의 Search Algorithm으로 해결합니다.

1) Tree Search Algorithm

- Root = Initial state -> Depth가 0 입니다.

- Branch = Action

- Node = State

① Initial Node를 Current Node로 설정합니다.

② Current Node가 Goal인지 검사하고 만약 Goal이라면 해결한 것 이므로 알고리즘을 종료합니다.

③ Current Node에서 가능한 다음 노트(상태)를 Frontier 자료구조에 Push합니다. (=Expand)

④ Frontier 자료구조에서 하나의 노드를 골라서 Frontier에서 Pop합니다.

⑤ 고른 노드가 Current Node가 되며 ② ~ ④를 반복합니다.

<언제까지 반복 하는가?>

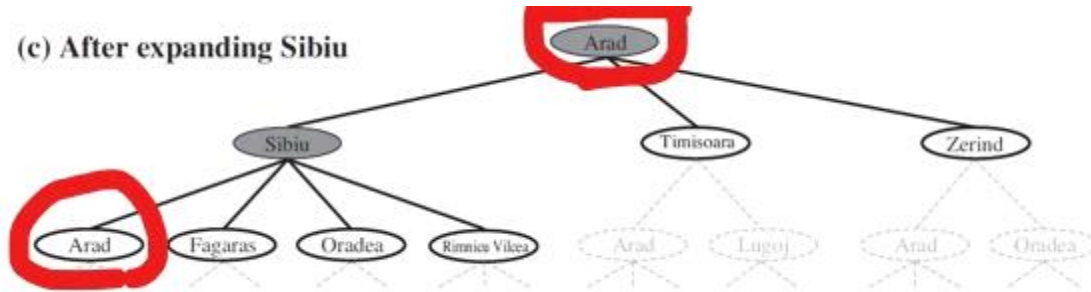
- 더 이상 Expand 할 Node가 없을 때 까지. = Frontier가 Empty일 때 까지.

- Goal인 Node를 찾을 때 까지.

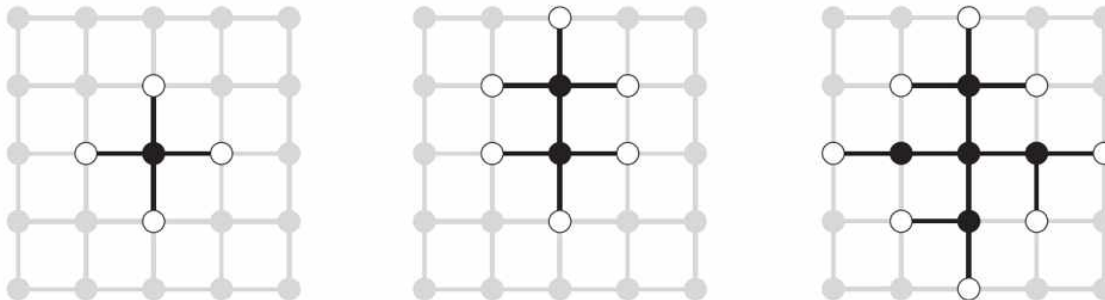
<단점>

- Loop(=Redundant Path)를 만들 수 있습니다.

ex1) 서울의 자식 노드가 대전인데 대전의 자식 노드가 서울이면 Loop가 발생하여 Optimal 하지 않는 결과가 나옵니다.



ex2) Rectangular Grid(격자에서 탐색) -> 자식의 개수는 4^{depth} 이므로 매우 많지만 실제로는 자식이 중복되므로 $2^d \times 2^d$ 입니다.



2) Graph Search Algorithm

= 한번 방문한 노드는 방문하지 않도록 원천 봉쇄 하여 탐색하는 방법입니다.

- Tree Search Algorithm의 단점을 보완합니다.

- Explored set을 사용합니다.

- Tree search와 모든 방법이 동일하지만 2가지 차이가 있습니다.

 - ① 탐색전에 빈 Explored set을 선언해줍니다.

 - ② 방문한 노드를 Explored Set에 Add 해줍니다.

- Current State -> Frontier에 다음에 가능한 Node를 모두 넣습니다. -> Explored Set에 중복되는 Node를 넣습니다. (이 순서는 반드시 지킵니다.)

5. Evaluation of Search Algorithm

= Solution의 기법 중 하나인 Search Algorithm을 평가하는 방법입니다.

1) Completeness

= Solution이 있는 문제를 해당 Search Algorithm으로 Solution을 찾아낼 수 있는지를 평가합니다.

2) Optimality

= 여러 개의 Solution 중에 가장 최적의 Solution으로 문제를 해결했는지 평가합니다.

3) Time Complexity

= 시간복잡도이며 , 알고리즘이 생성하는 노드의 개수에 비례합니다.

- 최악을 고려하므로 생성하는 노드는 항상 해당 조건에서 최대로 생성하게 됩니다.

4) Space Complexity

= 공간복잡도이며 , 알고리즘이 진행 되는 동안 가장 메모리가 많이 필요한 순간에 비례합니다.

ex) '알고리즘 시작 -> 3KB -> 6.5KB -> 8KB -> 4KB -> 1KB -> 5KB -> 알고리즘 종료' 라면 8KB에 비례합니다.

<3-4강_Uninformed Search> 9/30 ~ 10/6

*** Search Algorithm 중에서 Uninformed Search를 공부합니다. ***

<Uninformed Search>

= Blind Search 라고도 불립니다.

= 문제에서 주어진 정보 이외에 다른 정보를 갖지 않는 탐색

ex) BFS , DFS , Uniform-cost search , Depth-limited search , Iterative deepening depth-first search , Bidirectional search

<b , d , m>

① b = Branching Factor = Agent가 입력 받은 노드의 개수 = Agent가 현재 선택할 수 있는 노드의 개수

② d = State Space 에서 가장 얕은 깊이의 Goal이 갖는 깊이

③ m = State Space 에서 가장 넓은 너비의 Goal이 갖는 너비

- m은 d보다 클 수 있습니다.

④ State Space = 노드와 간선으로 만든 트리 구조

<Cost>

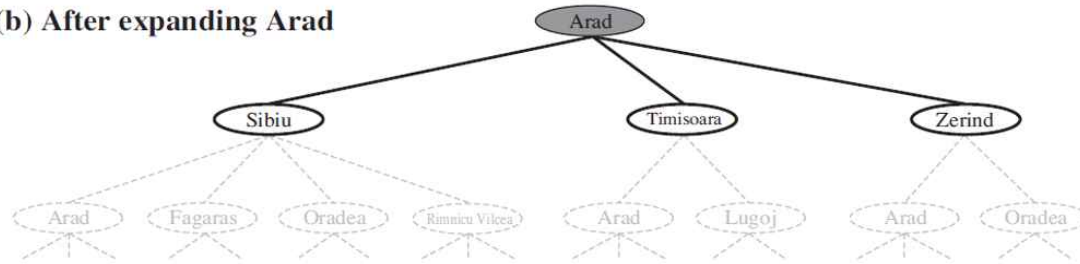
= 어떤 노드에서 다른 노드로 가는데 드는 비용 (ex : 거리 , 시간 , 돈)

- Edge에 올 수 있는 값을 모두 포괄하는 개념입니다.

1. 발전된 버전의 BFS

1) BFS Overview

(b) After expanding Arad



- = 루트 노드 Expand -> 3개의 자식 노드 생성 -> Sibiu 탐색 -> Timisoara 탐색 -> Zerind 탐색 -> Sibiu의 자식인 Arad 탐색
- = Frontier를 Queue 자료구조로 구현합니다.
- = 중복을 검사하는 Explored set을 선언합니다.
- = Frontier 에서 노드를 꺼내고 꺼낸 노드의 자식들을 검사하고 Frontier에 넣습니다.

<기존의 BFS와 다른 2가지>

① 루트 노드도 Goal 인지 검사합니다.

② 기존의 버전에서는 노드를 Frontier에서 꺼낼 때 골인지 검사를 합니다. 하지만 발전된 버전에서는 **꺼낸 노드의 자식을 Frontier에 넣을 때 explored set 과 Frontier에 존재하지 않는지를 검사(중복검사)하고 골인지 검사합니다.** 둘을 비교하면 Frontier에 넣을 때 검사를 한다면 Frontier에서 뺄 때 검사를 하는 것 보다 시간복잡도가 깊이 1개만큼 줄어 듭니다. $b^{(d+1)}$ 에서 b^d 로 줄어듭니다.

2) 평가

① Complete

= Branching Factor가 유한하다면 Complete 합니다.

② Optimal

- BFS는 기본적으로 다수의 Goal 중 가장 얇은 Depth의 Goal을 찾게 설계 된 알고리즘 입니다.
- 그러므로 Goal이 2개 이상 존재할 때 Depth와 Path-cost(경로비용)가 비례하면 가장 얇은 Depth의 Goal이 Optimal 하게 됩니다.
- 하지만 만약 Depth와 경로 비용이 비례하지 않으면 가장 얇은 Goal을 찾아도 그것은 Optimal 하지 않습니다.

6. Breadth-first search가 optimal할 핵심 조건을 기술하시오. (5점)

- Node(혹은 goal)의 depth와 path cost가 비례
- Path cost가 node(혹은 goal)의 depth에 대한 비감소(증가) 함수
- Optimal solution이 가장 얇은 depth를 가짐

③ Time Complexity

- = depth가 1일 때 b(노드 개수)가 10이면 depth가 2일 때 $10^2 + 10$ 이고 depth가 3일 때는 $10^3 + 10^2 + 10$ 이 됩니다.
- = $O(b^1 + b^2 + b^3 + \dots + b^d) = O(b^d)$
- 지수 수준에서는 자신 보다 낮은 차수는 무시 됩니다.

④ Space Complexity

- = $O(b^d)$
- Frontier 와 Explored Set에 저장되는 노드들은 메모리에 올라갑니다.
- Tree Search는 Explored Set이 없으므로 메모리를 적게 쓰지만 , 생각보다 둘의 차이는 크지 않습니다.

<Example>

- depth가 1일 때의 노드 개수가 10개 입니다.
- Depth가 10이면 3시간이 걸리고 10 테라바이트의 메모리가 필요합니다. -> 일반 컴퓨터 불가능
- Depth가 16이면 350년이 걸립니다. 10 엑사바이트의 메모리가 필요합니다. -> 현실에서 불가능
- Depth가 커지면 현실세계에서 해결하기가 어려워집니다.

2. Uniform-Cost Search

1) Uniform-Cost Search Overview

= BFS는 Depth와 Path-cost가 비례하지 않는다면 Optimal 하지 않는다는 문제를 해결하기 위해 만들어진 알고리즘입니다.

2) 발전된 BFS와 다른 부분

① 시간복잡도를 줄이는 이점을 포기하고 Optimality를 위해 다시 꺼낼 때 Goal 인지를 검사합니다.

② Goal을 찾았고 Goal을 Frontier에 넣습니다. 발전된 BFS처럼 넣을 때 Goal 인지를 검사하면 여기서 종료가 됩니다. 하지만 UCS는 Pop할 때 Goal을 검사하므로 Goal을 Pop할 때 까지 계속 진행 합니다. 그러다가 Frontier의 다음 노드들을 진행 하던 도중 그의 자식이 더 Optimal 한 Goal이라면 Frontier에 이미 Goal 이 있음에도 불구하고 새로 찾은 Goal을 Goal로 Replace 합니다.

3) 평가

① Complete

= Step Cost가 모두 양수라면 Complete 합니다.

② Optimal

= Yes

③ Time Complexity

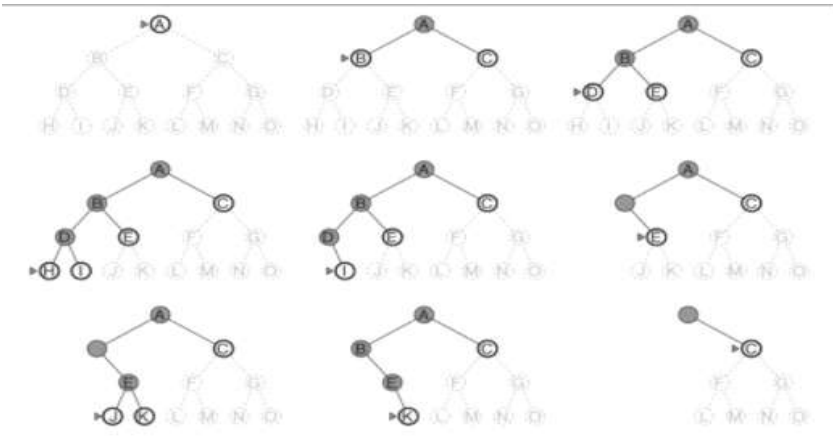
= depth와 상관없는 새로운 공식인데 너무 어려워서 넘어갑니다. BFS보다는 증가합니다.

④ Space Complexity

= depth와 상관없는 새로운 공식인데 너무 어려워서 넘어갑니다. BFS보다는 증가합니다.

3. DFS

1) DFS Overview



- A가 루트 -> A를 Expand 하여 B와 C를 frontier로 넣습니다. -> B를 Expand 하여 D와 E를 frontier로 넣습니다. -> D를 Expand하여 H와 I를 frontier로 넣습니다. -> 똑같이 진행 합니다.
- Frontier를 Stack으로 사용합니다.
- Stack을 사용하는 알고리즘은 **재귀함수**로 구현합니다.

2) 평가

① Complete

= Tree Search 라면 어떠한 경우여도 No , Graph Search 이고 State Space 가 유한하다면 Yes

ex) 루마니아 경로를 Tree Search로 생각해봅시다. Arad가 루트이고 자식이 Sibiu입니다. 근데 Sibiu의 자식이 Arad라면 무한히 반복하고 Goal을 찾을 수 없습니다.

ex) BFS는 State Space가 무한하여도 얕은 노드부터 연속적으로 검사하여 언젠가는 Goal을 찾을 것 입니다. 하지만 DFS는 계속 깊이 내려만 가 불연속 적으로 검사하므로 , Goal이 Depth가 2인 Node에 존재하여도 찾지 못할 수 있습니다. -> 연속성과 불연속성을 생각합시다.

② Optimal

= 루트 기준 왼쪽에서 Depth가 4일 때 Goal 이 있고 , 오른쪽에서 Depth가 2일 때 Goal 이 있습니다. 왼쪽부터 탐색할 때 무조건 Depth가 4인 Goal을 Goal로 판단하므로 Optimal 하지 않습니다.

③ Time Complexity

= Graph Search 라면 State Space 전체 입니다.

= Tree Search 라면 $O(b^m)$ 입니다.

④ Space Complexity

= Graph Search 라면 State Space 전체 입니다.

= Tree Search 라면 $O(b*m)$ 입니다.

- $O(b*m)$ 은 매우 작은 값이고 , 지금 까지 Complete , Optimal , Time Complexity가 모두 BFS 보다 좋지 않았지만 Space Complexity가 압도적으로 좋습니다.

- Tree search 일 때는 frontier에 저장합니다. DFS 그림을 참고하면 , 각각의 상황 마다 frontier에 저장되는 노드의 개수는 최대 b개입니다.

4. Depth-Limited Search

1) Depth-Limited Search Overview

- = DFS의 가장 큰 문제는 State Space가 무한하다면 무한루프를 돌게 됩니다. 이를 해결하기 위해 **depth에 제한을 겁니다.**
- Return 값이 Solution Find or Failure or Cutoff(Depth의 Limit에 걸림) 입니다.
- Goal도 아니고 limit = 0이면 cutoff 입니다.
- limit이 > 0 일 때 탐색 도중 cutoff(limit에 걸림)가 발생하면 return으로 cutoff 나오지만 , cutoff도 아니고 failure도 아니면 goal 입니다.(잘 이해 x)

2) 평가

① Complete

- = 만약 Limit을 Goal의 Depth 보다 낮게 설정하면 절대 Goal을 찾을 수 없습니다.

② Optimal

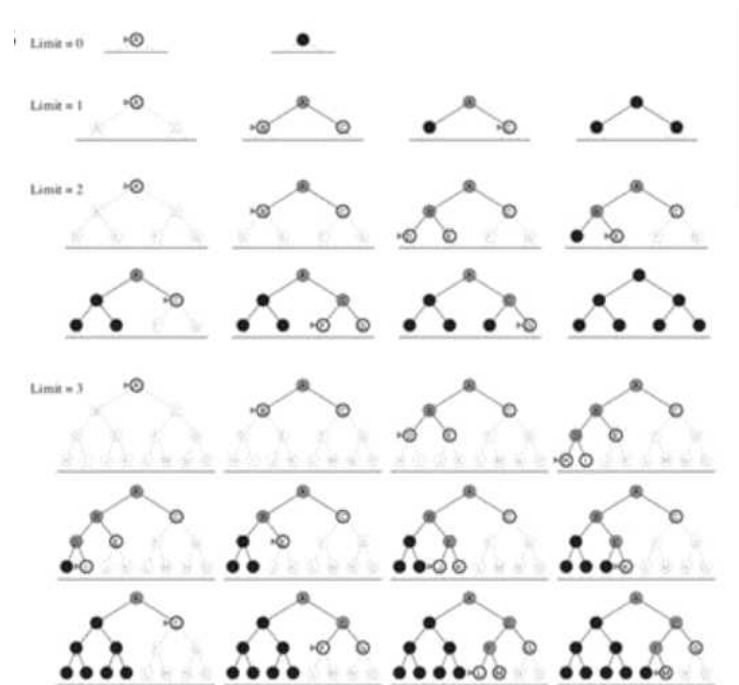
- = DFS 와 같은 원리로 Optimal 하지 않습니다.

③ Time Complexity & Space Complexity

- = DFS에서 m 값이 limit 값으로 바뀌므로 , 기본적으로 $\text{limit} < m$ 이므로 설정의 정도에 따라 복잡도의 감소량이 달라집니다.

5. Iterative Deepening DFS(IDS)

1) IDS Overview



- = Depth-Limited Search의 단점을 보완하여 Cutoff가 나오면 Limit을 점점 증가 시킵니다.
- Depth limit을 0 , 1 , 2 ,3 ... 순차로 증가시켜서 DFS를 진행 합니다.
- DFS와 BFS의 장점을 합친 탐색 방법입니다.

2) 평가

① Complete

= Depth를 제한시키고 DFS를 하기 때문에 해당 Depth 까지는 모든 노드를 탐색하게 됩니다. 그리고 Cut-off를 증가시키므로, Solution이 존재한다면 반드시 Solution을 찾게 됩니다.

② Optimal

= Depth를 제한하기 때문에 depth가 1까지에서 goal을 찾으면 goal의 depth는 1 이고, 1에서 찾지 못하여 depth가 2까지에서 goal을 찾으면 goal의 depth는 2가 됩니다. 즉, 같은 depth에서 찾은 goal 끼리는 depth가 같으므로 optimal 합니다.

③ Time Complexity

= 여러 번 반복하기 때문에 중복이 많아집니다. 그로 인해 시간복잡도가 BFS에 비해 훨씬 클 것 같지만 BFS보다 당연히 크지만 비슷합니다.
- 가장 깊은 곳의 노드는 한 번 만 생성이 되므로 실질적인 탐색횟수는 감소합니다.

④ Space Complexity

= DFS와 똑같습니다.

<IDS vs BFS>

7. Branching factor는 5, 가장 얕은 goal의 depth는 7일 때, breadth-first search와 iterative deepening search가 각각 생성하는 노드의 최대 개수를 적으시오. (10점)

BFS: $5 + 5^2 + \dots + 5^7 = 97655$

IDS: $7 \times 5 + (7-1) \times 5^2 + \dots + (7-6) \times 5^7 = 122060$

7. (a) Iterative deepening depth-first search 알고리즘과 breadth-first search 알고리즘 각각의 time complexity를 수식을 이용하여 기술하고, (b) 두 알고리즘의 time complexity 차이가 가지는 의미에 대해서 논하시오. (15점)

(a) b: branching factor, d: depth of the shallowest solution

$N(\text{BFS}) = b + b^2 + b^3 + \dots + b^d = O(b^d)$

$N(\text{IDS}) = (d)b + (d-1)b^2 + \dots + (1)b^d = O(b^d)$

가장 얕은 goal의 depth가 7일 때

- depth_1인 노드는 7번 생성

- depth_2인 노드는 6번 생성

- <생략>

- depth_7인 노드는 1번 생성 될 것 입니다.

6. Bidirectional Search

= Root Node와 Goal Node에서 각각 시작하여 양방향 탐색을 하게 됩니다.

= 시간복잡도와 공간복잡도가 매우 낮아지는 장점이 있지만 Goal Node를 반드시 알아야 사용할 수 있는 단점이 있습니다.

7. 6개의 Uninformed Search 비교

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

<3-5강_Informed Search(=Heuristic Function)> 10/7 ~ 10/12

= 정보를 갖고 문제를 해결하기 때문에 Uninformed Search 보다 효율적입니다.

<용어>

1) Heuristic

= 휴리스틱 또는 발견법 이라고 합니다.

= 불충분한 시간이나 정보로 인하여 합리적인 판단을 할 수 없거나, 체계적이면서 합리적인 판단이 굳이 필요하지 않은 상황에서 사람들이 빠르게 사용할 수 있게 보다 용이하게 구성된 **간편 추론**의 방법입니다.

2) $h(n)$

= Heuristic Function

= 노드 n 에서 Goal 까지 가장 저렴할 것이라고 추정되는 비용

- **추정한 값** 입니다.

3) $g(n)$

= Initial node부터 노드 n 까지 이동한 비용.

- **실제로 계산 된 값** 입니다.

4) $f(n)$

= Evaluation Function

= 어떤 노드를 선택해야 Goal로 갈 때 Optimal 한지 평가하는 함수 입니다.

- Best-first search를 사용할 때 사용합니다.

- $h(n)$ 을 포함합니다.

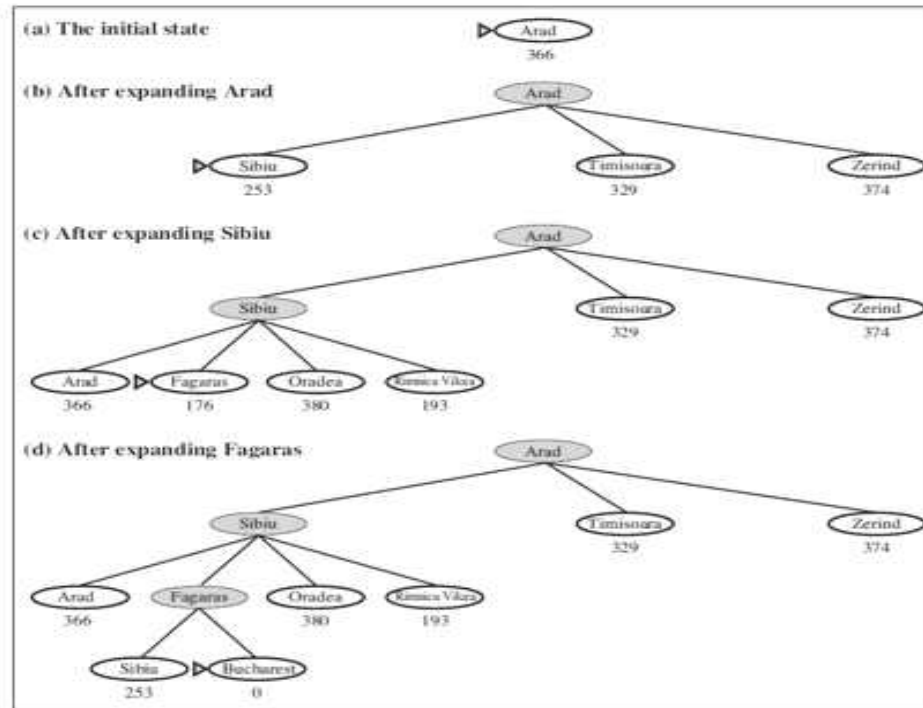
1. Greedy Best-First Search

1) Greedy Best-first Search Overview

= Greedy 기반이므로 , Goal에 가는 경로에서 Optimal 할 것 같은 노드들을 골라서 이동합니다.

- $f(n) = h(n)$
- 어떤 Node가 Optimal 한지 간단하게 평가하여 선택합니다.

<Example>



- Goal인 Bucharest에서 다른 도시까지의 거리에 대한 정보를 알고 있습니다.

ex) Arad(Init Node)는 366 , Sibiu는 253

- Frontier에 존재하는 노드들 중에서 주어진 정보 값이 작은 노드를 선택하여 Goal에 도착합니다.

2) Performance

① Complete

= DFS처럼 동일한 노드를 반복해서 이동 할 수 있으므로 Tree version 일 때는 Complete 하지 않습니다.

② Optimal

= Serach Cost는 Optimal 하지만 Result는 Optimal 하지 않습니다.

③ Time Complexity & Space Complexity

= $O(b^m)$

2. A* Search

1) A* Search Overview

- 대표적인 Best-first search 의 종류 입니다.

- $f(n) = g(n) + h(n)$

- 실제 값과 추정 값을 더한 것을 노드의 평가방식으로 사용합니다.

- $h(n) = 0$ 이 된다면 Uniform-Cost-Search와 동일하게 됩니다.

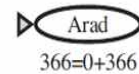
- Optimal 한 Search Algorithm 중에서 가장 효과 적입니다.

- 루마니아 문제(Arad에서 Bucharest 까지 현재의 도시에서 인접한 도시 하나를 반드시 지나서 도착하는 문제)를 예시로 하여 공부합시다.

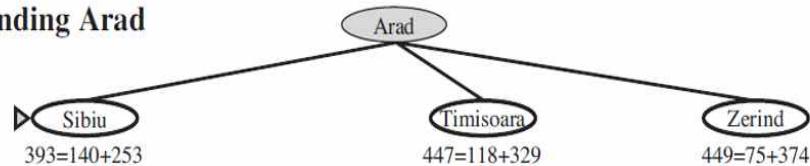
- 인접한 도시를 반드시 지나면서 이동하기 때문에 실제 이동거리는 반드시 Bucharest까지의 직선거리인 $h(n)$ 보다 크거나 같습니다.(admissible)

2) 루마니아 문제에 A* Algorithm을 적용합니다.

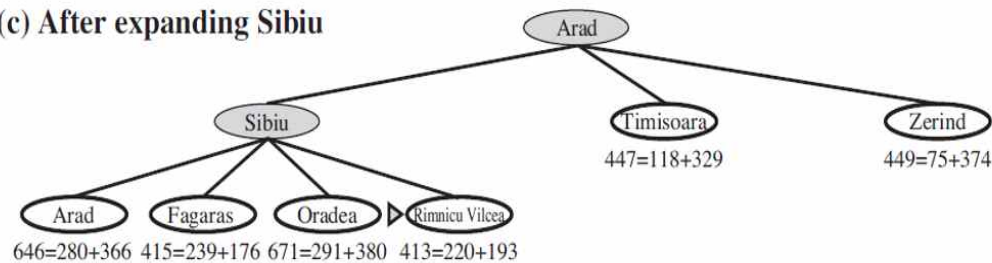
(a) The initial state



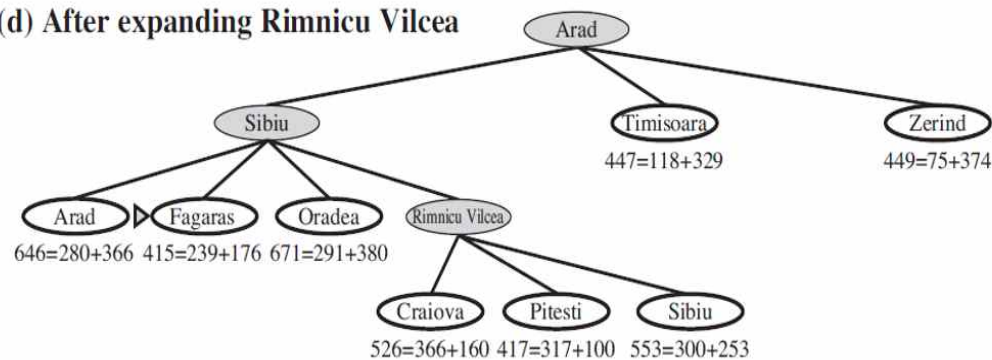
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



- $h(n)$ 은 해당 노드에서 골까지의 직선거리 이며 미리 알고 있는 정보(Informed)입니다.

- 그림은 $f(n) = g(n) + h(n)$ 으로 구성되어 있습니다.

- $f(n)$ 은 Arad에서 현재 노드를 지나서 골까지의 거리를 추정한 값입니다. (실제 값 + 추정 값 이므로 추정 값 입니다.)

- $g(n)$ 은 Arad에서 현재 노드까지 걸어온 거리를 계산한 실제 값입니다.

- $h(n)$ 은 현재 노드에서 Goal 까지 거리를 추정한 값입니다.

- A* Algorithm은 Frontier 에서 Pop할 때 Goal인지 검사합니다.

<탐색시작>

- Frontier가 비었는지 검사합니다.

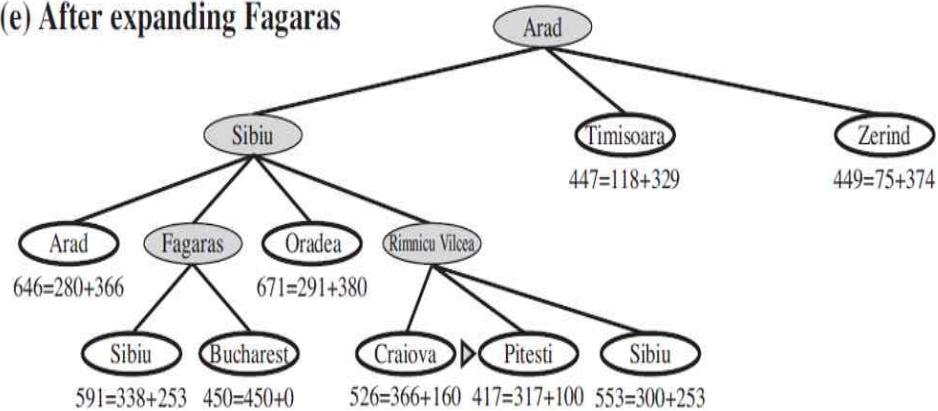
- (a) : 최초의 노드인 Arad만이 Frontier에 존재하므로 Arad를 선택(Pop) 합니다.

- (b) : 3개의 노드를 Expand하고 Frontier에 존재하는 3개의 노드 중에서 $f(n)$ 이 393으로 가장 작은 Sibiu를 선택하여 Frontier에서 Pop합니다.

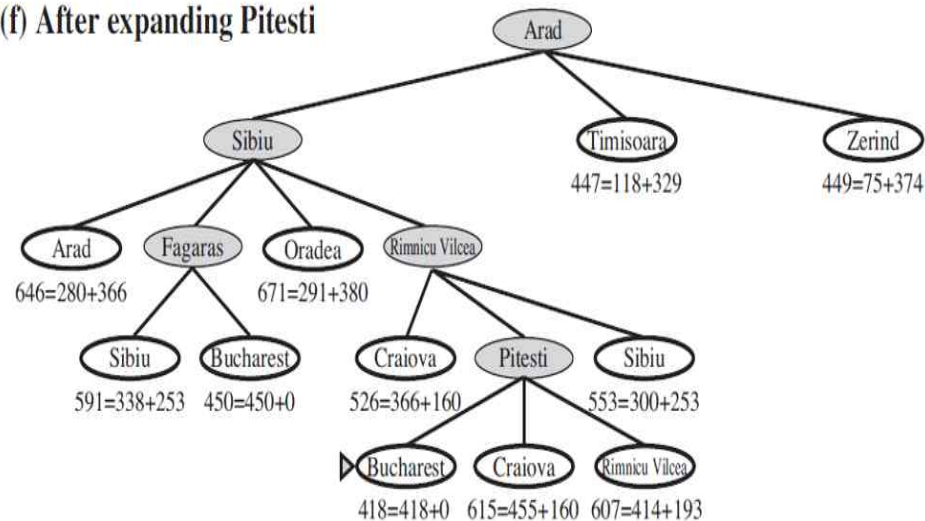
- (c) : 4개의 노드를 Expand하고 Frontier에 존재하는 6개의 노드 중에서 $f(n)$ 이 413으로 가장 작은 Rimnicu Vileea를 선택하여 Frontier에서 Pop합니다.

- (d) : 3개의 노드를 Expand하고 Frontier에 존재하는 8개의 노드 중에서 $f(n)$ 이 415로 가장 작은 Fagaras를 선택하여 Frontier에서 Pop합니다.

(e) After expanding Fagaras



(f) After expanding Pitesti



- (e) : 2개의 노드를 Expand하고 Frontier에 존재하는 9개의 노드 중에서 $f(n)$ 이 417으로 가장 작은 Pitesti를 선택하여 Frontier에서 Pop합니다.

Bucharest가 Frontier에 존재하지만 Frontier에 넣을 때 Goal인지 검사를 하지 않고 Frontier에서 Pop할 때 검사하므로 알고리즘은 계속 진행 됩니다.

- (f) : 3개의 노드를 Expand하고 Frontier에 존재하는 11개의 노드 중에서 $f(n)$ 이 418로 가장 작은 Bucharest를 선택하여 Frontier에서 Pop합니다.

- (최종) : Bucharest를 Expand 할 때 Bucharest를 Pop하면서 Goal인지 검사를 하게 됩니다. Bucharest가 Goal이므로 A* Search Algorithm이 Problem Solving을 완료했습니다.

3) A* Search 특성

① Admissibility

= 휴리스틱($h(n)$)을 과대추정 하지 않는 것을 전제하여서 언제나 어떤 노드까지 추정한 비용이 어떤 노드까지의 실제 비용보다 작은 특성.

② Consistency

= $h(n) \leq c(n, a, n') + h(n')$ <- 이 식을 만족하면 Consistent 합니다.

= n 에서 Goal까지 이동할 때 드는 비용의 추정 값 $\leq n \rightarrow n'$ 까지 Action_a를 해서 갈 때 드는 실제 비용 + n' 에서 Goal까지 이동할 때 드는 비용의 추정 값

- Admissibility를 좀 더 자세하게 다룬 개념입니다. 그러므로 Consistency를 만족하면 자동으로 Admissibility를 만족합니다.
- Admissibility를 만족하면 Consistency를 만족할 수도 있지만 , 만족하지 않을 수도 있습니다.
- Tree에서는 Admissibility를 만족하면 Optimal 합니다.
- Graph에서는 Consistency를 만족하면 Optimal 합니다.
- A* Search는 주로 Graph를 사용합니다.

<증명 : Graph에서는 Consistency를 만족하면 Optimal 합니다.>

① $f(n) \leq f(n')$

<Proof>

- 노드 n 에서 노드 n' 을 지나 노드 n'' 으로 간다고 가정합니다.
- $f(n) = g(n) + h(n)$ 은 A*의 정의로 증명이 가능합니다.
- $g(n) = g(n) + c(n, a, n')$ 은 n' 까지 지나온 비용이 n 까지 지나온 비용 + n 에서 Action_a를 통해 n' 으로 가는 비용이고 시작점에서 n 보다 n' 이 멀리 때문에 증명이 가능합니다. 따라서 $f(n) = g(n) + c(n, a, n') + h(n')$ 입니다.
- Consistency의 기본 부등식에 따라 $c(n, a, n') + h(n') \geq h(n)$ 이므로 $f(n) \geq g(n) + h(n)$ 임을 알 수 있습니다.
- $f(n) = g(n) + h(n)$ 이므로 , $f(n) \leq f(n')$ 이 성립함을 알 수 있습니다.

② Frontier에서 노드 1개를 선택하면 그 노드까지의 optimal한 경로를 찾은 것 입니다.

= 이 정리를 확장하면 , Goal 노드 선택한 경우도 optimal 한 것 입니다.

<Proof>

- 그래프의 노드는 unexplored node , frontier node , explored node로 구분할 수 있습니다.
- Unexplored node는 아직 expand 되지 않아서 한 번도 frontier에 들어가 보지 못한 노드입니다.
- Frontier node는 frontier에 들어 있는 노드입니다.
- Explored node는 frontier에 들어가 본 노드입니다.
- 즉 , Explored Set 에 들어가려면 반드시 Frontier에 들어 가 본 적이 있어야 합니다.

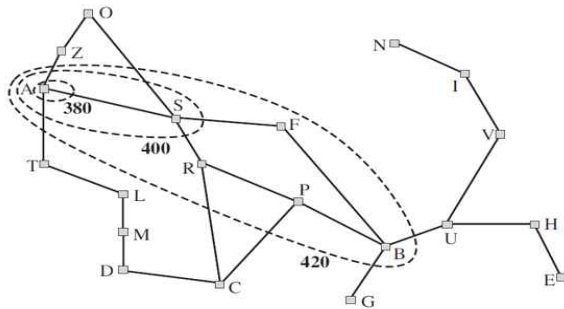
4) A*의 시간복잡도

= $h(n)$ 의 정확도가 뛰어나수록 시간복잡도는 감소합니다.

① 간단한 $h(n)$ 의 정확도

= Contours(등고선)으로 알 수 있습니다.

- 등고선 내부의 노드들은 등고선의 값 보다 작습니다.
- 등고선의 간격이 좁을수록 시간복잡도가 낮습니다.
- 등고선이 좁으면 탐색할 노드가 적어집니다.
- A -> B로 가는 경로라면 등고선_420 이내의 노드들만으로 경로를 구성합니다.



② 자세한 $h(n)$ 의 정확도

– Absolute error: $\Delta \equiv h^* - h$

– Relative error: $\epsilon \equiv \frac{h^* - h}{h^*}$

- h^* = 실제 비용
- h = 휴리스틱 비용
- 그러므로 h 가 consistent 하거나 admissible 하면 $h^* > h$ 을 만족하므로 $h^* - h > 0$ 입니다.
- relative error는 엡실론 공부하고 다시 듣자.

$O(b^{\epsilon d})$

5) 공간복잡도

= 너무 어려운 개념이므로 교수님께서 넘어가셨습니다.

<3-6강_Make Heuristic Function> 10/15

1. The 8-Puzzle Problem

*** 3-6강에서는 모두 8-Puzzle Problem을 예시로 사용합니다. ***

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- = Start State의 퍼즐 모델을 Goal State의 퍼즐 모델로 Optimal 하게 바꾸는 방법입니다.
- 사람은 평균 22번 움직여서 해결합니다. 이는 Depth가 22임을 의미합니다.
- BFS_트리는 3^{22} (300억)개의 노드를 생성 합니다. 노드 1개에 1m/s라 가정해도 3만초가 걸립니다.
- BFS_그래프는 10^{13} 개의 노드를 생성합니다.
- 휴리스틱을 고려하지 않는 Problem Solving은 현실적으로 불가능 합니다.

<Branching Factor>

= 현재 상태에서 가능한 행동

ex) 7번 퍼즐을 고려할 때 , 2번과 5번이 없다면 우측과 아래로 갈 수 있으므로 branching factor는 2가 됩니다.

2) A* Algorithm의 휴리스틱 함수와 성능비교

2. 2개의 휴리스틱 함수

1) h_1

= Start State에서 어떤 퍼즐의 위치가 Goal State에서 같은 퍼즐의 위치와 다른 개수입니다.

- 1개의 타일이라도 위치가 잘못되어 있으면 1 이상입니다.

ex) 그림의 Start State에서 모든 퍼즐의 위치가 Goal State에서의 퍼즐의 위치와 다르므로 8입니다.

2) h_2

= Start State에서 어떤 퍼즐이 Goal State에서 같은 퍼즐 위치까지의 Manhattan distance입니다.

- Manhattan distance는 시작점부터 도착점까지 상하좌우 이동을 하여 도착하는 최단거리 이므로 실제 거리보다 작거나 같습니다.

- 실제로 퍼즐 게임을 하다보면 , 최단으로 이동하는 거리가 있음에도 불구하고 다른 퍼즐도 신경 쓰기 때문에 최단거리로 이동하지 않는 경우가 대부분 입니다.

ex) 퍼즐 1은 3 , 퍼즐 2는 1 , 퍼즐3은 2 ... 퍼즐 8은 2 이므로 총 18입니다.

3. 다른 알고리즘과의 성능 비교

= IDS vs A*(h1) vs A*(h2)

d	Search Cost (nodes generated)			Effective Branching Factor		
	IDS	A*(h ₁)	A*(h ₂)	IDS	A*(h ₁)	A*(h ₂)
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

1) Search Cost

= 만들어 내는 노드

= Depth가 12일 때 IDS는 이미 너무 많은 노드가 필요하고 , Depth가 24여도 A*들은 각각 39135 , 1641이므로 , 초당 노드를 100만개 만든다고 가정할 때 매우 짧은 시간 내에 노드들을 만들어낼 수 있습니다.

2) Effective Branching Factor

= b^입실론

= 휴리스틱의 효율성을 알 수 있는 지표입니다. 구하는 방법도 설명해주셨지만 넘어가겠습니다.

- 입실론은 0 ~ 1 사이의 값입니다.

- IDS < A*(h1) < A*(h2) 순으로 성능이 좋습니다.

- 자료를 모두 검사하면 , A*(h1)보다 A*(h2)가 언제나 성능이 좋습니다. 이는 h2가 h1보다 항상 적은 노드를 생성함을 의미하고

h2 dominates h1 라고 부릅니다.

4. 휴리스틱 함수를 만드는 2가지 방법

<Relaxed Problems(=Simplified Problems)>

= Original Problem에 조건을 추가하여 간략하게 만든 문제

1) Relaxed Problems의 Solution이 Original Problems의 휴리스틱 함수입니다.



- Relaxed Problems 의 Solution이 admissible 하기 때문에 Original Problem의 Solution 보다 무조건 비용이 적게 듭니다. 그러므로 Original Problems에서는 휴리스틱 함수가 됩니다.

- Original Problem에서의 Solution은 당연히 Relaxed Problems에서의 Solution이 될 수 있습니다.

- Relaxed Problems에서 찾은 새로운 Solution은 Original의 Solution 보다 더 좋은 Solution 일 수 있습니다.

- Relaxed Problem이 Search 없이 Solution을 구할 수 있을 때만 Relaxed Problem이 의미가 있습니다. Relaxed Problem을 찾을 때도 Search가 필요하면 오히려 더 어려워지게 됩니다.

따라서 relaxed problem의 solution이 original problem의 solution이 아닐 수 있지만 admissible함(original problem의 solution 보다 무조건 더 짧으니까) 휴리스틱은 될 수 있다.

- 아래의 예시를 정독하는 것이 위의 정의 보다 더 쉽게 이를 이해할 수 있는 방향이라고 생각합니다.

<Example : Relaxed Problems 만들어 보기>

– A tile can move from square A to square B if

- 1) A is horizontally or vertically adjacent to B
- 2) B is blank

original

– Three relaxed problems

- A tile can move from A to B if A is adjacent to B
- A tile can move from A to B if B is blank
- A tile can move from A to B

Relaxed

= 원래는 Formal Language를 이용해서 Relaxed Problems를 만들지만 아직 배우지 않았으므로 자연어로 나타냅니다.

- 1번 Relaxed Problem = Original의 2)번 특성을 제거 합니다 = 빈칸이든 말든 상관없고 인접하면 이동이 가능합니다.

- 2번 Relaxed Problem = Original의 1)번 특성을 제거 합니다 = 인접하든 말든 빈칸이면 이동이 가능합니다.

- 3번 Relaxed Problem = Original의 1)번 2)번 특성을 제거 합니다 = 조건 없이 이동 가능이 가능합니다.

2) Sub-Problems를 이용합니다.

*	2	4
*		*
*	3	1

	1	2
3	4	*
*	*	*

- = 8개의 퍼즐 중에서 n 개만 goal을 만족하면 되도록 Solving을 합니다. 그리고 그 Solution을 휴리스틱으로 사용합니다.
- Sub-Problems의 solution이 언제나 Original Problems의 solution 보다 작으므로 admissible 합니다.
- 이전에 h_1 과 h_2 는 구하기 쉬움을 증명했습니다. 그러나 Sub-Problems는 적어도 h_1 보다 h_2 는 구하기가 어렵습니다.

<Pattern Databases>

- = Sub-Problems를 구하기 위해서 구축한 데이터베이스.
- = 임의의 node에서 다른 node들까지의 cost를 저장해서 Database로 저장하는 방식을 무수히 반복하면 모든 node에서 다른 node까지의 cost를 저장하는 Database를 완성하게 됩니다.
- h_1 , h_2 보다는 구하기 어렵지만 Original-Problem 보다는 훨씬 간단합니다.

<Example : Puzzle>

- puzzle에서
- $h(1234)$ = 퍼즐 1,2,3,4만 goal에 맞게 위치를 이동시키고 5,6,7,8도 저장은 합니다.
- $h(5678)$ = 퍼즐 5,6,7,8,만 goal에 맞게 위치를 이동시키고 1,2,3,4도 저장은 합니다.
- $h'(1234)$ = 퍼즐 1,2,3,4만 goal에 맞게 위치를 이동시키고 5,6,7,8이 포함된 경로는 완전히 무시합니다.
- $h'(5678)$ = 퍼즐 5,6,7,8만 goal에 맞게 위치를 이동시키고 1,2,3,4가 포함된 경로는 완전히 무시합니다.

- ① $h(1234)$ 와 $h(5678)$ 를 휴리스틱(Sub-Problems)으로 하여 문제를 해결하면 Manhattan distance를 사용하는 것보다 약 1000배나 빠릅니다.
- ② $h(1234)$ 와 $h(5678)$ 를 disjoint(합침)하여 문제를 해결하면 빠를 것 같지만 두 휴리스틱 함수는 공유하는 데이터가 존재하여 중복이 있습니다. 그로 인해 admissible 하지 않습니다. 하지만 $h'(1234)$ 와 $h'(5678)$ 를 disjoint하면 중복이 없기 때문에 문제를 해결할 수 있고, 이는 Manhattan Distance를 휴리스틱으로 하는 것보다 10000배 빠릅니다.

5. 휴리스틱 학습

*** 개론수업 이므로 깊게 하지 않습니다. ***

= 머신러닝으로 경험들을 토대로 학습합니다.

- 학습 경험은 node의 상태 , goal까지의 cost
- pattern data = subprogram 풀 때 필요한 데이터베이스
- Learning data = 휴리스틱 학습 때 필요한 data = Original program 해결 할 때 필요한 데이터베이스

<4-1강_Local Search> 10/15 ~ 11/03

- 3장에서는 Fully observable , Deterministic , Known한 문제들을 다루었습니다. 또한 Solution이 Action Sequences(연속된 액션의 결과)였습니다. 4장에서는 Partially observable , Non-Deterministic한 문제들을 다루겠습니다.

1. Local Search 정의

= 오직 Goal State를 찾는 방법에만 집중합니다.

- 3장에서는 Goal까지 어떻게 해야 최적의 경로(적은 Cost)로 올 수 있을지를 고민했다면 Local Search에서는 방법과 Cost에 초점을 두지 않고 Goal을 찾는 데에만 집중합니다.

- Goal을 찾기가 어렵습니다.

- 이전에 탐색한 노드들을 저장할 자료구조가 필요 하지 않습니다.

- Current node만 필요하기 때문에 저장하는 Node가 매우 적어서 메모리 문제와 관련이 없습니다.

2. Hill-Climbing Search

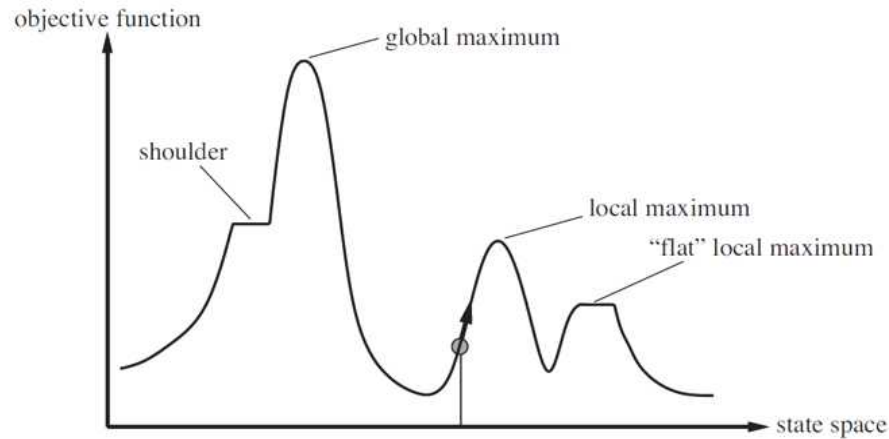
= Current Node에서 이웃 노드(현재 노드에서 갈 수 있는 노드)들을 찾고 , 이웃 노드들 중에서 가장 좋은(optimal) 노드를 골라서 자신과 비교했을 때 더 좋다면 이웃 노드를 Current Node로 하고 더 좋은 노드가 존재하지 않는다면 종료합니다.

- 가장 좋은 노드의 의미는 Current Node에 비해 값이 크거나 , 값이 크거나 같은 , 값이 작거나 , 값이 작거나 같은 일 수 있습니다. 이걸 문제에서 주어줍니다.

- 이웃 노드들이 매우 많을 때 사용합니다.

- Incomplete합니다. 올바른 해를 찾지 못할 가능성이 있습니다.

1) Simplified State-Space Landscape



- Initial Node를 어디로 설정하느냐에 따라서 Local Maximum에서 종료될 수 있고, Global Maximum에서 종료될 수 있습니다.
- Local Maxima에서 종료되면 Goal을 찾지 못한 경우입니다.

= Hill-Climbing Search를 해결하기 위해 State Space를 그려야 합니다. 3차원으로 표현하는 것이 좋지만 어렵기 때문에 그리기 간단한 간략한 버전을 이용합니다.

① X축

= State Space에서 다른 노드로 이동하는 경우입니다.

② Y축

= $h(n)$ 이며, 이 문제는 y축이 큰 값이 Optimal 한 경우입니다.

③ Global Maximum

= 전체 범위에서 가장 Optimal한 Node 입니다. 당연히 Current Node 기준, 왼쪽이나, 오른쪽으로 이동해도 자신 보다 Optimal한 경우가 없습니다.

= Goal 입니다.

④ Local Maxima

= X축에서 왼쪽이나 , 오른쪽으로 이동해도 자신 보다 Optimal한 경우를 찾을 수 없는 경우입니다. 하지만 **지역적인 범위에서만 Optimal 하며 전체 범위에서는 Optimal 하지 않을 수 있습니다.**

- 현대의 AI문제에서는 Local Maximum은 그래프에서 매우 많이 존재합니다.
- Maxima는 복수형 Maximum 입니다.

⑤ Shoulders

= 평행한 부분이 존재하여 , 평행한 부분을 따라가면 더 Optimal한 노드를 찾을 수 있습니다.

⑥ Flat Local Maxima

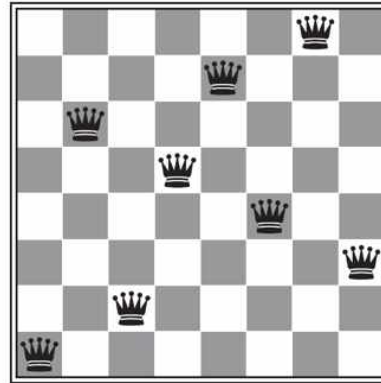
= 평행한 부분이 존재하지만 , 평행한 부분을 따라가도 더 Optimal한 노드를 찾을 수 없습니다.

⑦ Sideways moves(평행 이동)

= 기존에는 더 Optimal 한 노드는 큰 value를 갖는 것 또는 작은 value를 갖는 것이었습니다. 하지만 평행 이동을 접목시키면 **Current_node와 이웃 노드의 value가 같아도 Optimal한 것으로 고려하게 됩니다.**

2) Example : N-queen

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18



= 8x8 체스에서 8개의 queen을 놓고 퀸끼리 서로 죽이지 않게 만드는 Goal을 찾습니다.

- 하나의 퀸 당 세로를 기준으로 하면 7개의 위치로 이동할 수 있으므로 $7 \times 8 = 56$ 개의 이동 가능한 후속 노드(Successors) 가 존재합니다. 다시 말해 그림에서 64개의 체스보드가 모두 노드라고 생각하고 숫자가 적혀 있는 체스보드 56개가 Successors입니다.
- $h(n)$ = 서로를 공격할 수 있는 퀸의 쌍의 개수
- 56개의 Successors들 중에서 $h(n)$ 의 값이 가장 작은 노드를 고르고 그게 현재의 상황보다 좋아진다면 퀸을 해당 노드로 이동 시킵니다.
- 해당 그림에서는 $0 \leq h(n) \leq 28(8 \times 7 / 2)$ 입니다.
- N-Queen문제는 $h(n)$ 이 작은 이웃 노드가 Optimal 하기 때문에 퀸이 있는 노드를 제외한 56개의 Successors 중에서 값이 12인 노드 8개 중 1개를 무작위로 고릅니다. 이를 5번 하면 $h(n) = 1$ 인 successor를 찾게 됩니다. 그러나 , 그 다음 수행에서 1 보다 작은 $h(n)$ 을 찾을 수 없어 Hill-Climbing Search가 종료됩니다. -> 이에 관한 설명이 Local maxima 와 Global maximum 입니다.

<Incremental formulation>

= 퀸을 1개씩 넣어가며 배열합니다.

<Complete-State formulation>

= 64개의 칸에 무작위로 8개의 퀸을 배열합니다.

3) 변형 Hill-Climbing Search

① Stochastic Hill Climbing

= 이웃 노드 중에서 가장 Optimal한 노드로 이동하는 것이 아니라 , 자신 보다 Optimal 하긴 하지만 가장 Optimal하지 않은 노드도 확률적으로 고려해서 갈 수 있습니다.

- Local Maxima를 벗어 날 수 있습니다.

ex) Value가 큰 Node가 Optimal 하다고 가정하고 설명하겠습니다. Current_node의 Value가 5이고 , 이웃 노드의 Value가 10 , Value가 20인 노드를 찾았다고 가정합니다. 원래는 무조건 20인 노드를 Current_node로 변경하지만 , 확률을 도입해서 33.3%의 확률로 10인 노드 , 66.6%의 확률로 20인 노드를 Current_node로 변경합니다.

② First-Choice Hill Climbing

= 이웃 노드 탐색 과정에서 자신 보다 optimal 하면 Current_node로 변경합니다.

- successor가 매우 많을 때(100만개 정도) 사용하는 방식 입니다.

③ Random-Restart Hill Climbing

= Goal을 찾을 때 까지 Initial-State를 변경해가며 무한정 탐색(Loop)을 합니다.

- 몇 번 Loop를 돌아서 Goal을 찾는지가 중요하며 , Goal을 찾을 확률을 P라고 합니다.

<Example : N-Queen 문제에 적용해봅시다.>

- Sideways moves를 허용하지 않을 때 Goal을 찾는 경우 평균 4번의 이동을 하고 , Goal을 찾지 못하는 경우 평균 3번의 이동을 합니다.

- Sideways moves를 허용할 때 Goal을 찾는 경우 평균 21번의 이동을 하고 , Goal을 찾지 못하는 경우 평균 64번의 이동을 합니다.

- Sideways moves를 허용하지 않으면 P가 14%가 됩니다. 다시 말해 , 약 0.86/0.14번을 서로 다른(?) Initial-State에서 탐색을 시작하면 1번은 Goal을 찾습니다. 그러므로 , Goal을 찾기 까지 $1 * 4 + 0.86 * 3 = 3.58$ 회의 이동을 합니다.

- Sideways moves를 허용하면 P가 94%가 됩니다. 다시 말해 , 약 0.06/0.94번을 서로 다른(?) Initial-State에서 탐색을 시작하면 1번은 Goal을 찾습니다. 그러므로 , Goal을 찾기 까지 $1 * 21 + 0.06 * 64 = 25.36$ 회의 이동을 합니다.

- 그러므로 $3.58 < 25.36$ 이므로 Sideways moves를 허용하지 않는 경우가 더 좋은 알고리즘 입니다.

3. Simulated Annealing

- t : Loop의 Index
- T : Temperature(온도)
- ΔE : $\text{next_node} - \text{current_node}$

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem
schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow \text{next.VALUE} - \text{current.VALUE}$

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

< $T \leftarrow \text{schedule}(t)$ >

- t 는 Loop의 Index이며 , 이를 schedule함수에 Input으로 넣어서 Index에 맞는 온도를 T 로 하여 Return 합니다.

- T 가 0이면 goal state로 Return 합니다.

ex) T 의 값은 3000도 , 2900도 , 2800도 , etc , 0도 가됩니다.

<아래에서 4줄의 코드>

- Successor들 중에 무작위로 다음에 검사할 노드를 골라서 next_node로 설정합니다.

- $\Delta E > 0$ 이면 hill-climbing search와 동일하게 next가 더 좋은 노드이므로 next_node를 current_node로 설정합니다.

- $\Delta E \leq 0$ 이면 확률적 계산을 통해 next_node를 current_node로 변경하거나 변경하지 않습니다.

<확률 : $e^{\Delta E/T}$ >

- $\Delta E \leq 0$ 에서만 검사하므로 ΔE 는 언제나 0보다 작거나 같습니다.

- 교수님의 그래프를 참고하면 T 가 클수록 확률은 1에 가까워지고 , T 가 작을수록 확률은 0에 가까워집니다.

- 초기에는 T 가 크기 때문에 Annealing으로 움직이는데 T 가 작아질수록 점점 Hill-climbing과 동일해집니다.

4. Local Beam Search

1) Local Beam Search

- = Current_node만 저장하기 때문에 메모리 공간의 문제가 없으므로 여러 개의 current_node를 생성하는 방법이 도입됩니다.
- Initial_node를 K개 생성하여서 current_node를 K개로 계속 유지하여 진행합니다.
- K개의 current_node는 각각 successor를 만들고 successor 중에서 성능이 좋은 K개를 선택해서 current_node로 만듭니다.

<Hill-climbing 5회 반복 vs 5개의 current_node로 하는 Local beam search>

- Hill-climbing은 결국 1개의 current_node에서 생긴 successor를 이용하는 search를 5번 하는 것.
- Local beam search는 5개의 current_node에서 생긴 successor를 모두 고려하는 search를 1번 하는 것.
- 어떤 것이 더 좋은 성능을 갖는 지는 상황 마다 다릅니다.

2) Stochastic Beam Search

= 확률을 도입한 Local Beam Search.

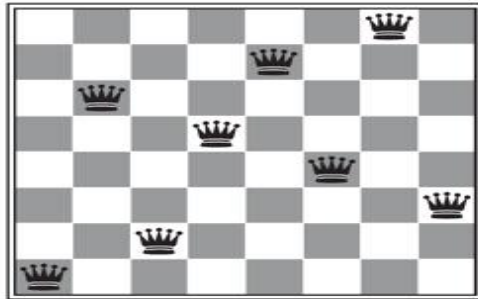
<Example>

- 상황 : node_A = 5 , node_B = 7 , node_C = 3 일 때 , 3개의 노드(successor)중에서 값이 가장 큰 2개를 선택하여 current_node로 합니다.
- 기존의 Local Beam Search라면 무조건 B , A를 선택 합니다.
- Stochastic Beam Search라면 A를 2/6 , B를 3/6 , C를 1/6 확률로 선택합니다.

3) Genetic Algorithm (유전 알고리즘)

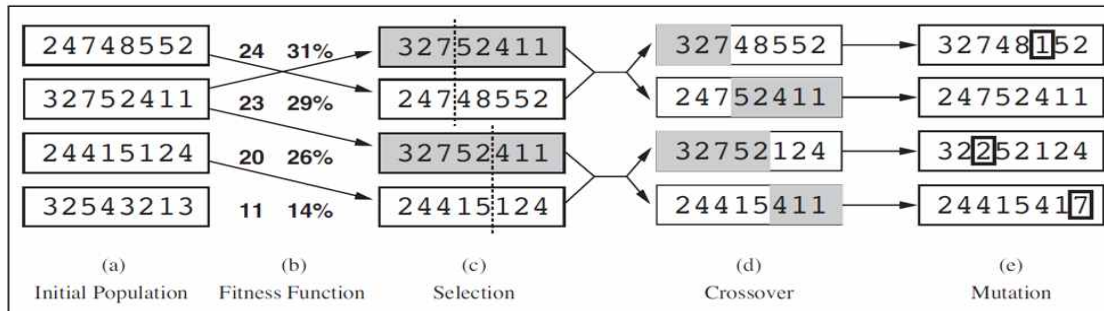
- Stochastic Beam Search의 변형입니다. 두 알고리즘 모두 다수의 state를 유지해 가며 알고리즘을 진행시킵니다.

<그림_1 : N-Queen>



- Population(집단) : K개의 current_state
- Individual(개체) : Population에 있는 각각의 node이며 String으로 표현합니다.
- 그림을 '1 6 2 5 7 4 8 3' 이라는 개체(문자열)로 나타낼 수 있습니다.
- 맨 왼쪽 퀸은 column에서 1번에 있고 , 그 다음 퀸은 column에서 6 번에 있습니다.

<그림_2 : N-Queen's Genetic Algorithm>



- 기존의 N-Queen 문제 알고리즘에서는 cost(서로 잡을 수 있는 여왕의 개수) 개념을 사용해서 cost가 0이면 goal state가 되었습니다.
- Genetic Algorithm에서는 서로 공격을 하지 못하는 여왕의 쌍으로 Fitness를 사용합니다. Fitness는 cost와 완전 반대의 개념이며 , 28개일 때 goal state가 됩니다.
- 그림_2번에서 확률은 '나의 fitness / 전체 fitness'입니다.

<독특한 Successor 선정 방식>

*** 그림_2를 참고하며 설명하겠습니다. ***

- 4개의 successor를 뽑힐 확률을 이용해서 4개의 successor를 다시 뽑습니다.

① Parent 선택

= 기존과 동일하게 Successor를 뽑습니다. (c)에 해당하는 4개의 문자열이 모두 Parent 입니다. (b)에서 주어진 확률로 Successor를 뽑기 때문에 중복이 될 수 있으며 , 확률적 뽑기 이므로 언제든지 parent는 달라질 수 있습니다. ('32752411'은 중복되었습니다.)

② Crossover

= (c)에서 만든 4개의 parent를 2개씩 쌍으로 하고 각각의 쌍에서 무작위로 자릅니다. 자른 부위(Crossover-Point)를 기준으로 뒷부분을 서로 교환합니다.

= Crossover 결과 4개의 문자열이 생성되고 이들 모두가 Offspring(자손)입니다.

- Crossover를 하지 않는 기존의 방식은 쿼리 하나만을 움직이지만 Crossover를 하면 쿼리를 하나 움직여서는 나타낼 수 없는 상태이므로 Search를 빠르게(여러 번) 진행 시킨 효과를 나타냅니다.

- Crossover를 할 때 Crossover-Point를 제대로 지정해야 합니다.

ex) 16257483의 각각의 값을 이진수로 변경시켜서 나타냈을 때 16257483의 16은 001/110이므로 00/1110으로 나누면 올바른 분할이 아니므로 주의해야 합니다.

③ Mutation

= 자손에서 무작위로 어떤 값을 무작위 값으로 변화시킵니다. 확률적으로 선택하기 때문에 아무 것도 선택하지 않을 수 있습니다.

ex) '24752411' 은 돌연변이가 발생하지 않았습니다.

④ 반복

= 1) ~ 3)을 계속 반복합니다.

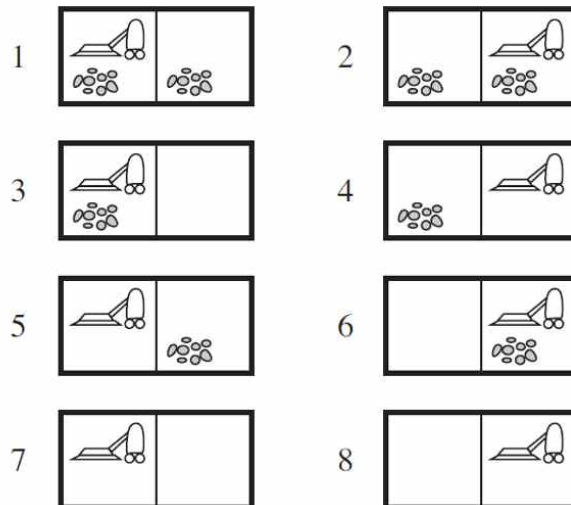
<4-3강_Non-Deterministic Problem> 11/07 ~ 11/08

- 지금까지는 Fully observable 하고 Deterministic한 문제들만 해결했습니다. 이제는 Partially observable 하거나 Non-deterministic한 문제를 해결해봅시다

1. Non-deterministic

- = Agent가 어떤 액션을 했을 때 결과가 하나만 나오지 않고 여러 개 나오는 경우 입니다.
- Action에 의해 여러 가지의 경우가 발생하므로 해당 Case 별로 모두 대비를 해야 합니다.
- 각각의 Case 마다 계획(=해결책)을 세우는 것을 Contingency Plan 이라고 합니다.

<Example : Vacuum Test>



- Initial_State : 1번

- Goal_State : 7번 OR 8번

- Action : Suck , Right , Left

<Deterministic>

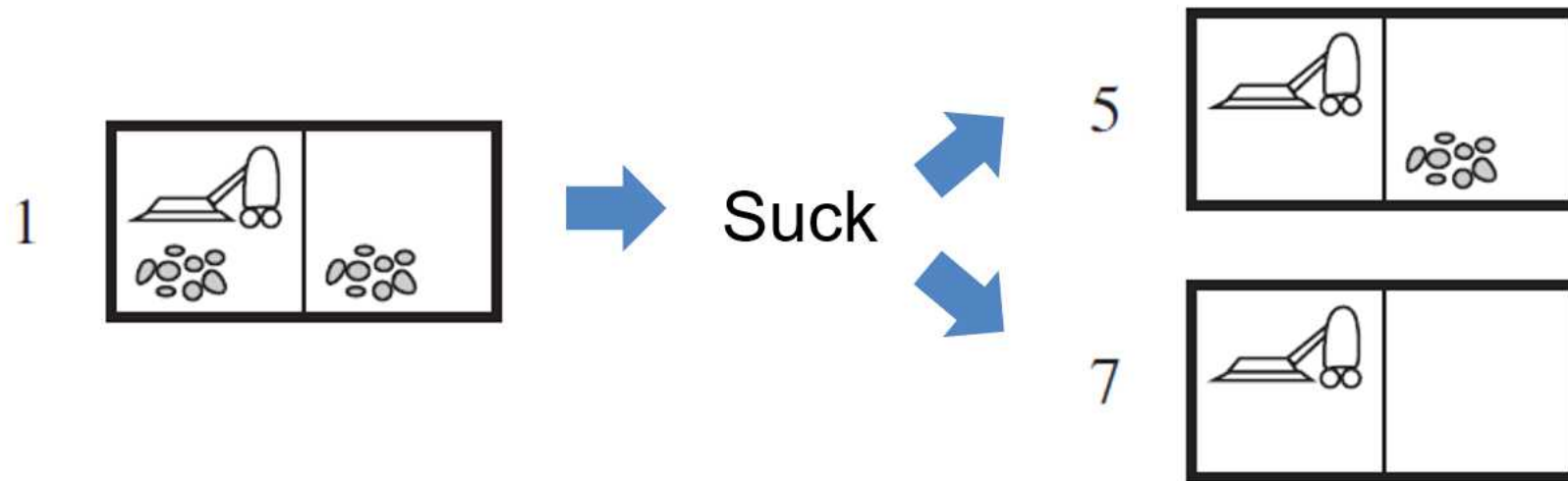
= 1 -> Suck -> Right -> 6 -> Suck -> 8(Goal)

1) The Erratic Vacuum World

= 'suck' action에서 non-deterministic이 발생합니다.

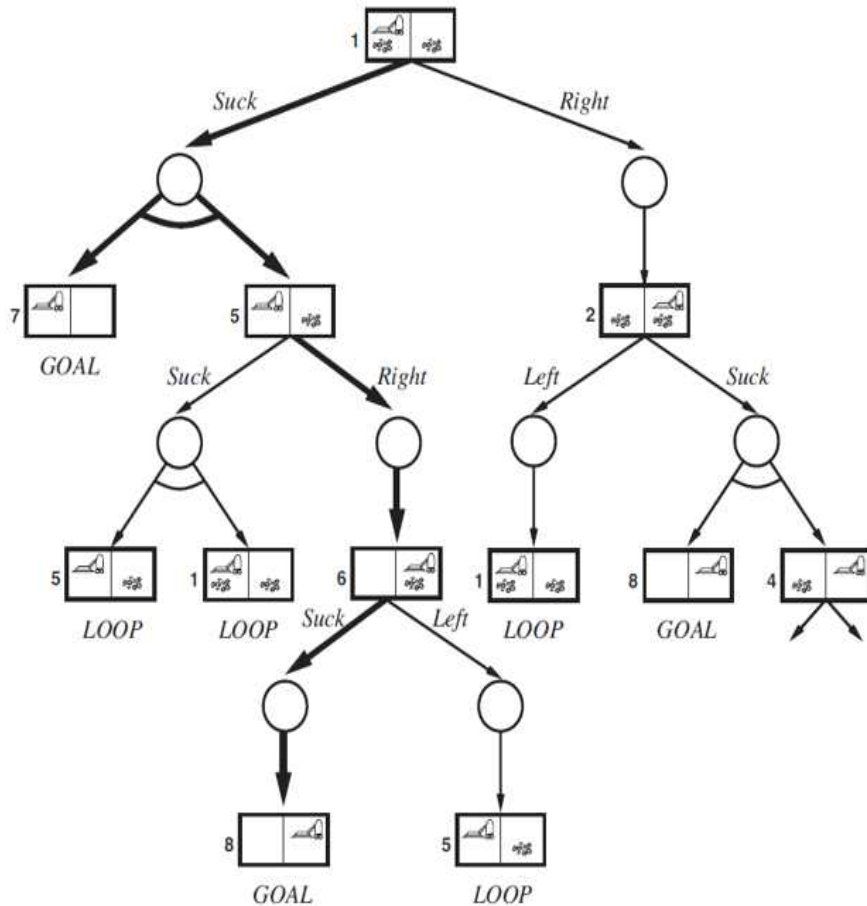
- ① Left_Room만 청소했는데 Right_Room까지 청소가 될 수도 있습니다.
- ② 깨끗한 방을 청소 하면 오히려 그 방이 지저분해 질 수도 있습니다.

<Example : Vacuum Test에서 발생한 Non-Deterministic>



- (1 -> 5) OR (1 -> 7)중에서 어떤 State가 Action의 결과가 될 지 알 수 없습니다.
- 그러므로 1에 있을 때 contingency plan(계획)을 세워서 5로 가는 경우와 7로 가는 경우를 모두 대비합니다.
- AND-OR Search Tree를 사용해서 Contingency Plan을 구현합니다.

<AND-OR Search Tree>



<동그라미 노드>

= AND Node

= Solution을 정할 때 **모든 자식노드에 대한 계획을** 세웁니다.

ex) 1번 state에서 'Suck'을 했을 때 5번 state가 될 지, 7번 state가 될 지 알 수 없으므로 두 경우 **모두** 해당 state에 도달했을 때의 계획을 정합니다. 5번 state인 경우에는 'suck' 또는 'right' 액션을 하고, 7번 state는 goal state이므로 종료시킵니다.

<네모 노드>

= OR NODE

= Solution을 정할 때 **자식노드 1개만 선택**합니다.

ex) 1번 state에서 'suck' OR 'Right' 중에서 반드시 한 개를 선택해야 합니다.

- Solution은 언제나 Sub-tree가 됩니다. Sub-tree는 그림의 굵은 Branch처럼 전체 Tree의 일부분을 의미합니다.

- Goal은 언제나 Leaf에서 탄생합니다.

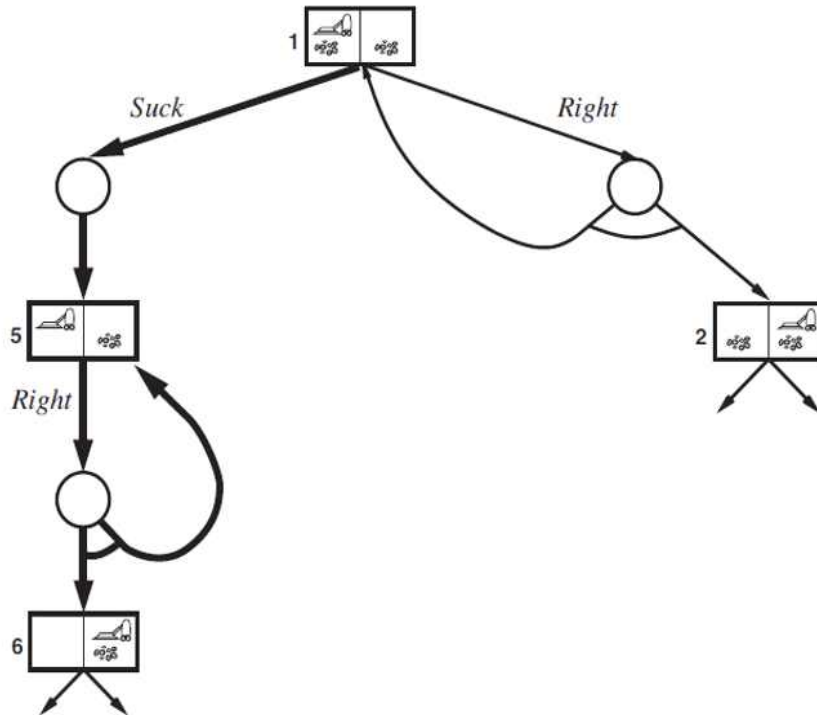
2) The Slippery Vacuum World

= Erratic Vacuum World의 Solution에는 cycle이 포함되지 않습니다. 하지만 Slippery Vacuum World의 Solution에는 반드시 cycle이 포함되어야 합니다.

= 이동하는 action(left, right)에서 non-deterministic이 발생합니다. 그 결과 Loop(=Cycle)가 발생합니다.

ex) Vacuum Test에서 Agent의 바퀴가 고장 났다고 가정합니다. 1번 state에서 right를 하면 기존에는 반드시 2번 state로 변경되었으나, Slippery Vacuum World에서는 2번으로 갈 수도 있지만 1번으로 다시 돌아올 수도 있습니다.

<Example : Vacuum Test에서 발생한 Non-Deterministic>



- 1번 state에서 right 액션을 하면 2번 state가 될 수 있지만 루프가 발생하여 1번 state로 되돌아 올 수도 있습니다.
- 5번 state에서 right 액션을 하면 6번 state가 될 수 있지만 루프가 발생하여 5번 state로 되돌아 올 수도 있습니다.

<Solution>

- 1 -> suck -> 5 -> right -> 6이 되면 Goal State이므로 종료.
- 1 -> suck -> 5 -> right -> 5가 되면 다시 right를 수행합니다.
- 6에 도달 할 때까지 계속 루프를 돌면서 right를 수행하면 됩니다.
- 5에서 6으로 언젠가는 state가 변한다는 것을 보장하면 괜찮지만 무한 루프를 수행하여도 계속 5로 돌아오는 문제가 발생할 수 있습니다. 그러므로 문제에서 발생하는 루프의 원인을 알아야 합니다.

<Example>

= 호텔에서 고장 난 카드키로 문을 계속 열려고 무한히 시도해보아도 문은 절대 열리지 않습니다.

<4-4강_No Observable Problem> 11/09 ~ 11/13

1. 용어

1) Physical States(P)

= Agent가 가능한 모든 States

- 기존의 문제에서 사용하던 States 개념을 Belief States와 구분하기 위해서 Physical을 접두사로 사용하였습니다.

2) Belief States(B)

An agent's current belief about the possible states it might be in

= 현재 Agent가 가능할 것이라고 믿는 모든 States의 집합

= Physical States에서 만들 수 있는 모든 부분 집합.

= Vacuum 문제에서 {1,2,3,4,5,6,7,8}, {2,4,6,8}, {1,7} 같은 집합이 모두 Belief States입니다.

- P가 N개면 B는 2^N 개입니다.

- 문제에서 모든 B를 사용하지는 않습니다.

ex) Vacuum Test에서 256개의 B가 나오지만 , 12개만 사용합니다.

2. Sensor-less Problem(=No observation Problem)

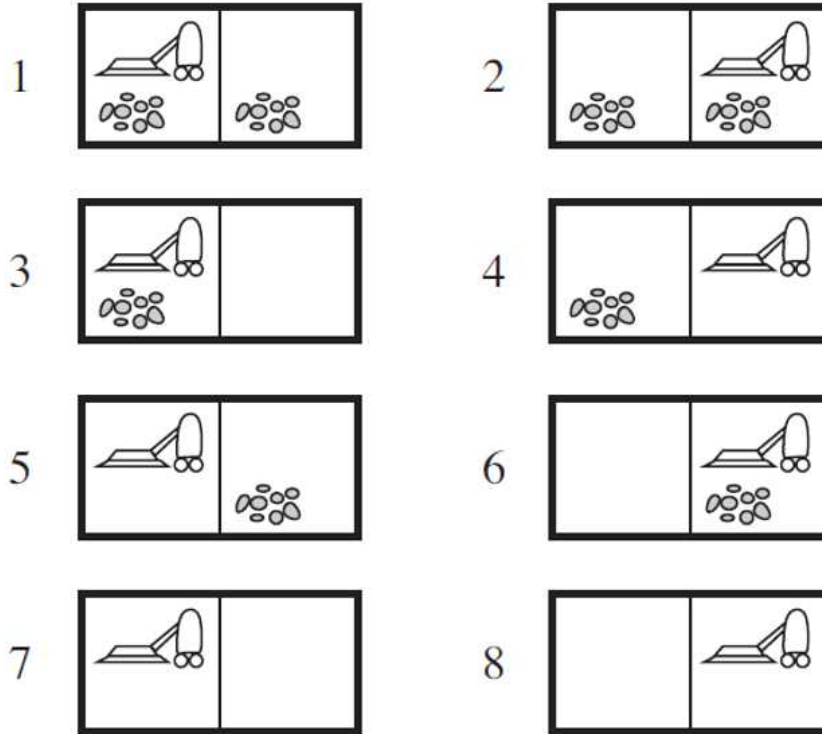
= Sensor가 없어서 percept가 없는 문제입니다.

- 현실세계에서 Sensor를 제작하는 것이 비싸기 때문에 Sensor 없이 해결하는 것이 비용과 효율성에서 뛰어납니다.

ex_1) 제품이 어느 방향을 가리키는지 카메라(Sensor)로 인식하고 방향을 바꿀 수도 있으나 , 카메라로 위치를 인식하지 않고 어떠한 경우에도 원하는 방향을 가리키게 할 수 있습니다.

ex_2) 전문적인 검사를 해서 전문적인 약을 지어줄 수 있지만 , 처음에는 대부분의 병에 효과가 있는 항생제 처방부터 합니다.

<Example : Dirt Sensor(먼지 유무 파악)는 존재하지만 Location Sensor(장소 파악)는 존재하지 않는 상황>



- 예제의 세상은 deterministic , partial observable 합니다.
 - Initial State도 알 수 없지만 {1,2,3,4,5,6,7,8} 중에 하나 일 것입니다.
 - 'Right'하면 Belief State가 {2,4,6,8}이 됩니다. Deterministic 하기 때문에 바퀴가 고장 난 경우는 고려하지 않습니다.
 - 'Suck'하면 Belief State가 {4,8}이 됩니다. Deterministic 하기 때문에 깨끗한 장소를 청소해도 먼지가 다시 발생하는 경우는 고려하지 않습니다.
- = 'Right -> Suck -> Left -> Suck' 단계로 액션을 취하면 무조건 Solution을 구할 수 있습니다.

3. Problem Using Belief States

- Sensor-less Problem의 경우 일반적인 States를 사용하지 않고 Belief States를 사용하기 때문에 문제를 새롭게 정의해야 합니다.
(P28과 비교합니다.)
- Vacuum 문제를 예시로 사용합니다.

1) Initial state

= Physical States의 집합.

ex) {1,2,3,4,5,6,7,8}

2) Actions

$$ACTIONS(b) = \bigcup_{s \in b} ACTIONS_P(s)$$

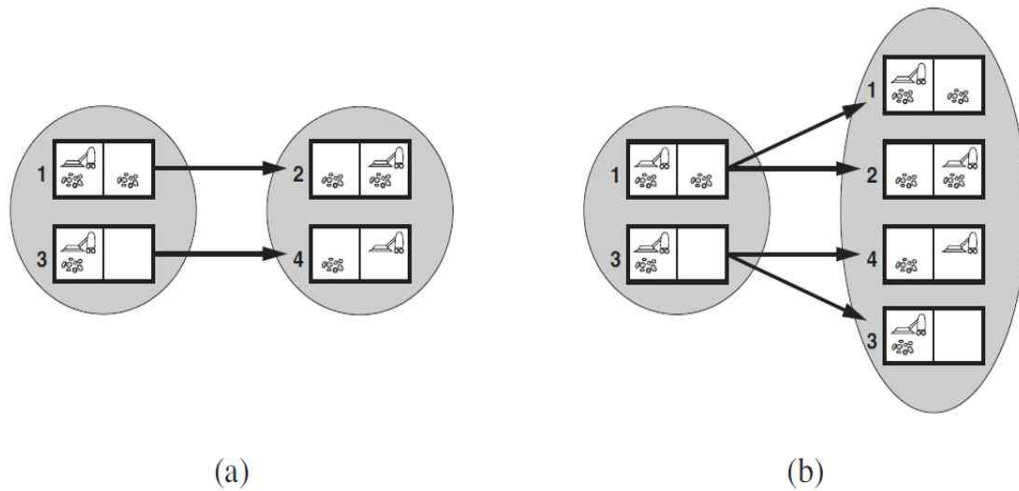
= Belief States에서 할 수 있는 모든 액션은 Belief States를 구성하는 states에서 할 수 있는 액션의 합집합과 같습니다.

3) Transition model

For deterministic actions, $b' = RESULT(b, a) = \{s' : s' = RESULT_P(s, a), s \in b\}$

For nondeterministic actions, $b' = RESULT(b, a) = \{s' : s' \in RESULTS_P(s, a), s \in b\}$

- a : action
- b , b' : belief state
- s , s' : state



① (a) : Deterministic

= 1,3이 있는 집합이 b(belief state)이고 2,4가 있는 집합이 b'입니다.

= 1이 s면 2가 s'이고 3이 s면 4가 s'이 됩니다.

② (b) : Non-Deterministic

= Slippery Vacuum World.

= 1이 s면 1,2가 s'이고 3이 s면 3,4가 s'이 됩니다.

4) Goal test

- Vacuum Test는 7과 8이 Goal입니다.
- {2,4,6,8}은 8만 goal이므로 goal test를 통과 하지 못합니다.
- {7,8}은 7과 8이 모두 goal이므로 goal test를 통과합니다.
- Belief State의 모든 원소(state)가 goal이어야 goal test를 통과합니다.

5) Path cost

- = 모든 문제의 cost가 같으면 쉽게 구할 수 있습니다.

4. Belief State Search

- Belief state를 사용해도 지금까지 배운 모든 Search Algorithm을 사용해서 문제를 해결할 수 있습니다.
- Graph search의 성능이 증가할 가능성이 있습니다.

<Belief State를 사용하면 Graph Search의 성능이 증가할 수 있는 이유>

- Belief State를 {1,3,5,7}이라고 가정하고 이 belief state를 b라고 하겠습니다. Agent가 b를 통과해서 goal로 갈 수 있다면 b에 존재하는 모든 state가 goal state여야 합니다. 그러므로 b의 부분집합인 b'(ex : {3,5})도 b'에 존재하는 모든 state가 goal state입니다.
- 위의 내용을 명제로 해석하면 , '{1,3,5,7} is True(goal) -> {3,5} is True(goal)'가 성립합니다.
- **대우 명제**인 '{3,5} is False(not goal) -> {1,3,5,7} is False(not goal)' 또한 성립합니다.
- 기존의 graph Search에서는 Frontier에 {3,5} 있으면 중복해서 {3,5}를 Frontier에 넣지 않습니다. 그런데 명제를 이용하면 {3,5}가 있으면 {1,3,5,7} 또한 Frontier에 넣지 않아도 됩니다. 그러므로 **Frontier에 넣는 State가 적어지므로 Search도 더 적게 할 수 있게 됩니다.**

5. Sensor-less Problems의 문제점

1) 문제점

= **Belief state의 크기가 너무 큼니다.**

<Example : 100개의 방을 청소해야 하는 Vacuum>

- 100개의 방을 청소할 때 필요한 search는 100×2^{100} 이 됩니다.
- 10^{32} 를 모두 표시해야 하는데 10^{32} 비트가 필요하므로 , 현재의 컴퓨터(64비트)로는 불가능합니다.

2) 해결책

= Incremental belief-state search

ex) Vacuum이라면 1번방부터 문제점을 찾아보고 , 그 다음 2번방부터 문제점을 찾아보고 ...

- 그러나 8-puzzle같은 문제처럼 sensor가 없으면 절대 풀 수 없는 문제가 현실에는 많이 존재합니다.

<4-5강_Partially Observable Problem> 11/13

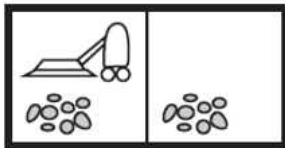
1. 4-4강과의 차이

- 4-4강의 No observable Problem은 문제를 해결하는데 sensor가 전혀 존재하지 않아 아무런 percept를 agent가 받지 못합니다.
- 하지만 4-5강의 Partial Observable Problem은 문제를 해결하는데 일부 sensor를 주어주기 때문에 어느 정도의 percept를 agent가 받게 됩니다.

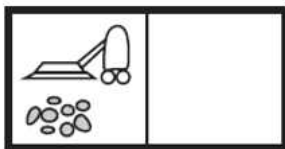
<Example : Vacuum Test>

- 진공청소기가 원래는 2개의 Sensor(위치, 먼지)를 갖습니다.
- 하지만 이 환경에서는 먼지 유무 파악 sensor만 갖습니다.
- Percept의 형태가 [A, Dirty]로 주어집니다.

1



3



- [A, Dirty] 라면 1번 state OR 3번 state 입니다.

2. 5단계로 문제를 정의합니다. (P28과 비교합니다.)

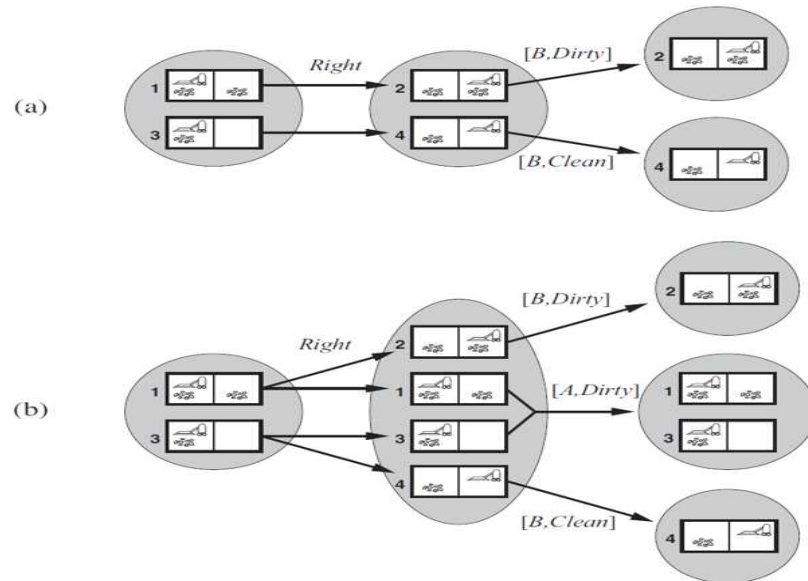
1) Initial state

= 주어진 정보가 줄어 들었기 때문에 Fully Observable 보다 적은 정보를 갖는 Initial State가 됩니다.

2) Action

= No-observation과 동일합니다.

3) transition model



= Search Tree를 만들어 나가는 과정에서 그 다음 State를 생성할 때마다 Prediction -> Observation prediction -> Update 과정을 거칩니다.

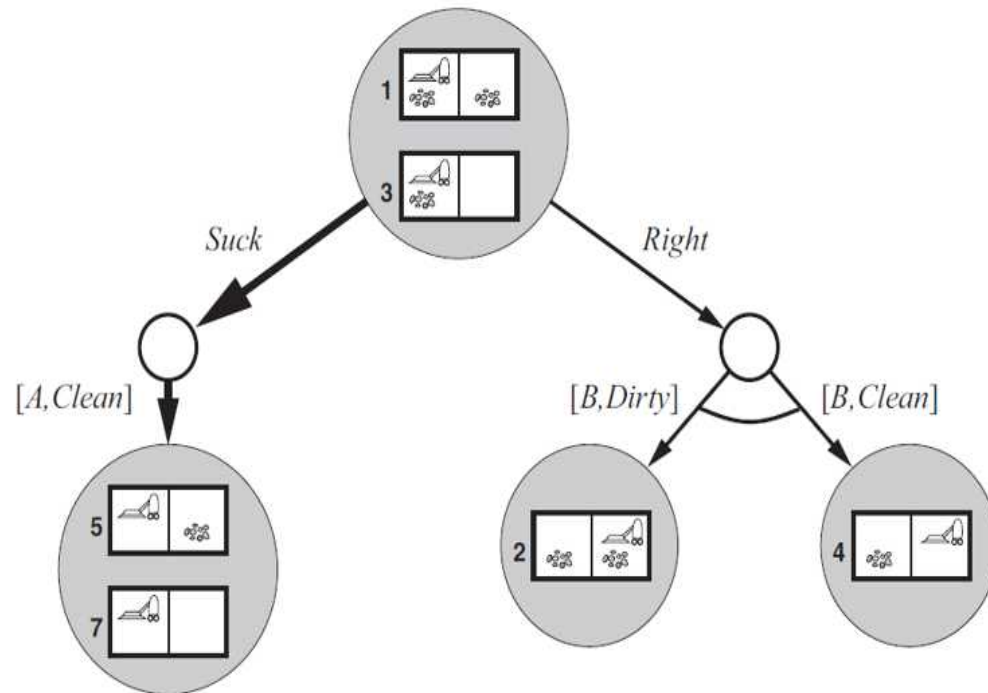
- (a) : Right를 하면 belief-state로 {2,4}가 생성됩니다.

no-observable problem에서는 percept가 없으므로 belief-state만을 이용했지만 partially-observable problem에서는 dirt percept가 주어지므로 percept에 따라 state를 2 OR 4로 확정 지을 수 있습니다.

- (b) : non-deterministic 중에서 Slippery 문제입니다. Belief-state {2,1,3,4}에서 (a)와 마찬가지로 주어진 dirt percept에 따라 state를 2 OR {1,3} OR 4로 확정 지을 수 있습니다.

4) Goal Test 와 Path Cost는 설명하지 않으셨습니다.

3. AND-OR Search

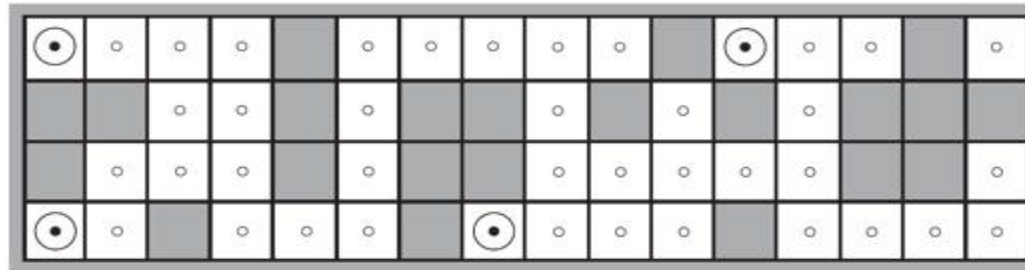


- Initial Percept = [A,Dirty]이므로 Initial Belief State는 {1,3}이 됩니다.
- Suck을 하고 [A,Clean]이 Percept로 주어지면 {5,7}이 belief state가 됩니다.
- Right를 하고 [B,Dirty]이 Percept로 주어지면 {2}가 belief state가 되고 [B,Clean]이 Percept로 주어지면 {4}가 belief state가 됩니다.

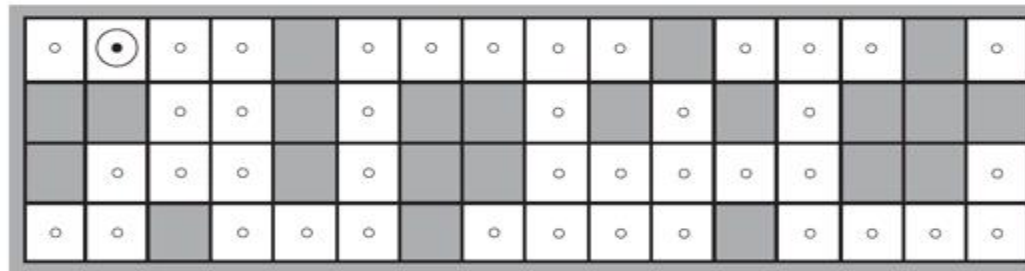
4. Maintaining One's Belief State

- Robot의 핵심기능은 belief state를 유지하는 것이고 이를 Monitoring , Filtering , State Estimation , Localization 이라고 불립니다.

<Example : Robot's Localization>



(a) Possible locations of robot after $E_1 = \text{NSW}$



(b) Possible locations of robot After $E_1 = \text{NSW}, E_2 = \text{NS}$

- 로봇은 초음파 sensor인 sonar를 이용하여 4방향의 벽의 유무를 파악합니다.
- 로봇의 E1(위치에 대한 첫 번째 검사 결과)은 NSW(north , south , west에 벽이 존재함)이므로 belief state는 4가지 state입니다.
- 로봇은 E1에서 right 액션을 취합니다.
- 로봇의 E2(위치에 대한 두 번째 검사 결과)는 NS(north , south에 벽이 존재함)이므로 기존의 belief state(4가지 state)에서 검사를 해보면 가능한 state는 오직 1가지입니다.

<5-1강_Games> 11/14

1. Games with AI

1) 게임의 특징

*** 여기서 설명하는 특징은 바둑이나 체스 같은 종류의 게임에 대한 특징입니다. ***

① multi-agent(2-Player)

= 대부분의 게임은 adversary(적)가 존재하며 , 협력 OR 경쟁(competitive) 구도 입니다.

② Contingency(우연성) 발생

= 다음의 상황을 예측할 수 없기 때문에 우연성이 존재합니다.

③ Deterministic

④ Turn-Taking

= 돌아가면서 각자의 턴 마다 수행합니다.

⑤ Zero-Sum

= 둘 다 이기거나 둘 다 지는 경우가 없습니다. 반드시 한 명이 이기거나 한 명이 지거나 비겨야 합니다.

⑥ Perfect Information

= Full-Observable

<Opposite>

- Non-deterministic : 주사위 게임
- Turn-Taking의 반대인 실시간 게임 : LOL , 배틀 그라운드
- Perfect Information의 반대로 한정된 정보가 주어지는 게임 : 포커

2) AI로 게임 연구가 어려운 이유

= Search space가 매우 큼니다.

ex) Chess의 branching factor(한 번에 취할 수 있는 액션)가 보통 35개이고 게임 종료까지 플레이어 당 평균 50번의 턴을 사용합니다, 그러므로 단순 계산해도 35^{100} 개의 노드가 생기게 됩니다. 이 수치는 현실에서 계산하기 어려운 크기입니다.

2. Game Search Problem 정의 (ai study_p28 참고)

- Player 2명을 각각 MAX(Agent = 나), MIN(Agent = 상대)으로 하겠습니다.
- 기존에 없었던 개념에 중요 표시를 하였습니다.
- 6가지의 개념을 정의하면 Game Tree를 만들 수 있습니다.

1) Initial State

2) PLAYER(s)

= 어떤 플레이어의 차례인지 나타냅니다.

ex) MAX OR MIN

3) ACTIONS(s)

= Player가 정해지면 할 수 있는 액션입니다.

ex) Chess : 어떤 말을 어디로 움직이는지

4) RESULT(s,a)

= Transition model과 동일합니다.

= 현재 state에서 액션으로 다음의 state가 생성

5) TERMINAL-TEST(s)

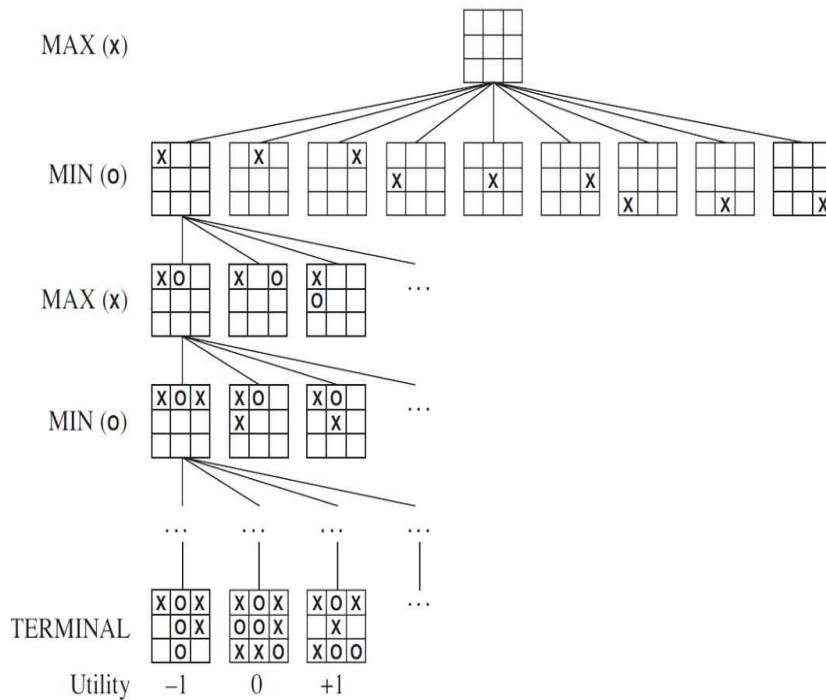
= Goal-Test와 유사합니다.

= 게임이 끝났을 때의 State를 Terminal State라고 합니다.

6) UTILITY(s,p)

= 게임이 끝난 상태에서만 정의되는데 게임이 끝나면 p(player)에게 어떠한 값을 주는지 나타냅니다.

3. Game Tree_1 : Tic-Tac-Toc



- 체스나 바둑보다 훨씬 간단한 게임입니다.
- Depth가 1이면 9가지의 state가 존재합니다.
- Depth가 2이면 8가지의 state가 존재합니다.
- 가장 깊은 Depth라면 1가지의 state가 존재합니다.
- 위에 기술한 Depth 별 State는 최악의 경우이고 , 최악의 경우 $9! = 362,880$ 개의 노드가 생깁니다.
- Utility -1 : 나의 패배 // Utility 0 : 무승부 // Utility 1 : 나의 승리

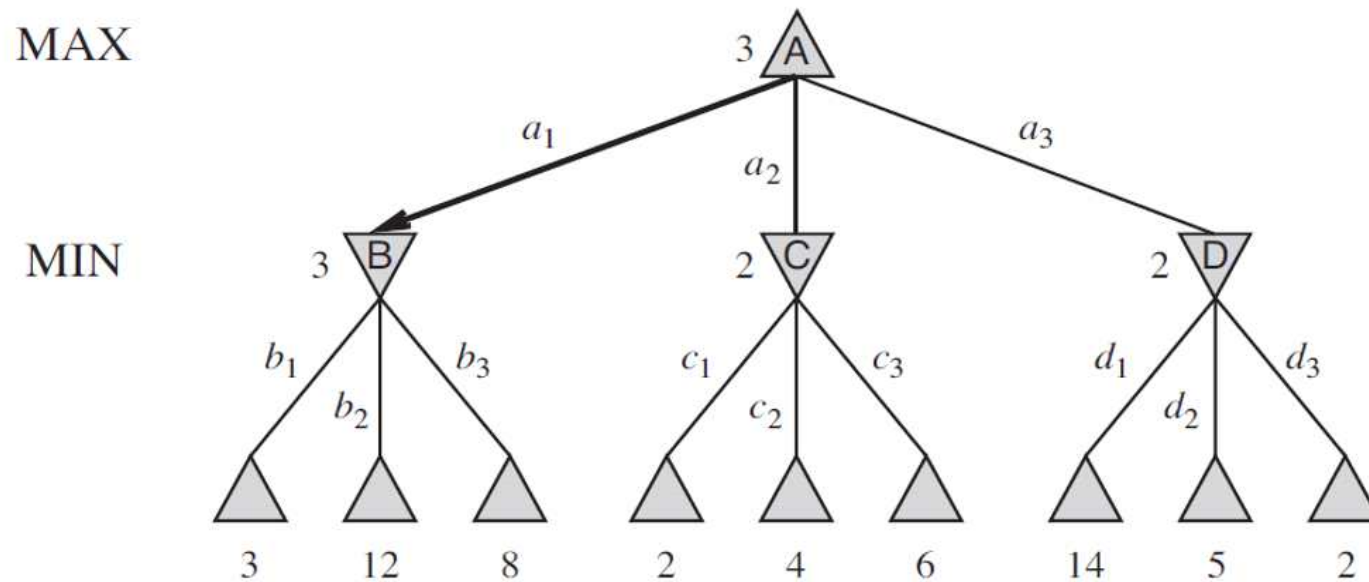
<MAX가 어떻게 하면 Optimal Decisions를 통해 승리할 수 있을까?>

= MAX가 initial-state일 때 턴이라면 9개의 노드 중에서 어떤 노드를 골라야 최선의 선택일까?

= MIN이 어떤 노드를 골랐을 때 그 다음 턴에서 MAX는 어떤 노드를 골라야 최선의 선택일까?

- 위의 질문에 대해서 MAX는 미리 Optimal Decisions를 정해놓고 MAX의 턴일 때 Optimal Decisions로 찾은 노드를 선택해야 합니다. 이는 AND-OR Search와 유사합니다.

4. Game Tree_2 : Two-Ply



- MAX와 MIN이 action을 각각 1번만 수행하면 끝나는 매우 간단한 게임입니다.
 - MAX는 자식 노드에서 가장 큰 숫자를 고르고, MIN은 자식 노드에서 가장 작은 숫자를 고릅니다.
 - 최하단의 숫자 9개(3,12,8,2,4,6,14,5,2)는 utility 입니다.
 - Optimal decision을 결정(Solution 찾기)하려면 각 노드마다 MINIMAX(n)값을 정해야 합니다.
 - 그러나 일반적인 게임에서 MINIMAX를 모두 구하는 것은 불가능합니다.
- ex) MAX가 A에서 a2로 C를 골랐습니다. 그 후 MIN이 C에서 action을 취해야 하는데 이 때 utility를 구해보면 2, 4, 6 이므로 MIN이 이기려면 작은 utility 값을 선택해야 합니다. 그러므로 c1액션을 취하고 MINIMAX는 2가 됩니다.

<5-2강_Game Algorithm> 11/16 ~ 11/18

1. MINIMAX

1) MINIMAX 구하는 방법

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

<MINIMAX>

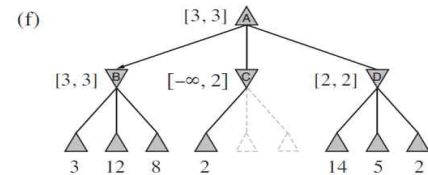
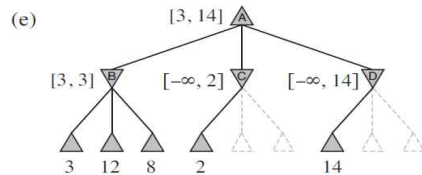
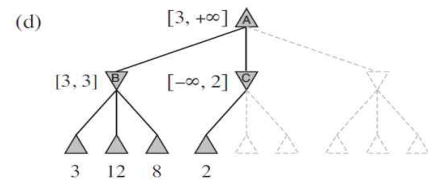
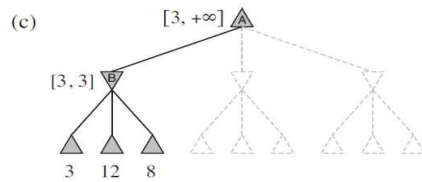
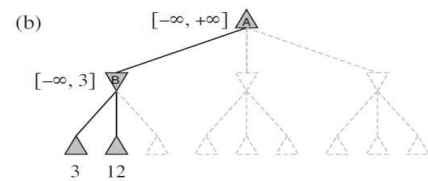
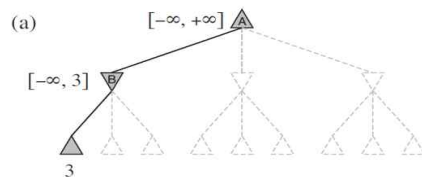
- = 노드 n부터 시작해서 두 플레이어가 최적화로 했을 경우 게임이 끝났을 때 얻는 **utility 값**
- Terminal State라면 MINIMAX는 자신의 UTILITY가 됩니다.
- Terminal State가 아니라면 **Bottom-Up을 통해 재귀적**으로 구할 수 있습니다.
- 가장 깊은 깊이의 바로 위는 MIN의 차례입니다. MIN은 하위 depth의 utility중에서 가장 작은 값을 선택해야 하므로 , B : 3 , C : 2 , D : 2를 고르게 됩니다.
- 그 다음 Depth는 Root이고 MAX의 차례이므로 하위 depth의 utility중에서 가장 큰 값을 선택해야 하므로 A : 3을 선택하게 됩니다.
- 그러므로 **Two-Ply에서 A에서 MAX는 a1을 통해 B를 선택하고 B에서 MIN은 b1을 통해 utility가 3인 노드를 선택하는 것이 Optimal Decision 이 됩니다.**

2) MINIMAX 알고리즘의 문제점

- = Search 방법이 DFS와 유사하기 때문에 시간복잡도가 $O(b^m)$ 이 됩니다. 그러므로 시간복잡도가 매우 큽니다.

2. Alpha Beta Pruning

<Example : Two-Ply Game>



- (a) : A는 아무런 정보가 없으므로 $[-\text{INF}, +\text{INF}]$ 사이의 값을 가질 수 있습니다. B도 처음에는 $[-\text{INF}, +\text{INF}]$ 지만 탐색을 하여 utility가 3인 노드를 만나면 $[-\text{INF}, 3]$ 이 됩니다. MIN은 나머지 노드에서 3보다 큰 노드라면 선택하지 않기 때문에 최댓값이 3이 됩니다.

- (b) : 다음 노드가 12이고, 이는 3보다 크므로 MIN은 선택하지 않습니다.

- (c) : 다음 노드가 8이고, 이는 3보다 크므로 MIN은 선택하지 않습니다. B는 $[3, 3]$ 으로 결정됩니다. 그 결과 A 또한 $[3, \text{INF}]$ 으로 결정됩니다. MAX는 3보다 큰 값을 원하기 때문에 depth_1에서 3보다 작다면 어차피 선택하지 않기 때문에 최솟값을 3으로 확보할 수 있습니다.

- (d) : 다음 노드가 2이므로 $[-\text{INF}, 2]$ 를 확보합니다. MIN은 2보다 큰 노드라면 선택하지 않을 것이고, 2는 어차피 MAX 기준 3보다 작기 때문에 나머지 2개의 노드는 탐색할 필요가 없습니다.(Pruning)

- (e) : 다음 노드가 14이므로 $[-\text{INF} \sim 14]$ 를 확보합니다. 14는 3보다 크므로 탐색을 더 진행합니다.

- (f) : 탐색 결과 MIN은 2를 선택하므로 MAX는 3을 선택하고 탐색이 종료됩니다.

1) MINIMAX vs Alpha Beta Pruning

- Alpha Beta Pruning MINIMAX 알고리즘의 성능을 개선한 알고리즘입니다.
- MINIMAX는 주어진 branch factor를 모두 검사하지만 , Alpha Beta Pruning에서는 굳이 검사하지 않아도 되는 branch factor는 스킵 하여 복잡도를 개선시킵니다.
- 확보 된 점수가 더 이상 상대방의 결정에 영향을 미치지 않는다면 관련 branch factor를 모두 탐색하지 않습니다,
- Alpha : MAX가 얻을 수 있는 최댓값
- Beta : MIN이 얻을 수 있는 최솟값

2) Mover Ordering

= Alpha Beta pruning이 효과적인 이유입니다.

- 만약 Two-Fly Game에서 MIN이 우측하단에서 utility가 2인 노드를 utility가 14인 노드보다 먼저 탐색했다라면 같은 depth의 나머지 노드를 탐색할 필요가 없게 됩니다. 이를 통해 우리는 자식 노드를 방문하는 순서에 따라 Alpha Beta Pruning의 정도가 차이가 남을 알 수 있습니다.
- 평균적으로 Alpha Beta Pruning을 하면 $b^m \rightarrow b^{(0.75m)}$ 정도로 감소합니다.
- 언제나 Best-Successor를 고른다고 가정하면 복잡도는 $b^m \rightarrow b^{(0.5m)}$ 로 감소합니다. 하지만 언제나 Best-Successor를 고르는 것은 현실적으로 불가능합니다. 그러나 이렇게 복잡도가 감소해도 체스의 경우 $10^{154} \rightarrow 10^{77}$ 이 되는 것이므로 아직도 너무 큰 복잡도를 가집니다.

3. Heuristic MINIMAX : Imperfect Real-Time Decisions

- 앞에서 배운 Alpha-Beta Search는 어느 정도의 terminal states까지는 도달을 해야 하지만 , 체스의 경우 도달하지 못하므로 체스는 Alpha-Beta Search로 구현할 수 없습니다. 만약 구현을 하더라도 실용적이지 못합니다. 그러므로 **terminal states 까지 탐색 하지 않고 어느 시점에 탐색을 중단하는 방법**인 Heuristic MINIMAX를 통해 해결합니다.
- 3장에서 배운 DFS의 단점을 Depth Limited Search로 해결하는 방식과 유사하며 Heuristic Algorithm의 성질과도 비슷합니다.
- Terminal State가 Cut-Off로 대체됩니다. 그러므로 Cut-Off에 도달하면 MINIMAX를 예측해서 계산합니다.

1) Evaluation Function(EVAL)

= **Cut-Off에 도달 했을 때 , 추정(heuristic)해서 계산한 utility**

- 다시 말해 , 일반적인 MINIMAX에서의 utility가 Heuristic MINIMAX에서는 Evaluation Function이 됩니다.

<Heuristic MINIMAX에 이용되는 Evaluation Function이 갖추어야 할 조건>

- ① 2개의 Terminal States를 각각 A , B 라고 했을 때 일반적인 MINIMAX에서 계산한 'utility(A) > utility(B)'가 성립하면 Evaluation function으로 추정한 'EVAL(A) > EVAL(B)'도 만족해야합니다. 다시 말해, Utility Function과 동일하게 terminal state가 정렬되어야 합니다.
 - ② 효율적인 계산 : 계산하는데 오랜 시간이 걸리면 EVAL의 가치가 떨어집니다.
 - ③ 2개의 Non-Terminal States를 각각 C , D 라고 했을 때 EVAL(C) > EVAL(D) 이면 C_State에서 이길 확률이 D_State에서 이길 확률 보다 커야합니다.
- 이 3가지 조건을 모든 State에서 완벽하게 만족하지는 않아도 됩니다. 대부분의 State에서 만족한다면 좋은 Evaluation Function입니다.

First, the evaluation function should order the *terminal* states in the same way as the true utility function: states that are wins must evaluate better than draws, which in turn must be better than losses. Otherwise, an agent using the evaluation function might err even if it can see ahead all the way to the end of the game. Second, the computation must not take too long! (The whole point is to search faster.) Third, for nonterminal states, the evaluation function should be strongly correlated with the actual chances of winning.

2) Example : Chess

- Feature는 State를 나타내는 특징입니다.

ex) 흰색 폰의 개수 , 검은색 폰의 개수 , 쿼의 개수 , etc

	10		-10
	30		-30
	30		-30
	50		-50
	90		-90
	900		-900

① 좌측 그림

- 숫자들은 해당 기물의 가중치(w)를 의미합니다.

② 하단 그림

- 아래 그림의 공식은 Evaluation Function을 구하는 공식입니다.

- 공식은 가중치를 포함한 Weighted Linear Function 입니다.

- w_i : 각각의 Feature의 가중치

- f_i : Feature

ex) w_1 은 하얀 폰의 가중치(10), w_2 는 검은 폰의 가중치(-10), etc

ex) f_1 은 하얀 폰의 개수 , f_2 는 검은 폰의 개수 , etc

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

<5-3강_State-of-the-Art Game Programs> 11/18

1. 체스

- 1997년에 Deep Blue가 인간 챔피언을 최초로 이겼습니다.
- Deep Blue는 서버컴퓨터 30개를 병렬식(parallel)으로 연결한 슈퍼컴퓨터이고 Alpha-Beta Search를 적용했습니다.
- Move Ordering을 잘하기 위해 480개의 특별한 processor 칩을 장착했습니다.
- Evaluation Function을 8000개의 features를 사용했습니다.
- **A large endgame database** : 기물이 얼마 남지 않았을 때 , 남은 턴에서 무조건 이기는 방법 또한 장착했습니다.

2. 바둑

- 2016년에 알파고가 대한민국의 이세돌 선수를 이겼습니다.
- Monte Carlo Tree Search + Deep neural networks를 사용했습니다.

<6-1강_Constraint Satisfaction Problems> 11/20 ~ 11/26

1. Constraint Satisfaction Problems(CSP)

= 제약 충족 문제

= 특정한 상황이 변수를 제약하는 상황에서 그러한 제약 조건을 모두 만족시키는 변수의 쌍을 구하는 문제

- 지금까지 배운 search algorithm에서의 state는 하나의 표현방법으로 나타내었습니다. 이번 강의에서는 다양한 표현방법으로 state를 나타내는 방식인 CSP를 배웁니다.

- **General Purpose Heuristic**(문제 마다 정의 되는 휴리스틱이 아닌 , 모든 문제에 범용 적으로 적용 되는 휴리스틱)이 존재합니다.

1) CSP의 정의

① X (Variable)

= 변수들의 집합

② D (Domain)

= 변수들이 갖는 값의 집합

③ C (Constraints)

= 제약조건의 집합

- C는 <scope , rel> 의 형태로 나타냅니다.

- Map Coloring의 예제에서는 위의 형태로 나타내지 않았습니다.

- scope는 제약조건에 참여하는 변수.

- rel은 relation의 약자이며 변수 간에 관계(컴퓨터수학2 에서 배움).

- 제약조건이 없는 CSP라면 , Search Space가 엄청 클 것입니다.

$\langle (X_1, X_2), \{(A, B), (B, A)\} \rangle$

- (X_1, X_2) : Scope
- $\{(A, B), (B, A)\}$: Relation
- 변수 X_1 이 A면 , X_2 는 반드시 B여야 합니다.
- 변수 X_1 이 B면 , X_2 는 반드시 A여야 합니다.
- 두 변수는 같은 값을 가질 수 없는 제약조건입니다.

2) CSP의 State

= 변수들 전부 혹은 일부에 값이 할당된 상태입니다.

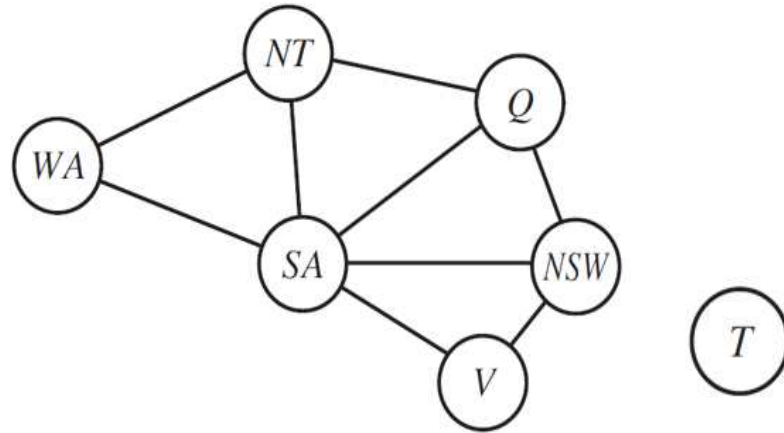
ex) 변수 X_1 , X_2 의 도메인이 (A , B)라 할 때 ' $X_1 = A$, $X_2 = A$ ' 인 경우나 ' $X_1 = B$, $X_2 = ?$ ' 인 경우 모두 State입니다.

① Legal(Consistent) Assignment : State들 중에서 모든 제약조건을 위반하지 않는 State.

② Complete Assignment : 모든 변수가 전부 값을 할당 받은 상태

- CSP의 solution은 legal assignment 와 complete assignment를 둘 다 만족한 것입니다.

3) Constraint Graph



= CSP를 그래프로 만든 것입니다.

- Vertex에는 변수를 할당하고, Edge에는 제약조건을 할당합니다.

- 그림은 'Example_1 : Map-Coloring'을 Constraint Graph로 나타낸 것입니다.

- 변수가 7개이므로 Vertex도 7개입니다.

- 제약조건이 9개이므로 Edge도 9개입니다.

- CSP로 만든 Constraint Graph를 사용하여 search를 하는 경우, general-purpose heuristic이 존재하기 때문에 기존의 Search Algorithm 보다 빠르게 Search 문제를 해결할 수 있습니다.

4) CSP의 종류

- ① 이산적인 변수와 유한한 도메인(대부분)
- ② 이산적인 변수와 무한한 도메인
- ③ 연속적인 변수

④ 제약조건은 여러 개로 나눌 수 있습니다.

- **Unary Constraints** : 변수 1개의 관계로 이루어진 제약조건
- **Binary Constraints** : 변수 2개의 관계로 이루어진 제약조건
- Ternary Constraints : 변수 3개의 관계로 이루어진 제약조건

Unary constraints involve a single variable,

- e.g., $SA \neq \text{green}$

Binary constraints involve pairs of variables,

- e.g., $SA \neq WA$

Higher-order constraints involve 3 or more variables,

- e.g., $SA \neq WA \neq NT$

5) CSP 예제

<Example_1 : Map-Coloring>



- 7개의 도시로 이루어진 나라를 3가지 색으로 칠하는 문제입니다. 단 , 인접한 도시끼리는 동일한 색으로 칠하면 안 됩니다.

① X : 7개의 도시

② D : 3가지 색

③ C : 인접한 지역(9개)은 동일한 색으로 칠하면 안 됩니다.

$$X = \{WA, NT, Q, NSW, V, SA, T\}$$

$$D_i = \{\text{red, green, blue}\}$$

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$$

<Example_2 : Cryptarithmic Puzzle>

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$

- 알파벳 마다 알맞은 숫자를 대입해서 수식을 성립시키는 문제입니다.

① X : 알파벳

② D : 자연수

③ C : 수식을 성립시키기 위한 기본적인 수학 제약조건

Variables: O, R, W, U, T, F, C₁, C₂, C₃

Domains: 0-9

Constraints

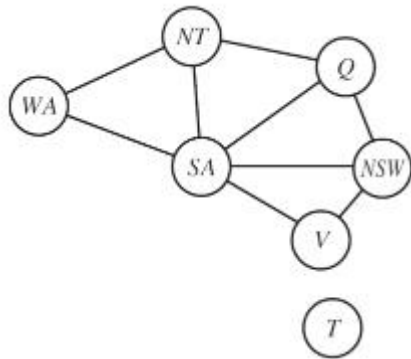
- $O + O = R + 10 \times C_1$, $T > 0$, Alldiff(F, T, U, W, R, O), ...

- Alldiff : 인자의 알파벳은 대입된 숫자가 모두 달라야 합니다.

- Constraint Hypergraph : 3개의 정점을 간선으로 묶어주는 정점을 만듭니다. 그 간선들이 hyper-edge이고 hyper-edge가 있는 그래프가 Constraint Hypergraph입니다.

2. CSP 해결 방법 : Search Space 줄이기

- CSP는 Search 와 Constraint Propagation(제약 확산)을 혼합하여 문제를 해결합니다.
- **Constraint propagation은 작업(제약 조건을 추가시켜 변수가 가질 수 있는 도메인을 줄임)을 통해 변수의 legal assignments(values)를 줄여서 Search Space를 줄이는 방법입니다.**
- Constraint propagation을 search 이전에 할 수도 있고 , search 중간에 할 수 도 있습니다.



- CSP 해결 방법을 설명할 때 모두 사용하는 Map Coloring 예제
- All_Diff는 모든 예제에서 만족합니다.

1) Local Consistency(지역적 일관성)

- Constraint propagation의 가장 핵심적인 아이디어 입니다.

① Node Consistency

= 어떤 변수가 **unary constraint**를 모두 만족하면 그 변수를 **node consistency** 라고 합니다.

<Example>

- SA 도시는 원래 3가지 색상을 칠 할 수 있지만 , 초록색은 칠하지 못하도록 unary constraint를 설정하였습니다.
- SA의 도메인을 {red , green , blue}로 설정하면 node consistency를 만족하지 않지만 , {red , green}으로 설정하면 node consistency를 만족 합니다.

② Arc Consistency

X_i is arc-consistent **with respect to** another variable X_j if for every value in D_i there is some value in D_j that satisfies the binary constraint on the arc (X_i, X_j)

- 변수 X_i 변수 X_j 에 대해서(방향성이 존재함) X_i 에 존재하는 모든 도메인 값이 각각 X_j 의 도메인 중에서 1개라도 binary constraint를 만족할 때 X_i 가 X_j 에 대해서 arc consistency 하다고 합니다.

- 예시를 통해 이해하는 것이 빠를 것이라고 생각합니다.

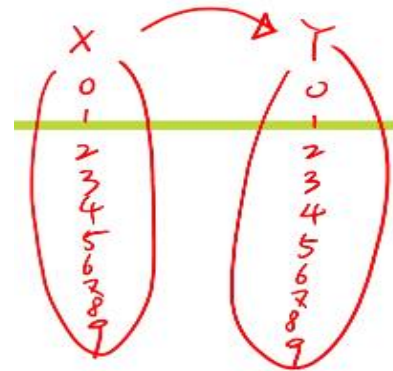
<Example>

Variables: X, Y

Domains: 0-9

Constraints: $Y = X^2$

- $\langle (X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\} \rangle$



- 변수 X 에 올 수 있는 모든 도메인에 대해서 만족해야 합니다. X 의 도메인인 2를 검사해보면 (2,4)이 변수 Y 의 도메인 중 4에 대해서 성립하므로 1개라도 변수 Y 의 도메인에서 성립하므로 변수 X 의 도메인인 2는 성립합니다. 그러므로 0부터 검사를 해보면 (0,0), (1,1), (2,4), (3,9)는 성립을 하지만 4부터 성립하는 도메인이 Y 에 존재하지 않습니다. 그러므로 X 에 대한 Y 의 arc consistency는 성립하지 않습니다. **Y 에 대한 X 의 arc consistency는 $Y \rightarrow X$ 이므로 완전 다른 개념입니다.**

- Arc consistency를 만족하지 않는 변수 X 의 도메인 4 ~ 9를 없앨 수 있기 때문에 search space를 줄일 수 있습니다. 하지만 arc consistency를 모두 만족하는 문제의 경우가 search space를 줄일 수 없기 때문에 arc consistency가 의미가 없게 됩니다.

③ Path Consistency

- = 변수_X , 변수_Y가 제 3의 변수_Z에 대해서 (X , Z) / (Z , Y)의 제약을 만족하는 경우
- Arc consistency를 사용해도 search space를 줄이지 못하는 문제를 해결하기 위해 고안된 방법입니다.

<Example : 2가지 색으로 Map Coloring>

- WA와 SA의 path consistency를 검사할 때 (WA , NT) / (SA , NT)을 검사합니다.
- WA가 red , SA가 green 일 때 NT에 green을 칠하면 SA와 같게 되고 , NT에 red를 칠하면 WA와 같기 때문에 칠할 수 없게 됩니다. 그러므로 path consistency를 만족하지 않습니다.

2) K-Consistency

- Node Consistency , Arc Consistency , Path Consistency를 모두 **일반화 한 Consistency**입니다.
- Node consistency가 1-Consistency이고 , Arc Consistency가 2-Consistency이고 , Path Consistency가 3-Consistency이므로 , 같은 원리에 따라 귀납적으로 일반화 시킬 수 있습니다.
- Strongly K-Consistent : 1-Consistency ~ K-1 Consistency를 모두 만족하는 Consistency입니다.
- N개의 변수를 갖고 n-consistent하면 $O(n^2 * d)$ 의 복잡도를 갖게 됩니다.

3) Global Constraints

= 임의의 변수(모든 변수 X)를 포함하는 제약 조건

ex) Alldiff

<Example>

- 3가지 색으로 칠해야 하고 , 이미 WA에는 red , NSW에는 red를 칠했습니다.
- NT , SA , Q끼리는 서로 다른 색을 가져야하지만 WA와 NSW가 red이므로 red는 칠할 수 없습니다. 그 결과 3개의 도시를 2가지 색상으로 모두 다르게 칠해야 하므로 말이 되지 않습니다.

<6-2강_Backtracking Search for CSP> 11/28

1. Backtracking

- CSP를 해결할 때 사용하는 방법입니다.
- 노드가 유망(Promising)한지 검사하며 DFS를 진행합니다.
- 유망하지 않는 노드는 가지 않고, 가장 깊은 depth 까지 도달했을 때 정답을 찾지 못하면 다시 올라와서 유망한 노드부터 다시 탐색합니다.

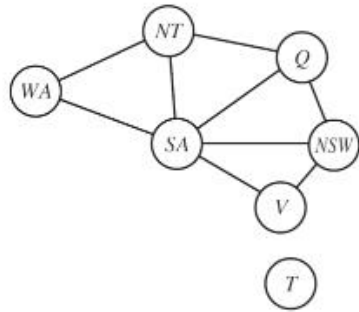
<Backtracking Algorithm>

function BACKTRACKING-SEARCH(*csp*) **returns** a solution, or failure
 return BACKTRACK(*{ }*, *csp*)

function BACKTRACK(*assignment*, *csp*) **returns** a solution, or failure
 if *assignment* is complete **then return** *assignment*
 var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(*csp*)
 for each *value* **in** ORDER-DOMAIN-VALUES(*var*, *assignment*, *csp*) **do**
 if *value* is consistent with *assignment* **then**
 add { *var* = *value* } to *assignment*
 inferences $\leftarrow$ INFERENCE(*csp*, *var*, *value*)
 if *inferences* $\neq$ failure **then**
 add *inferences* to *assignment*
 result $\leftarrow$ BACKTRACK(*assignment*, *csp*)
 if *result* $\neq$ failure **then**
 return *result*
 remove { *var* = *value* } and *inferences* from *assignment*
 return failure

2. 선택

- Backtracking을 진행하는 과정에서 선택의 갈림길에 놓이게 되는데 어떠한 것을 선택할지 heuristic 하게 정해놓았습니다.



- 3가지 색으로 모두 다르게 칠하기.

1) Variable Ordering

= 어떤 노드를 선택해서 Search를 진행할지 정해야 합니다.

- Backtracking Algorithm에서 'select unassigned variable'에 해당 하는 부분입니다.
- 2가지 모두 heuristic 이므로 , 언제나 가장 답을 빨리 발견하는 것은 아닙니다.

① Minimum-Remaining-Values(MRV)

ex) WA는 red로 칠하고 , NT는 green으로 칠했을 때 다음으로 칠할 노드를 5개의 remaining node 중에서 어떤 것을 골라야하는지 정해야 합니다. SA는 무조건 blue / Q는 red OR blue / NSW , V , T는 3가지 색 모두가 가능합니다. 이런 경우 가능한 도메인이 가장 적은 SA를 다음 노드로 선택하여 search를 진행합니다.

② Degree heuristic

= 아직 아무 색도 칠하지 않아서 모든 노드가 3가지 색을 칠할 수 있을 때 MRV를 할 수 없게 됩니다. 그런 경우 차수가 가장 높은 노드를 선택해서 search를 진행합니다.

ex) SA가 차수가 5이므로 , SA를 선택합니다.

2) Value Ordering : Least-Constraining-Value(LCV)

<Example>

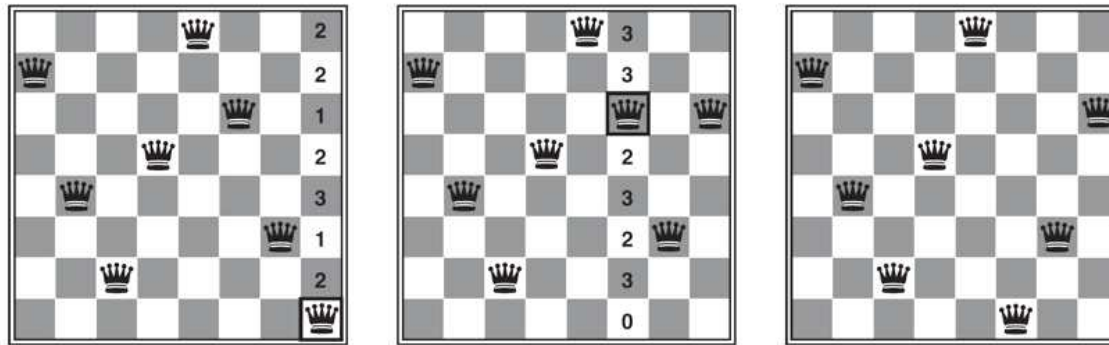
WA는 red를 칠하고 , NT는 green을 칠하고 다음에 탐색할 노드로 Q를 선택했습니다. Q에서는 red OR blue를 칠할 수 있는데 어떤 색부터 칠해야 하는지 정해야합니다.

- 인접한 remaining-node가 칠할 수 있는 색이 많은 것(제약을 적게 갖는 것)을 고릅니다.
- Red 칠하기 : SA는 green OR blue , NSW는 green OR blue
- Blue 칠하기 : SA는 X , NSW는 red OR green
- blue 고르면 sa는 x , nsw r,g
- Remaining-node 들이 칠할 수 있는 경우가 많은 Red를 고릅니다.

<6-3강_Local Search for CSP> 11/30

- 4-1강에서 배운 Local Search를 CSP를 해결하는데 사용해봅시다.

<Example : N-Queen>



- Initial state : complete 방식이므로 , 64개의 위치(노드)중에서 랜덤으로 선택하여 8개의 퀸을 놓습니다.

① 그림_1

- 8개의 퀸 중에서 constraint를 위반(다른 퀸을 죽이는)한 노드를 고릅니다. 다른 퀸을 죽이지 않는 위치에 있는 퀸은 선택하지 않습니다.
- (7,7)의 퀸을 선택하면 그 퀸은 7개의 위치로 이동이 가능합니다.
- 제약 조건(가장 작은 값을 갖는 위치로 이동하여야 함)을 위반하지 않는 위치로 이동하므로 값이 1인 위치로 이동시킵니다.
- 값이 1인 위치가 2개이므로 둘 중에 맘에 드는 위치를 선택합니다. 그림에서는 위에 있는 위치를 선택했습니다.

② 그림_2 + 그림_3

- 그림_1과 마찬가지로 8개의 퀸(이전에 옮긴 퀸 포함) 중에서 constraint를 위반(다른 퀸을 죽이는)한 노드를 고릅니다.
- (2,5)의 퀸을 선택하면 0인 위치가 발생하므로 해당 위치로 이동시키면 solution을 찾게 됩니다.

- N-Queen을 이런 방식으로도 해결 할 수 있습니다.

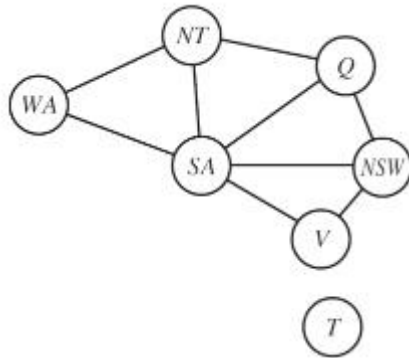
<6-4강_Structure of CSP> 11/30 ~ 12/02

*** 몇 가지 기술을 이용하여 CSP 문제를 빠르게 해결해봅시다. ***

1. CSP 빠르게 해결하기_1 : Independent Subproblems

= Connected Components(연결 요소) 별로 문제를 나눠서 해결하는 방법.

<Example : Map-Coloring>



- 도시_T는 다른 6개의 도시와 constraint가 없기 때문에 독립적으로 떨어져있습니다. 그러므로 도시_T는 3가지 색 중에서 아무거나 칠해도 무방합니다.
- 독립성을 이용해 6개의 도시 + 1개의 도시를 푸는 문제로 분해하여 해결할 수 있고 그 결과 search space를 줄일 수 있습니다.
- 6개의 도시 집합과 1개의 도시는 각각 연결요소가 됩니다.
- 1개의 그래프 문제를 2개의 연결요소로 나누어서 푸는 문제로 변경하였습니다.
- Independent Subproblems는 매력적이지만 현실세계에서 절대 흔하지 않습니다.

2. CSP 빠르게 해결하기_2 :Tree-Structured CSP

= 그래프 내부에 임의의 두 노드를 선택했을 때 , 항상 두 노드 사이에 경로가 1개라면 그 그래프는 트리가 됩니다. 트리인 연결요소에서 진행하는 CSP를 Tree-Structured CSP라고 합니다.

- 노드 1개로 구성된 그래프도 Tree-Structured CSP 입니다.
- Tree-Structured CSP 문제는 선형 시간(Linear)으로 계산가능
- Tree-Structured CSP에서는 노드 간에 순서가 존재합니다.
- Tree-Structured CSP에서 위상정렬을 통해 노드 간에 순서를 결정하고 , 순서대로 DAC를 검사합니다.

<Directed Arc Consistency (DAC)>

① 교재의 정의

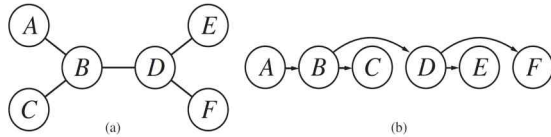
= If every X_i is arc-consistent with respect to each X_j for $j > i$ given an ordering $X_1, X_2, \text{etc}, X_n$

② 내가 정리한 정의

= 노드 간에 하나의 방향으로 방향성을 갖는 arc consistency



3. Tree-Structured CSP 해결 과정



- 그림_(a)의 그래프는 그래프에서 임의의 노드 두개를 잡아도 두 노드 사이에는 경로가 하나만 존재하므로 트리 구조입니다.

1) Root Node 결정

= 트리의 임의의 노드를 선택해서 Root로 결정합니다. 어떠한 노드를 Root로 선택해도 결과는 동일합니다.

- 노드_A를 Root로 선택했습니다.

2) 위상정렬

= Tree-Structured CSP에서 노드의 순서가 존재하므로, 노드의 순서를 결정합니다.

- 위상정렬의 시간복잡도 : $O(n)$

- 위상정렬로 순서를 결정했으므로 DAC를 검사해야 합니다.

3) DAC 검사

= 위상정렬에서 정한 순서를 따라서 arc-consistency를 검사합니다.

① A→B arc-consistency 검사, 만족하지 않은 도메인을 제거합니다. $O(A \text{의 도메인의 개수}^2)$

② B→C arc-consistency 검사, 만족하지 않은 도메인을 제거합니다. $O(B \text{의 도메인의 개수}^2)$

③ B→D arc-consistency 검사, 만족하지 않은 도메인을 제거합니다. $O(B \text{의 도메인의 개수}^2)$

④ D→E arc-consistency 검사, 만족하지 않은 도메인을 제거합니다. $O(D \text{의 도메인의 개수}^2)$

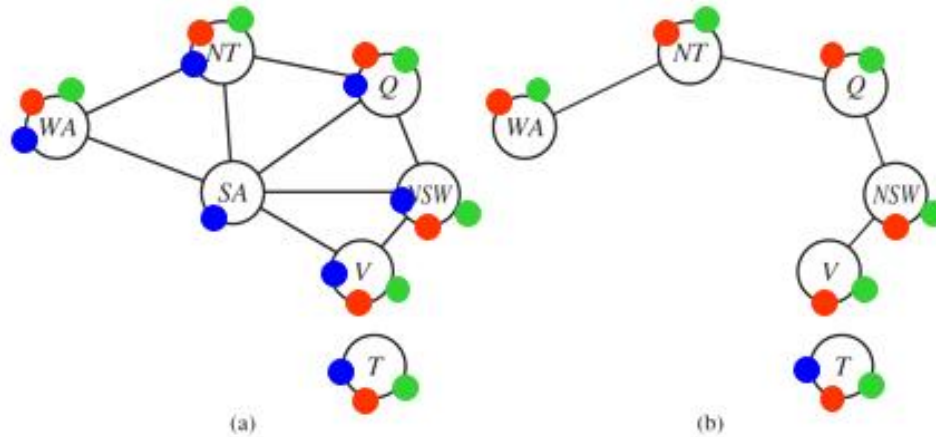
⑤ D→F arc-consistency 검사, 만족하지 않은 도메인을 제거합니다. $O(D \text{의 도메인의 개수}^2)$

4) A의 남아있는 도메인 중에서 임의의 도메인을 골라서 A→B, B→C, B→D, D→E, D→F(DAC 검사로 인해 해당 관계의 노드는 반드시 존재함)를 모두 해서 알맞은 값이 존재하면 Solution 입니다.

4. 트리가 아닌 그래프를 Tree로 변경하여 해결하기

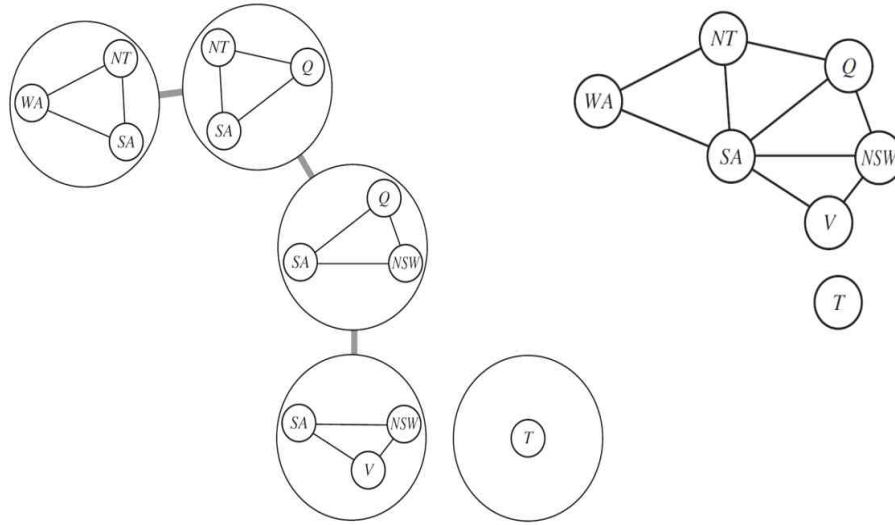
- Tree-Structured CSP는 매우 훌륭한 복잡도를 갖지만 트리 구조가 아니면 적용할 수 없는 문제입니다.
- 트리 구조가 아닌 그래프를 작업을 통해 트리로 변형시켜서 CSP를 쉽게 해결할 수 있도록 합니다.

1) 트리로 변경 방법_1 : 노드 제거



- 그림에서는 SA를 제거하여 트리구조를 만들었습니다.
- 그래프에서 어떤 노드를 제거했을 때 그래프가 트리 구조가 된다면 해당 노드를 **Cycle Cutset** 이라고 부릅니다.

2) 트리로 변경 방법_2 : 노드 모으기



- 노드들을 모아서 원 안에 집합으로 만들면, 그 집합의 노드가 하나의 노드가 됩니다. 집합으로 이루어진 원(노드)들은 트리구조를 이룹니다.

<원 집합을 만들 때 주의사항>

1) 오리지널 그래프의 변수가 트리에서도 반드시 존재해야 합니다.
2) 오리지널 그래프에서 연결된 변수들은 트리에서도 연결되어야 합니다.

3) 두 개의 원 안에 공통된 변수가 존재하면 두 원 사이의 원 안에도 그 변수가 존재해야 합니다.

ex) 1번에도 SA가 있고 4번에도 SA가 있으므로 그 사이의 2번 3번에도 SA가 존재합니다.

<7-1강_Knowledge-Based Agents> 12/04

1. Knowledge

- = 지식은 knowledge representation language를 사용하여 Sentence(문장)로 나타냅니다.
- 이전의 챕터(3강, 4강, 6강)에서는 전체에 공통적으로 정의되는 지식이 표현되지는 않는 문제를 해결했습니다.
- N-puzzle 문제에서 '한 퍼즐이 여러 개의 타일을 점유하지 못함' 같은 지식을 표현하지 않았습니다.
- 이번 챕터에서 이런 지식들에 관해서 공부하겠습니다.

2. Knowledge Base(지식의 집합) : KB

- = 개발자가 문제를 해결하기 위해 문제에서 처음에 주어진 지식(명제) + 문제를 해결해 가며 얻은 지식(명제)
 - 개발자는 문제를 해결할 때 KB를 위배하면 안 됩니다. -> KB를 True로 만족시키는 Logic을 찾아야합니다.
- ex) (1,2)에 플레이어가 존재합니다.

3. Inference (추론)

- = KB에 저장된 문장들로부터 추론하여 새로운 문장을 도출합니다.
- 개인적인 생각 : KB에 엄청난 양의 지식과 데이터를 sentence로 저장하고 이를 컴퓨터에게 학습 시켜서 새로운 지식을 스스로 추론하도록 하는 것이 머신러닝인 것 같습니다. 사람은 어마어마한 데이터와 지식을 다룰 역량이 부족하므로 사람은 컴퓨터가 도출한 새로운 지식을 만들 수 없기에 가치 있는 학문인 것 같습니다.

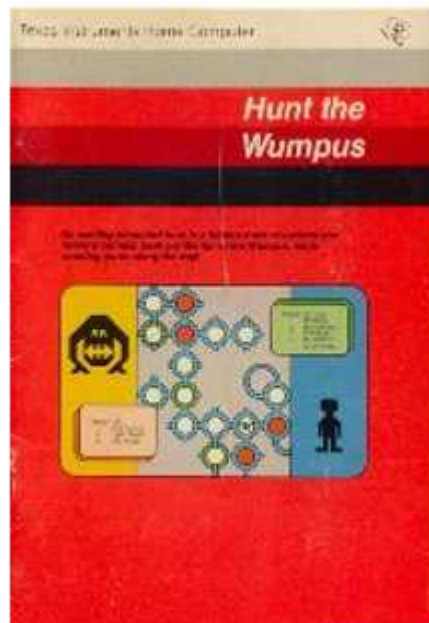
<과정>

- 1) Tell : Agent가 받은 Percept를 문장으로 변화시키고 KB에 문장을 추가합니다.
- 2) Ask : KB에 질문을 하면 KB가 새로운 action을 취하라고 알려줍니다.
- 3) Tell : Agent가 Ask의 해답으로 액션을 취하면, 해당 내용을 KB에 문장으로 추가합니다.

4. Declarative approach

- 사용자가 Tell 함수와 Ask 함수를 만들거나 사용.

<7-2강_Knowledge-Based Agents Example : The Wumpus World> 12/04



1. The Wumpus World

- = Agent가 시작점에서 출발하여 괴물과(Wumpus) 함정(PIT)을 피해서 금을 찾는 게임입니다.
- 함정의 사방에는 바람(breeze)이 있습니다.
- 괴물의 사방에는 악취(stench)가 납니다.
- 이 게임은 transition model이 incomplete 합니다. 다시 말해 어떤 칸에 가보지 않고 알 수 없는 경우가 존재합니다.
- 게임을 잘 하기 위해서는 **logical reasoning**이 필요합니다.

2. Task Environment

1) Performance measure

- 금 : 1000
- 괴물 , 함정 : -1000
- 행동 : -1
- 활로 괴물 쏘기 : -10

2) Environment

- 게임 필드 : 4 X 4
- 금과 괴물은 각각 1개
- 금과 괴물 그리고 함정은 시작점에 존재하지 않습니다.
- 함정은 15칸 중에서 3 군데에 존재합니다.

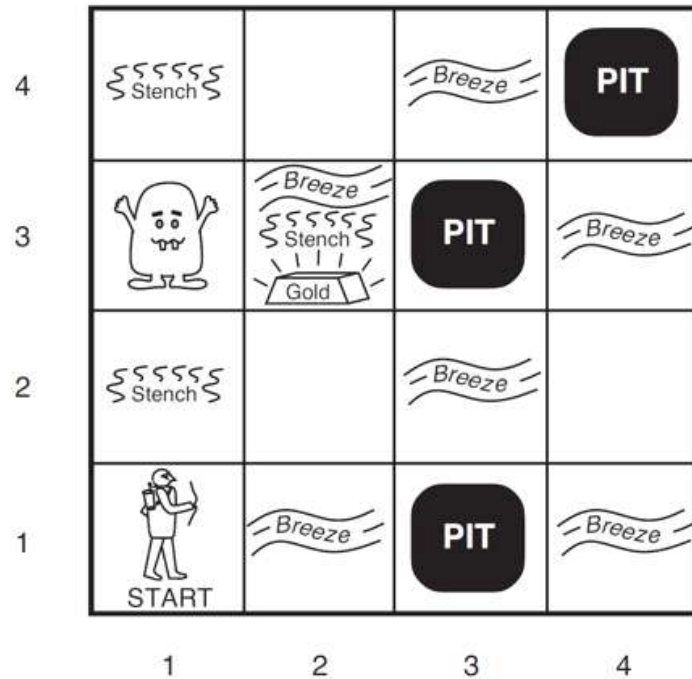
3) Actuator

- 이동
- 금 줍기
- 바라보는 방향으로 화살 쏘기

4) Sensor

- 악취 파악
- 바람 파악
- 금이 있는 방을 가면 금의 존재를 알 수 있습니다.
- 벽으로 이동하면 갈 수 없음을 압니다.
- 화살이 괴물에게 적중하면 괴물은 죽습니다.

3. Logical Reasoning을 고려한 게임 진행



<Logical Reasoning>

= 주어진 지식을 통해 Agent가 추론을 하여 알 수 있는 지식
- 아래의 진행 과정에서는 무수히 많은 Logical Reasoning을 수행합니다.

- 1) Agent는 [1,1]에서 시작하고 , [1,1]은 현재 안전합니다. [1,1]에 약취나 바람이 없으므로 Agent는 [1,2]와 [2,1]이 안전하다는 것을 추론할 수 있습니다.
- 2) Agent가 [2,1]로 이동합니다. [2,1]에는 바람이 존재합니다. 그러므로 [2,2] OR [3,1]에는 함정이 있을 것이므로 둘 다 안전함을 보장 할 수 없습니다.
- 3) Agent는 [1,1]로 돌아간 후 [1,2]로 이동합니다. 2가지 logical reasoning을 하게 됩니다.
 - ① [1,2]에서 약취가 나기 때문에 [1,3] OR [2,2]에 괴물이 있음을 추론할 수 있습니다. 하지만 [2,2]에 괴물이 있다면 [2,1]에서도 약취가 나야하므로 [2,2]에는 괴물이 없고 [1,3]에 괴물이 있음을 알 수 있습니다.
 - ② [2,2]에 함정이 있다면 [1,2]에서 바람이 불어야 하므로 [2,2]는 함정이 없음을 확신하고 , [2,2]로 이동합니다. [2,2]에 함정이 없으므로 [3,1]에 함정이 있다는 사실도 알 수 있습니다.
- 4) 이런 과정을 통해 금을 찾아 낼 수 있습니다.

- Logical Reasoning 또한 문장으로 표현됩니다.

- 프로그램이 게임을 진행하는 방식이 실제 사람이 진행하는 방식과 매우 유사하며 우리는 프로그램이 어떻게 이를 구현하는지 7-3강에서 배우게 됩니다.

<이산수학 : 명제> 12/20

1. 명제

= True / False를 선언하는 문장

- 명제와 문장을 엄밀하게 구분하면 다르게 생각할 수 있지만 학부생 수준에서는 같게 봐도 무방하다고 생각이 됩니다. 결국, 명제도 하나의 문장으로 구성되어 있기 때문입니다. 앞으로 문장과 명제에 대해서 크게 구분하지 않겠습니다.
- 문장(명제)을 단순 명제(P) OR 복합 명제($P \rightarrow Q$)로 구분할 수 있습니다. 결국, 명제 기호 1개도 하나의 명제가 될 수 있습니다. 그러므로 명제와 명제 기호에 대해서도 크게 구분하지 않겠습니다.

<Example_1 : 명제>

- P : 마이클의 PC의 운영체제는 리눅스입니다.
- 단순한 문장이지만 명제입니다.

<Example_2 : 명제>

- P : 나는 기말고사에서 100점을 맞았습니다.
- Q : 기말고사에서 100점을 맞은 학생은 A학점이 부여됩니다.
- $P \rightarrow Q$: 내가 기말고사에서 100점을 맞으면 나는 A학점을 받습니다.
- 위의 3문장 모두 명제입니다.

2. 영어 단어 공부

1) Proposition : 명제

2) Negation : 부정

3) Conjunction

= 논리곱

= P And Q

- And 연산은 'TTT' : 두 명제

<Example>

- P : 나는 오늘 점심으로 샐러드를 먹을 계획입니다.

- Q : 나는 오늘 저녁으로 스테이크를 먹을 계획입니다.

- P AND Q : 나는 오늘 점심으로 샐러드를 먹고 , 저녁으로 스테이크를 반드시 모두 먹어야 P AND Q가 True가 됩니다.

4) Disjunction

= 논리합

= P OR Q

- OR 연산은 'FFF'

<Example>

- P : 나는 오늘 점심으로 샐러드를 먹을 계획입니다.

- Q : 나는 오늘 저녁으로 스테이크를 먹을 계획입니다.

- P AND Q : 나는 오늘 점심으로 샐러드를 먹거나 저녁으로 스테이크를 먹어야 P OR Q가 True가 됩니다. 다시 말해 , 두 번의 식사에서 한 번이라도 계획대로 식사를 하면 True가 됩니다.

5) Premise : 전제

6) Conclusion : 결과

7) Implication

= 함축

= $P \rightarrow Q$

= 전제가 True일 때 결과가 True면 Implication은 True고 , 결과가 False면 Implication은 False입니다.

= 전제가 False일 때 결과에 상관없이 Implication은 True 입니다.

<Example>

- P : 시험에서 100점을 맞았습니다.

- Q : 100점을 맞은 사람의 학점은 A입니다.

- 시험에서 100점을 맞았고 , 학점이 A면 True입니다.

- 시험에서 100점을 맞았는데 , 학점이 A가 아니면 False입니다.

- 시험에서 100점을 맞지 않은 경우 , 학점이 뭐가 나오던 True입니다. (이해가 가지 않음)

[표 3-7] 조건명제 진리표

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

8) Biconditional

= 상호 조건

= $P \leftrightarrow Q$

= 두 명제가 모두 True거나 모두 False면 True입니다.

- 역이 성립합니다.

3. Implication vs Entail

- = Implication은 P가 True면 Q가 True를 의미를 내포하고 , Entail은 P가 true면 Q가 True이기 때문에 동일한 개념으로 생각해도 무방합니다.
- Entail은 조금 더 엄격한 Implication 입니다.

<https://www.quora.com/What-is-the-difference-between-%E2%80%9Cimplies%E2%80%9D-and-%E2%80%9Centails%E2%80%9D-in-logic>

<7-3강_Logic> 12/05 ~ 12/06

1. Logic의 정의

= Logical reasoning을 할 수 있는 agent를 구현하기 위해 사용하는 방법 : Logic(논리)

ex) Propositional Logic(명제 논리)

2. Logic의 구성요소

1) Syntax(문법)

= 문장이므로 문법이 필요합니다.

2) Semantics

= 어떤 World(=Model)에서 해당 문장의 참/거짓을 판별합니다.

<Semantics 표현>

- 모델을 m , 문장을 a 라고 합시다.
- m satisfies a : 문장 a 가 True
- m is a model of a : 문장 a 가 True
- $M(a)$: 문장 a 를 만족시키는 모델의 집합

3. Logical Entailment

- Entail : 파이를 90도 회전 시킨 기호

$\alpha \models \beta$: 문장_a가 True면 문장_B도 참이 됩니다. 다시 말해 문장_B가 문장_a를 entail(수반하다) 합니다.

$\alpha \models \beta$ if and only if $M(\alpha) \subseteq M(\beta)$

<Example_1 : $x = 0$ entails $x*y = 0$ >

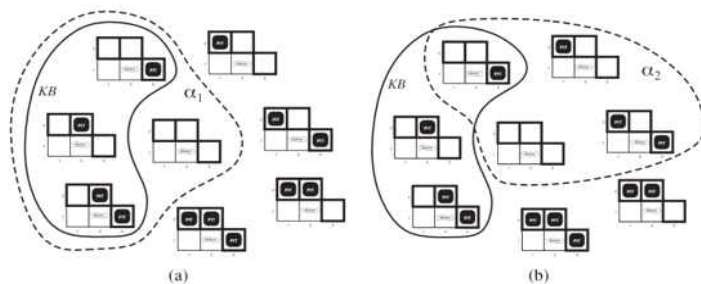
- $x = 0$ 이 True면 $x * y = 0$ 이 True입니다. 역은 성립하지 않습니다.
- $x = 0$ 인 모델 보다 $x*y = 0$ 이 더 크기 때문에 , ' $x=0$ ' $\subset$ ' $x*y = 0$ '이 성립합니다.

<Example_2 : Wumpus World>

① KB

- [1,1]은 아무것도 없습니다.
- [2,1]은 바람이 불니다.

② 추론한 문장의 Entail 검사



- 문장_a : [1,2]에 함정이 없는가?
- 문장_b : [2,2]에 함정이 없는가?
- a가 KB를 entail 하는지(a는 KB가 참이면 참인지) , b가 KB를 entail 하는지(b는 KB가 참이면 참인지) 검사합니다.
- $M(KB)$ 는 3개가 있고 , $M(a)$ 는 4개가 있는데 $M(a)$ 에 $M(KB)$ 가 모두 포함됩니다.
- $M(KB)$ 는 3개가 있고 , $M(b)$ 는 4개가 있지만 $M(a)$ 에 $M(KB)$ 가 모두 포함되지 않습니다.

$KB \models \alpha_1$ (O) $KB \models \alpha_2$ (X)

4. Logical Inference

= Logical entailment를 확인하는 방법

1) Logical Inference_1 : model checking

- Brute Force와 비슷합니다.
- '7.4강 3.'에서 자세하게 배웁니다.

2) Logical Inference_2 : Theorem Proving

- 명제 이론 + 수학으로 해결합니다.
- '7-4강 6.'에서 자세하게 배웁니다.

<Example : KB entails a1을 어떻게 프로그램으로 알 수 있을까>

$KB \models \alpha_1$ (O)

$KB \models \alpha_2$ (X)

- 개발자가 문제를 푸는 과정에서 처음에 주어진 지식과 , 문제를 해결 하는 과정에서 얻은 지식들을 합쳐서 KB라고 하고 , KB에 저장된 지식은 모두 True입니다. KB에 저장된 지식들이 True임을 이용해서 Sentence_a1 이나 Sentence_a2가 True 인지를 파악합니다.
- a1이 True라면 KB entail a1이라고 합니다. a2가 False 라면 KB not entails a2라고 합니다. 이 과정 전부를 Logical Inference라고 합니다.

5. Logical Inference의 특성

- Logical inference Algorithm은 Sound 하지 않지만 Complete 합니다.

<7-4강_Syntax of Propositional Logic> 12/11 ~ 12/12

1. BNF(문법)

1) Sentence

= Atomic Sentence | Complex Sentence

2) Atomic Sentence (단순 문장)

= True | False | P | Q | R | W1,3(=Wumpus가 (1,3)에 존재함)

- True ~ W1,3을 모두 Propositional Symbols(명제 기호)라고 부릅니다.

- True , False는 특수하게 이름부터 참과 거짓을 판별할 수 있고 , P ~ W1,3은 참과 거짓을 구해야 합니다.

3) Complex Sentence (복합 문장)

= 두 개 이상의 sentence를 합칩니다.

<명제 기호(Propositional Symbols)들을 연결해주는 5가지 접속사(Connectives)>

$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- 1변 기호(Negation) : not

- 2변 기호(Conjunction) : AND

- 3변 기호(Disjunction) : OR

- 4변 기호(Implication) : Premise $\rightarrow$ Conclusion

- 5변 기호(Biconditional) : equal

- 명제 기호 들을 접속사로 연결하면 하나의 명제를 이룹니다.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

2. Propositional Logic

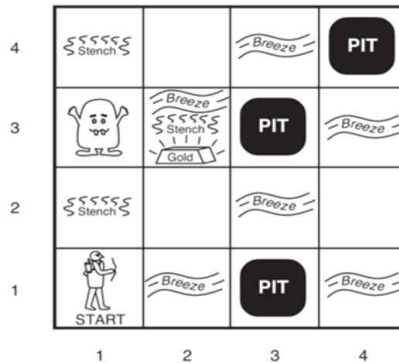
= 명제에 대해서 True / False를 정해놓은 logic

ex) $P(1,2)$, $P(2,2)$, $P(3,1)$ 가 존재하는 세상에서 ' $p(1,2) = \text{True}$ / $P(2,2) = \text{False}$ / $P(3,1) = \text{True}$ '를 정해놓습니다.

- True는 무조건 true
- False는 무조건 false
- Propositional symbol의 참 거짓은 propositional logic에서 참 거짓을 정해져 놓았기 때문에 판별 가능합니다.

3. Logical Inference_1 : Model Checking

<Example : Wumpus World>



1) 기호의 의미

- $P(x,y)$: 함정의 유무
- $W(x,y)$: Wumpus의 유무
- $B(x,y)$: 바람의 유무
- $S(x,y)$: 악취의 유무

- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- $R_4: \neg B_{1,1}$
- $R_5: B_{2,1}$

- R : Rule
- R1 : (1,1)에는 함정이 없습니다.
- R2 : '(1,1)에 바람이 분다.'의 의미는 '(1,2)에 함정이 있음' OR '(2,1)에 함정이 있음'과 동일하게 해석할 수 있습니다.
- R3 : '(2,1)에 바람이 분다.'의 의미는 '(1,1)에 함정이 있음' OR '(2,2)에 함정이 있음' OR '(3,1)에 함정이 있음'과 동일하게 해석할 수 있습니다.

- R4 : (1,1)에는 바람이 불지 않습니다.
- R5 : (2,1)에는 바람이 불니다.

2) [1,2]에 함정이 있는가?

① Query Sentence

• Is $\neg P_{1,2}$ true?

- Wumpus 문제를 해결할 때 개발자가 궁금해 하는 문제를 Propositional Logic으로 변환하면 위의 식으로 나타낼 수 있습니다.

② KB (Knowledge Base)

KB

- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- $R_4: \neg B_{1,1}$
- $R_5: B_{2,1}$

③ 문제해결 방식 : Model Checking

= 가능한 모든 모델을 검사하는 방식. 검사과정에서 Truth Table을 많이 사용합니다.

- Model checking algorithm : $M(KB)$ 가 $M(a1)$ 의 부분집합일 때 $a1$ 을 만족하는 집합 안에 KB를 만족하는 집합이 있어야 합니다.

- 가능한 모델의 개수 : KB의 5개 Rule과 Query Sentence에 존재하는 모든 propositional symbols는 7개이므로 $2^7(=128)$ 개

④ Truth Table (진리표)

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	true
false	true	false	false	false	true	false	true	true	true	true	true	true
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

- Truth Table을 통해 True인 KB를 찾아서 Query Sentence의 entail(=KB entails Query)을 확인하여 Query Sentence의 T/F를 판별합니다.
- 분류 관련 행을 제외하고 128행 & 13열로 이루어져 있습니다.
- 좌측 7개열 : Propositional Symbol
- 우측 6개열 : 5개의 Rule + Knowledge Base
- 생략한 부분이 있지만 가능한 모든 경우를 고려하면 'All False' ~ 'All True' 까지 총 128행이 존재합니다.
- 128가지의 경우에서 각각 7개의 Propositional Symbol을 모두 검사해서 얻은 KB에 존재하는 5개의 Rule에 대한 결과가 모두 True면 KB가 True가 되며 3가지 경우가 존재합니다.
- KB가 True인 3가지 모델에 대해서 Query Sentence가 참인지 검사합니다.
- Query Sentence가 P1,2의 not 연산입니다. 3가지 모델에서 P(1,2)가 모두 False 이므로 not 연산을 수행하면 True가 됩니다. 그 결과 , 3가지 모델은 모두 True를 만족함을 알 수 있고 KB entails P(1,2)를 만족하기 때문에 Query Sentence는 True가 됩니다.

⑤ Sound , Complete , Complexity

- Sound?
 - Yes, because it directly implements the definition of logical entailment
- Complete?
 - Yes, because there are a finite number of models
 - N.B., the number of propositional symbols is finite in propositional logic
- Complexities?
 - n : # of propositional symbols
 - Space complexity is not bad: $O(n)$
 - Time complexity is not practical: $O(2^n)$

4. Logical Equivalence

= 교환법칙 , 결합법칙 , 역 , 이 , 대우 , 드 모르간의 법칙 같은 수학적 성질

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

- 쉬운 공식을 설명하지 않겠습니다.

① Contraposition

= 대우

② Implication elimination

= 진리표를 계산해보면 같은 결과가 나옵니다.

③ Biconditional elimination

= 쌍방향 Implication입니다.

= 진리표를 계산해보면 같은 결과가 나옵니다.

5. 추가 개념

1) Validity

= 어떤 sentence가 valid하면 그 sentence는 모든 모델에 대해서 True를 만족합니다.

ex) $P \text{ or not } P$ 는 언제나 모든 모델에 대해서 참입니다.

2) Satisfiability

= 어떤 sentence가 satisfiable하면 그 sentence는 어떤 모델에 대해서 True를 만족합니다. 다시 말해, 어떤 sentence가 satisfiable 하지 않으면 어떤 모델에 대해서도 True를 만족하지 않음을 의미합니다.

ex) $R1 \text{ AND } R2 \text{ AND } R3 \text{ AND } R4 \text{ AND } R5 = \text{satisfiable}$

ex) $P \text{ AND not } P$ 는 satisfiable 하지 않습니다.

<SAT problem>

= 입력으로 sentence를 주고 satisfiable한지 물어보는 문제

- SAT은 NP-complete 문제입니다.

3) Proof by refutation

= 모순에 의한 증명

$\alpha \models \beta$ 를 증명하기 위해, $\alpha \wedge \neg\beta$ 가 unsatisfiable임을 증명함.

- $a \text{ entails } b$ 와 $a \text{ and not } b$ 가 unsatisfiable 하다는 동일합니다.

- Logical equivalence로 증명이 가능합니다.

6. Logical Inference_2 : Theorem Proving

= Model Checking은 모든 모델을 검사하기 때문에 복잡도가 매우 크기에 느립니다.

- Model checking처럼 모든 모델을 검사하지 않고, 다양한 Inference Rules를 적용해서 유도가 되는지 검사합니다.

1) Inference Rules

- 가운데에 Bar를 하나 그립니다.

- Top Sentence : Bar 기준 위쪽에 적은 Sentence들의 집합, 입력 받은 Sentence들의 집합입니다.

- Bottom Sentence : Bar 기준 아래쪽에 적은 Sentence들의 집합

- Top Sentence들이 모두(AND) True 라면, Bottom Sentence들이 모두 True가 됩니다.

- Inference Rules는 명제들이 True일 때 성립하는 법칙이므로 Theorem Proving 과정에서 KB는 모두 True라고 결정한 후에 Inference Rules를 통해 새로운 명제를 유도해냅니다.

- 아래의 3가지 말고도 여러 가지 Inference Rules가 존재하는 것 같으며, Logical Equivalence도 Inference Rules에 포함되는 것 같습니다.

– Modus Ponens

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

– And-Elimination

$$\frac{\alpha \wedge \beta}{\alpha}$$

– Logical equivalences

• E.g.,

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

and

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

① Modus Ponens

- $a \rightarrow B$ 가 True 이고 a 가 True 면 B 도 True입니다.
- Top sentence와 Bottom sentence를 바꿔도 성립하지 않습니다. (역 성립 X)
- Modus Ponens의 경우 Top Sentence를 2개의 Sentence($a \rightarrow b$, a)로 구성합니다.
- 정식 명칭은 아니지만 편의상 2개의 Sentence를 각각 Top Sentence 구성 Sentence라고 하겠습니다.
- Implication의 진리표를 확인하면 너무 당연함을 알 수 있습니다.

② And-Elimination

- $a \text{ AND } B$ 가 True면 a 도 True입니다.
- Top sentence와 Bottom sentence를 바꿔도 성립하지 않습니다. (역 성립 X)
- Conjunction의 진리표를 확인하면 AND(TTT)이므로 너무 당연함을 알 수 있습니다.

③ Logical Equivalences (논리적 동치)

- $a \leftrightarrow B$ 가 True면 $(a \rightarrow B) \text{ AND } (B \rightarrow a)$ 도 True입니다.
- Top sentence와 Bottom sentence를 바꿔도 성립합니다. (역 성립 O)
- Biconditional elimination입니다.

2) Example : Wumpus World

- '3. Logical Inference_1 : Model Checking'에서 공부한 예제와 동일한 예제를 Logical Inference_2인 Theorem Proving으로 해결해봅시다.
- '4. 와 6.'을 참조해서 해석합시다.

• $KB \models \neg P_{1,2}$?

• KB

- $R_1: \neg P_{1,1}$
- $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- $R_4: \neg B_{1,1}$
- $R_5: B_{2,1}$

• Proof

- $R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- $R_7: ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- $R_8: (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$
- $R_9: \neg(P_{1,2} \vee P_{2,1})$ (applying Modus Ponens to R_4 and R_8)
- $R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$
- $R_{11}: \neg P_{1,2}$ (the goal sentence)

- Model Checking에서와 동일한 Query Sentence , KB를 갖습니다.

- KB의 5개 Rule을 이용해서 Logical Equivalence 와 inference rule을 적용해봅시다.

- R6 : R2에 Logical Equivalence의 biconditional elimination을 적용

- R7 : R6에 And-Elimination을 적용

- R8 : R7에 contraposition(대우)을 적용

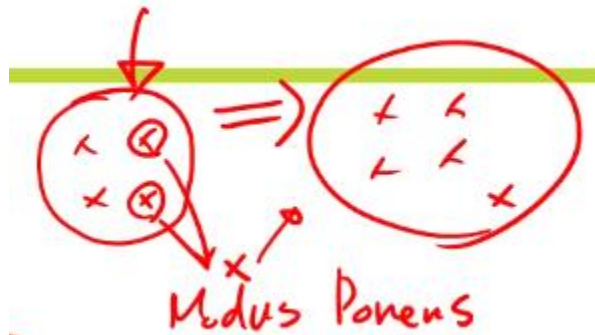
- R9 : R4와 R8에 Modus Ponens를 적용

- R10 : R9에 드 모르간의 법칙을 적용

- R11 : R10에 And-Elimination을 적용

3) Define Problem

- ① **States : Sentences**
- ② Initial State : 최초에 Knowledge Base에 저장되어 있는 Sentences
- ③ Actions : Inference Rules
- ④ Transition model : States에 존재하는 여러 가지 Sentence에 Inference Rules를 적용하여 만든 새로운 Sentence를 추가한 States.



- Transition model : 기존의 States가 Actions로 인해 변하여 새로운 States가 됩니다.

- ⑤ Goal test : Current States가 goal sentence를 포함하고 있는지를 평가합니다.
- 이렇게 만든 문제는 지금까지 배운 모든 Search Algorithm으로 해결할 수 있습니다.

<7-5강_The Resolution Inference Rule> 12/12

1. Overview

- 앞에서 배운 Theorem Proving은 sound 하지만 세상에 존재하는 inference rules가 너무 많기 때문에 complete함을 증명하는 것이 매우 어렵습니다.
- 이번 강의에서는 complete 함을 증명하기 위해 단 하나의 inference rule만 사용한 theorem proving을 배웁니다.

<Resolution>

- = 단 하나의 inference rule을 사용해서 Theorem Proving을 하고 , 그 inference rule을 Resoulution 이라고 합니다.
- OR(FFF)를 이용한 Inference Rule입니다. 다시 말해 , P OR Q에서 하나의 명제라도 True면 P OR Q는 True인 성질을 이용합니다.

2. The Resolution Inference Rule

- Resolution도 Inference Rule이기 때문에 Top에 존재하는 모든 Sentence가 True 라면 Bottom Sentence가 True인 성질은 동일합니다.

1) Unit resolution rule (간단한 형태)

Unit resolution rule

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

- Unit resolution rule에 나오는 모든 propositional symbols($\ell_1 \sim \ell_k$, m)는 모두 literal입니다.
- Top Sentence($\ell_1 \sim \ell_k$ 의 OR Sentence 와 m)가 모두 True $\Rightarrow$ Bottom Sentence(ℓ_i 를 제외한 $\ell_1 \sim \ell_k$ 의 OR Sentence)도 True입니다.
- ℓ_i 와 m 은 반드시 Complementary literals(P와 not P) 관계여야 합니다.

2) Full resolution rule (복잡한 형태)

Full resolution rule

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

- Full resolution rule에 나오는 모든 propositional symbols($L_1 \sim L_k$, $M_1 \sim M_n$)은 모두 literal입니다.
- Top Sentence($L_1 \sim L_k$ 의 OR Sentence와 $M_1 \sim M_n$ 의 OR Sentence)가 모두 True $\Rightarrow$ Bottom Sentence(L_i , M_j 를 제외한 $L_1 \sim M_n$ 의 OR Sentence)는 True입니다.
- L_i 와 m_j 는 반드시 Complementary literals(P 와 $\neg P$) 관계여야 합니다.

<Example : Full resolution rule>

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

- 첫 번째 Top Sentence에서 $P(1,1)$ 와 두 번째 Top Sentence에서 $\neg P(1,1)$ 가 Complementary literals 관계고 L_i , M_j 로 해석할 수 있습니다.
- Bottom Sentence 연산에서 $P(1,1)$ 와 $\neg P(1,1)$ 을 제외하고 OR 연산을 수행합니다.
- $P(1,1)$ 이 True 라면 $\neg P(1,1)$ 은 반드시 False 입니다. Top Sentence는 모두 True여야 하므로 $P(3,1)$ 은 True OR False 중에 아무거나 선택 해도 무방하나, $\neg P(2,2)$ 은 필연적으로 True 여야 합니다. 그러므로 $P(3,1)$ OR $\neg P(2,2)$ 는 필연적으로 True를 만족합니다.

3. Resolution Algorithm을 알기위한 개념

1) Clauses (절)

= Disjunction of literals

= Literal 들의 OR 연산

$$P_{1,2} \vee \neg P_{2,1}$$

2) Conjunctive normal form (CNF)

= Conjunction of clauses

= Clauses 들의 And 연산

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

- Unit resolution rule 과 Full resolution rule의 Top Sentence 구성 Sentence는 각각 OR 연산을 하기 때문에 Clauses입니다.
- Top Sentence 구성 Sentence들은 모두 clauses고 , 그들끼리 And 연산을 하여 Top Sentence를 구성하기 때문에 Top sentence는 CNF입니다.
- Modus Ponens 같이 앞에서 배운 다양한 inference rules 들은 반드시 Top Sentence가 CNF일 필요가 없습니다. 그러나 Resolution은 반드시 Top Sentence가 CNF가 되어야 합니다. 모든 propositional logic은 CNF로 변화할 수 있기 때문에 이는 전혀 문제가 되지 않습니다.

4. CNF로 변환하기

<Example : 아래의 Sentence를 CNF로 변화시켜 봅시다.>

- Logical Equivalence를 사용해서 4단계로 변화시킵니다.

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) .$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}) .$$

3. CNF requires $\neg$ to appear only in literals, so we “move $\neg$ inwards” by repeated application of the following equivalences from Figure 7.11:

$$\neg(\neg\alpha) \equiv \alpha \quad (\text{double-negation elimination})$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad (\text{De Morgan})$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad (\text{De Morgan})$$

In the example, we require just one application of the last rule:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}) .$$

4. Now we have a sentence containing nested $\wedge$ and $\vee$ operators applied to literals. We apply the distributivity law from Figure 7.11, distributing $\vee$ over $\wedge$ wherever possible.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

5. Resolution Algorithm

- 'KB entails 알파'를 증명하기 위해 'KB AND not 알파'가 satisfiable하지 않음을 보이면 됩니다.

1) KB AND not 알파를 CNF로 변환시킵니다.

2) clauses에 대해서 resolution rule을 적용해서 새로운 clauses를 만듭니다. 이를 반복합니다.

<Example : KB와 알파가 다음과 같은 매우 간단한 문제에서 Resolution Algorithm을 통해 해결해봅시다.>

KB

- $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

- $R_4: \neg B_{1,1}$

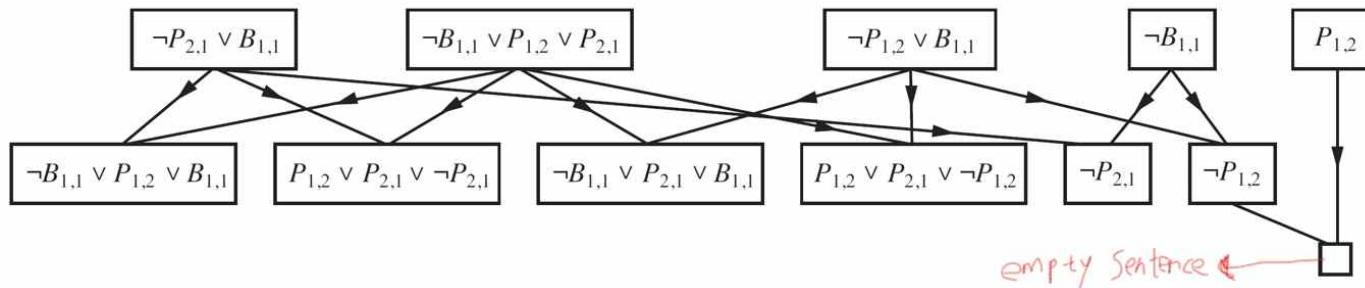
α

- $\neg P_{1,2}$

- R2 : '(1,1)에 바람이 분다.'의 의미는 (1,2) OR (2,1)에 함정이 없음과 동일합니다.

- R4 : R2에서의 정보를 알았지만 Wumpus World를 진행하다보니 (1,1)에는 바람이 불지 않음을 알아냈습니다.

- 알파 : (1,2)에 함정이 없습니까?



- R2와 R4에서 얻은 sentence와 그 sentence들을 CNF 형태로 변환 시키고 resolution inference를 반복하여 알파를 도출해내는 그림입니다.

- 반복 결과 empty sentence가 나오거나, 새로운 sentence가 더 이상 나오지 않으면 entail 하지 않음이 정답입니다.

- Resolution Algorithm은 completeness 합니다. 이에 대한 증명은 너무 어렵기 때문에 넘어가셨습니다.